

所属类别	2022 年“华数杯”全国大学生数学建模竞赛	参赛编号
本科组		CM2200023

基于统计模型、集成树模型和改良遗传算法的材料性能研究 摘要

针对问题一，由于样本无法通过 Kolmogorov-Smirnov 正态性检验，因此先利用非参数方法中的 **Wilcoxon 符号秩检验**对插层前后的样本进行分析。结果表明，结构变量和产品性能的六个特征在插层前后变化极其显著。然后，利用 **Friedman 秩和检验**，对不同插层率水平下的数据进行检验，检验结果表明不同插层率下的差异并不显著。最后，为避免假阳性，利用 **Nemenyi 检验**对数据两两比较，发现插层率的变化并不会对结构变量和产品性能造成较大的影响，只能显著降低孔隙率。

针对问题二，首先添加类似 x^2, x^3, x_1x_2 的非线性数据特征。接着，我们将特征的关系分为整体和个体。对于工艺参数和结构变量两组变量整体间的关系，利用典型相关分析分析组间的关系。对于两组变量中共五个特征之间的关系，不仅使用经典统计的多元线性标准化逐步回归，以标准化系数作为指标判断变量之间的线性关系；还使用可解释性较好的**随机森林、AdaBoost、GBDT、XGBoost、LightGBM 和 CatBoost**，利用**特征重要性**观察变量之间的非线性关系。预测 x_1, x_2, x_3, x_4, x_5 时，拟合性能最好的模型分别为 XGB, XGB, Cat, Cat 和 XGB, R^2 分别为 **0.9999, 0.9999, 0.9999, 0.9293, 0.9750**。结果表明，以 x_1, x_3 的线性影响最大， x_3, x_4, x_5 三者之间的交互作用的非线性影响最大； x_2 为因变量时， x_3 的线性影响最大， x_3, x_5 的非线性影响最大；以 x_3, x_4 为因变量时， x_1 和 x_2 的线性影响最大， x_1, x_2 的交互作用的非线性影响最大；以 x_5 为因变量时， x_1^2 和 x_2^2 的线性影响最大，所有特征的非线性影响较为平均。

针对问题三，首先研究结构变量与产品性能的关系。先进行典型相关分析，再对个体分析。预测 $x_3, x_4, x_5, x_6, x_7, x_8$ 最优的模型中，只有 x_3 为 GBDT，其余均为 XGB，最优的 R^2 都高于 **0.9999**。预测 x_3 和 x_4 时， x_7 和 x_7, x_8 的交互作用的线性影响最大， x_6, x_7, x_8 的交互作用的非线性影响最大；预测 x_5 时， x_6 和 x_6, x_8 的交互作用的线性影响最大， x_6, x_7, x_8 的交互作用的非线性影响最大；预测 x_6 时， x_3 和 x_3, x_5 的交互作用的线性影响最大，预测 x_7 时， x_3, x_5 本身及其交互作用的线性影响最大，预测 x_8 时， x_3, x_3^3 和 x_3, x_5 的交互作用的线性影响最大，对三者的非线性关系影响最大的都是 x_3, x_4, x_5 的交互作用。其次，研究结构变量之间和产品性能之间的关系，预测 x_3 时， x_4 的线性和非线性影响最大预测 x_4 时， x_3 的线性影响最大， x_3 和 x_5 的交互作用的非线性影响最大预测 x_5 时， x_3, x_3 和 x_4 的交互作用的线性、非线性影响最大；预测 x_6 时， x_7 的线性影响最大， x_7 和 x_8 的交互作用的非线性影响最大；预测 x_7 时， x_8 的线性、非线性影响最大；预测 x_8 时， x_7 的线性、非线性影响最大。最后，以最大化过滤效率作为目标函数，将结构变量和工艺参数非负作为约束条件，以使用**增强精英保留改进的遗传算法(SEGA)**进行单目标寻优。在目标函数的选择上，分别使用拟合性能更好、表达非线性关系的机器学习模型，和拟合能力较弱、表达线性关系的回归方程进行寻优。使用机器学习时的最大过滤效率为 83.15%，此时的接受距离和热风速度分别为 0.29cm 和 8085.94r/min；使用回归方程时的最大过滤效率仅为 48.82%，但是接受距离和热风速度的值都为 0.0001。这说明变量之间的非线性关系更加显著，且最终结果为 **83.15%**。

针对问题四，我们同样选用进化算法，且是进化算法中最先进的**基于参考点改进的非支配排序遗传算法(NSGA-III)**。使用机器学习时，最大的过滤效率为 **83.17%**，最小的过滤阻力为 **27.92Pa**。由于存在多个种群个体都寻到最优解，接受距离处在区间 [11.32, 18.28] 内，均值为 13.41，而热风速度处在 [1192.32, 1964.71] 之间，均值为 1666.05。
关键词：非线性特征衍生；相关与回归；集成树模型；SEGA；NSGA-III

一、问题重述

1.1 问题背景

随着社会生产力的快速提高和全球对环境保护重视程度的加深，以及新型冠状病毒等全球安全突发性事件的诱导，国内外医用口罩的需求急剧增加^{错误!未找到引用源。}。插层熔喷非织造材料是口罩生产的重要原材料，因其独特的插层技术可以插入功能性纤维使得材料性能稳定，具有孔径小、孔隙率高等特点，与其他工艺制得的非织造材料相比过滤效果更好、屏蔽性能更强^{错误!未找到引用源。}。因此，优化插层熔喷非织造材料的工艺过程势在必行，通过建立插层熔喷非织造材料的工艺模型，优化工艺参数提升材料性能，实现高通量织造过滤性能优越、产业线成本低廉的目标，为大范围工业化推广提供了可靠的理论基础。插层熔喷非织造材料技术的成熟与推广，将会极大拓宽熔喷材料的市场规模，不仅能更好的满足国家疫情防控的战略需求，还能推动相关纺织熔喷行业的快速稳定发展。

1.2 问题提出

在本文中，我们需要根据题目给出的实验数据集，构建工艺参数与结构变量之间、结构变量与产品性能之间的关系模型，分析参数关系，构建预测模型，并基于构建的模型寻找适用于工业生产的最优工艺参数。本文要解决的问题如下。

1.2.1 问题一的提出

题目在 data1 数据集已经给出了插层前后的对照实验结果，其中包括在插层前后的厚度、孔隙率、压缩回弹性这些结构变量以及过滤阻力、过滤效率、透气性这些产品性能。为了研究插层对熔喷非织造材料制备工艺的影响，我们需要根据数据集，分析在插层这一变量的影响下，结构变量、产品性能是否发生显著变化，并进一步分析插层率的大小是否会显著影响这些指标的变化。

1.2.2 问题二的提出

本题要求利用所给的 data3 数据集探究工艺参数与结构变量之间的关系，即建立接受距离、热空气速度与厚度、孔隙率及压缩回弹性之间的关系模型，实现根据已知工艺参数数据集预测结构变量的作用，并完成题中表 1 的结构变量数据预测。

1.2.3 问题三的提出

本题与问题二类似，要求我们利用所给 data3 数据集探究结构变量与产品性能之间的关系，以及结构变量之间、产品性能之间的关系，并建立关系模型。而后在第二问的基础上，得到从工艺参数到产品性能之间的关系预测模型，并且利用寻优算法求得产品过滤效率最高时的工艺参数。

1.2.4 问题四的提出

本题在问题三和问题二所求的函数的基础上，添加了针对接受距离、热风速度、厚度和压缩回弹性的约束条件，并且要求对最大化过滤效率和最小化过滤阻力双目标进行寻优。

二、问题分析

2.1 问题一的分析

针对问题一，首先要确定插层这一因素的有无是否会显著影响结构变量及产品性能的变化，假设检验是一种切实可行的方法。由于各指标数据的非正态性，需引入非参数检验中 Wilcoxon 符号秩检验对插层前后的每一指标数据进行检验，判断插层的有无是否会造成显著影响。而后为进一步分析插层率水平对指标变化的影响，通过将低中高三种插层率水平下的各指标数据进行 Friedman 秩和检验并采用 Nemenyi 检验进行两两比较，检验插层率水平与各指标变化之间的关系，评判插层率水平会对哪些指标的变化造成显著影响。

2.2 问题二的分析

针对问题二，我们对数据进行统计检验和特征衍生。为探究变量整体间的关系，我们使用典型相关分析。为探究五个特征之间的关系，我们不仅使用了经典统计的标准化逐步回归方法，还使用了可解释性较强的机器学习集成树模型。分别利用标准化回归系数和特征重要性判断变量之间的关系。最后，我们选用拟合性能最好的模型，采用十折交叉验证法对模型进行性能评估，最后对题目表 1 的数据进行预测。

2.3 问题三的分析

针对问题三，与问题二类似，我们延续第二问的模型，同样分整体和个体进行建模和求解。在得到参数之间的函数关系后，将之带入增强精英保留改进的遗传算法进行寻优，寻找到最优的工艺参数。在寻优结束后，我们使用回归方程进行二次寻优，对结果进行对比和补充。

2.4 问题四的分析

针对问题四，同样选用启发性算法进行建模。与第三问最后的寻优相同，我们在设置启发性算法中的最大化和最小化目标时，使用了拟合能力最强的集成树模型的预测方法建立算法。在寻优结束后，我们同样使用回归方程进行二次寻优，最终，最优的结果依旧是使用树模型的预测方法。

三、模型假设

为方便建模和分析，我们提出如下假设：

1. 附件提供的数据真实可信，是经过真实的实验所得的数据，且不存在人为的删改和破坏。
2. 在实验中，各组实验除了给出的特征以外，其余的所有影响因子均保持不变，实验数据都是基于控制变量法所得的有效数据。
3. 每一条实验数据来自的每一次实验都相互独立，不存在样本之间的影响。

四、符号说明

符号	说明
$x_1, x_2, \dots, x_8$	分别表示题目所给的八个特征
U, V	典型相关的变量对
R^2	回归分析的拟合优度
N	遗传算法的种群数目
$F(x_3, x_4, x_5)$	代指以 x_3, x_4, x_5 为自变量, x_7 为因变量的预测模型
$G(x_3, x_4, x_5)$	代指以 x_3, x_4, x_5 为自变量, x_6 为因变量的预测模型
$g(x_1, x_2)$	代指以 x_1, x_2 为自变量, x_3 为因变量的预测模型
$h(x_1, x_2)$	代指以 x_1, x_2 为自变量, x_4 为因变量的预测模型
$z(x_1, x_2)$	代指以 x_1, x_2 为自变量, x_5 为因变量的预测模型
Z^s	NSGA-III的参考点
Z^a	NSGA-III的期望点
P_t	NSGA-III的父代
S_t	NSGA-III的种群集合
Q_t	NSGA-III产生的新群体
R_t	NSGA-III的父代和新群体的集合

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 数据特征观测

为方便编程和论文中图标的表示,将数据中的接收距离、热风速度、厚度、孔隙率、压缩回弹性、过滤阻力、过滤效率、透气性分别标记为 $x_1, x_2, \dots, x_7, x_8$ 共八个特征。

为研究插层前后的特征变化规律,以及插层率的数值是否对变化有影响,可以使用统计模型进行计算。经典统计中的几乎所有模型的建立都依赖于原始数据或残差符合正态分布的假定,而服从正态分布的数据在回归任务中往往表现得比非正态的数据更加稳定。而且,符合正态分布的数据可以使用经典统计方法,而非正态的数据就需要数据变换或使用非参数统计的方法进行统计计算。因此,在建模之前,首先要对数据的正态性进行检验。Kolmogorov-Smirnov 检验是非参数统计的分布检验方法,原假设是数据符合给定的分布。我们利用 KS 检验对原始数据的正态性检验,结果如表 1 所示,表中所有的 P-value 值都远远小于 0.05,需要拒绝原假设,原始数据不符合正态分布。采用核密度图直观地展示数据样本的分布特征,如图 1 所示,显然,插值前后的结构变量和产品性能的各指标均不符合正态分布。

表 1 Kolmogorov-Smirnov 检验

插层因素	特征	ks 统计量	P-value
插层前	厚度	0.8078	<0.0001
	孔隙率	>0.9999	<0.0001
	压缩回弹性	>0.9999	<0.0001
	过滤阻力	>0.9999	<0.0001
	过滤效率	0.9585	<0.0001

插层后	透气性	>0.9999	<0.0001
	厚度	0.9591	<0.0001
	孔隙率	>0.9999	<0.0001
	压缩回弹性	>0.9999	<0.0001
	过滤阻力	>0.9999	<0.0001
	过滤效率	>0.9999	<0.0001
	透气性	>0.9999	<0.0001
	接受距离	>0.9999	<0.0001
	热风速度	>0.9999	<0.0001
	厚度	0.9552	<0.0001
在固定插层率下	孔隙率	>0.9999	<0.0001
	压缩回弹性	>0.9999	<0.0001
	过滤阻力	>0.9999	<0.0001
	过滤效率	>0.9999	<0.0001
	透气性	>0.9999	<0.0001

采用核密度图直观地展示插层前后数据样本的分布特征，如图 1 所示，显然，插值前后的结构变量和产品性能的各指标均不符合正态分布。

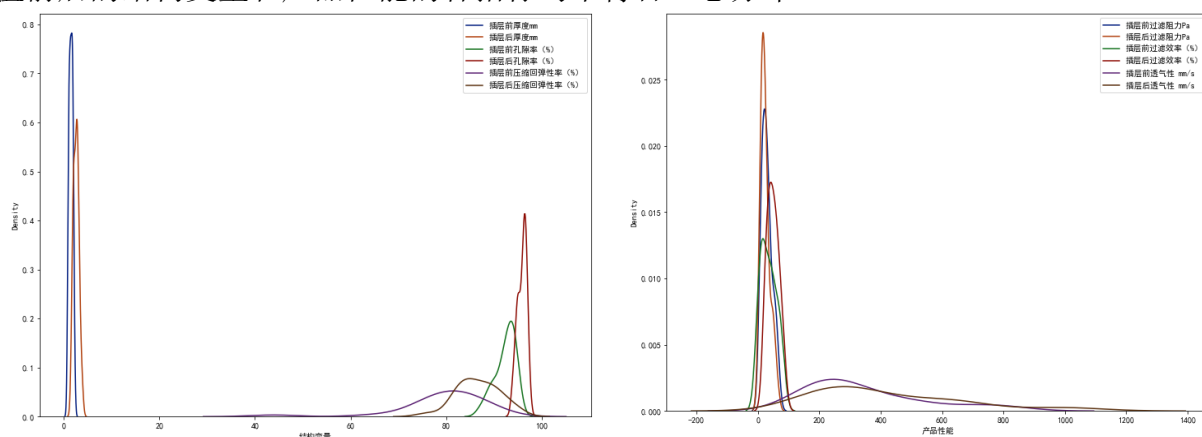


图 1 插层前后结构变量及产品性能的核密度分布图

而后，采用核密度图直观地展示在插层率固定的情况下数据样本的分布特征，如图 2 所示，显然，工艺参数、结构变量和产品性能各指标均不符合正态分布。

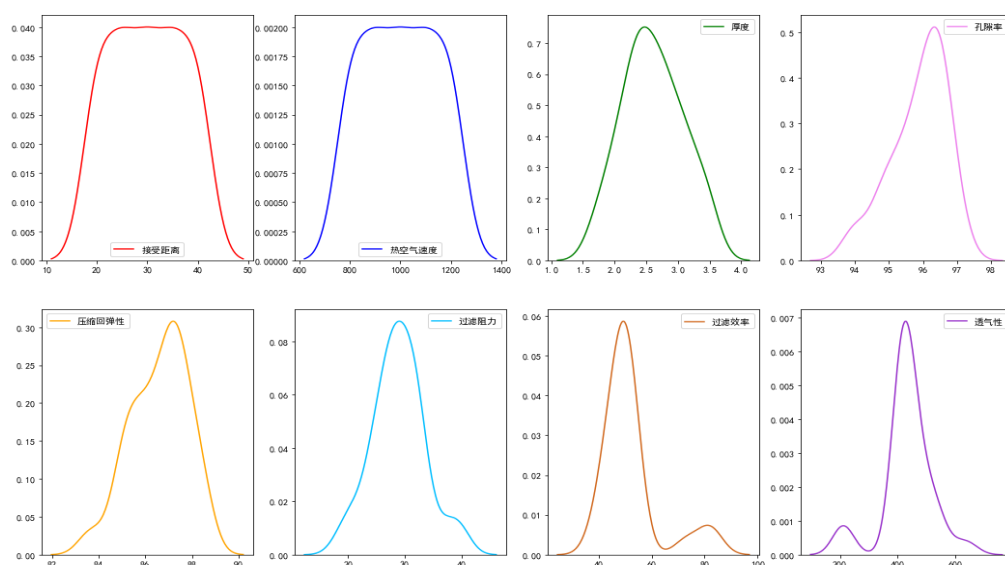


图 2 在插层率固定的情况下各指标的核密度分布图像

5.1.2 非参数检验方法——Wilcoxon、Friedman 和 Nemenyi 检验

假设检验是一种由一个样本或一系列样本所得的结果来推断总体特征的统计推断方式，它根据总体的理论分布和小概率原理，对未知的总体提出两种对立假设，由已知样本信息来判断总体参数或总体分布的假设是否可信，做出统计意义上的假设判断。在本题中，需要对总体数据的关系进行评估，但是无法得知总体数据集，仅能利用已知实验数据做出统计推断，且各指标样本均不服从正态分布，无法进行参数检验，所以为探究插层会如何对非织造材料的结构变量、产品性能造成影响，我们对插层前后各指标样本两两进行 Wilcoxon 符号秩检验。

配对样本的 Wilcoxon 符号秩检验，是一种适用于未知总体分布的非参数检验方法的步骤如下：

- 样本含量分别为 n_1, n_2 ($n_1 \leq n_2$) 的两样本数据混合从小到大排列进行编秩。
- 计算较小样本 n_1 的秩和 T 。
- 当两样本容量均大于 10 时（此时 $n_1 = n_2 = 25$ ），秩和 T 的分布接近于正态分布，计算 Z 值构建统计量，完成假设检验。

$$Z = \frac{T - \frac{n_1(n_1 + n_2 + 1)}{2}}{\sqrt{\frac{n_1 n_2 (n_1 + n_2 + 1)}{12}}} \quad (1)$$

配对样本的 Wilcoxon 符号秩检验的结果能够清晰反应出有无插层是否能显著影响结构变量和产品性能，而后为了更加深入理解插层是如何影响结构变量和产品性能的变化，我们将进一步分析插层率对于这些指标变化的影响。将插层率的按照大小分为低插层率、中插层率和高插层率 3 组（2.499%~19.35%，19.35%~24.74%，24.74%~50.87%），在不同的指标下，进行 Friedman 秩和检验并采用 Nemenyi 检验进行两两比较，步骤如下：

- 计算秩和 T 并计算统计量 M 值，确定 P 值做出统计推断。

$$M = \sum \left(T_i - \frac{\sum T_i}{k} \right)^2 \quad (2)$$

- 任意两组数据进行多重比较。设 R_i 和 R_j 分别为第 i 组和第 j 组的秩和，计算检验统计量 Z_{ij} ，做出统计推断。

$$Z_{ij} = \frac{\overline{R_i} - \overline{R_j}}{\sigma_{\overline{R_i} - \overline{R_j}}} \quad (3)$$

根据秩和检验的结果能够清晰的展示观测插层水平对于各指标变化的影响。问题一模型的建立，深入剖析了插层技术对熔喷非织造材料的意义。

5.1.3 模型求解

首先，根据 Wilcoxon 符号秩检验的结果，分析得到结构变量中厚度、孔隙率、压缩回弹性率和产品性能中过滤阻力、过滤效率和透气性在插层前后均具有极显著性差异，如下图 2 所示。由图像可以观测到各指标在插层前后均发生了不同程度的优化，这说明了插层会显著影响熔喷非织造材料的工业生产性能和产品质量，是优化工艺流

程的可能手段。

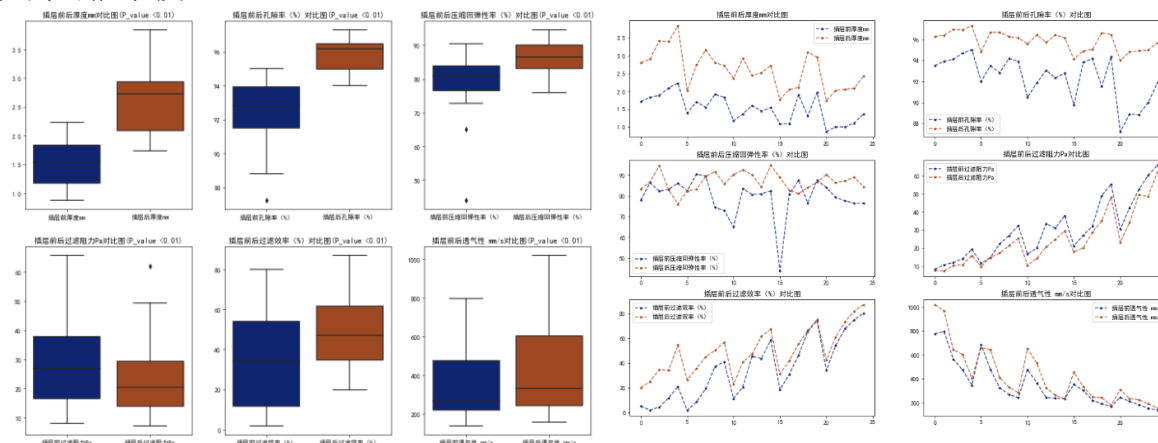


图 3 插层前后结构变量及产品性能对比图

而后，根据 **Friedman 秩和检验**，分析得到结构变量中厚度、孔隙率在不同插层率水平下具有显著性差异，而压缩回弹率及各产品性能指标均在不同插层率下无显著性差异。为了避免假阳性，采用 **Nemenyi 检验**进行两两比较，探究插层率水平两两之间是否均具有显著性差异。结果表明，厚度指标在高中低三种插层率水平两两之间均无显著差异，而孔隙率在低-中、低-高水平上具有显著性差异，但在中-高水平上不具有显著性差异。如下图 3 所示，虽然孔隙率三种插层率水平下具有明显差异，但中-高水平之间差距较小这验证了我们模型的可靠性。该结论也说明插层率水平并不会对结构变量与产品性能造成较大影响，但是较低插层率将能显著降低孔隙率，因此为了保证产品在工业生产中各指标的稳定性，插层率应保持在较高水平。

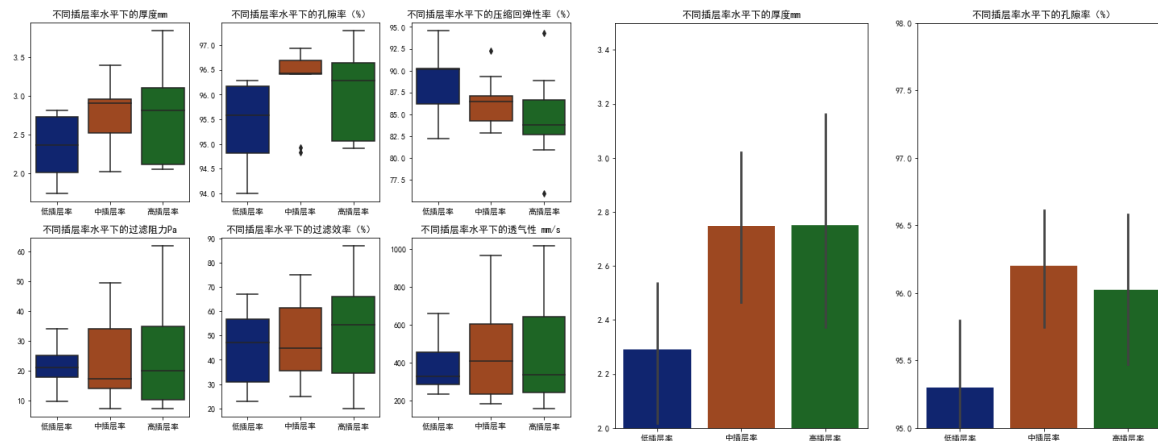


图 4 低中高三种插层率水平下结构变量和产品性能对比图

5.2 问题二模型的建立与求解

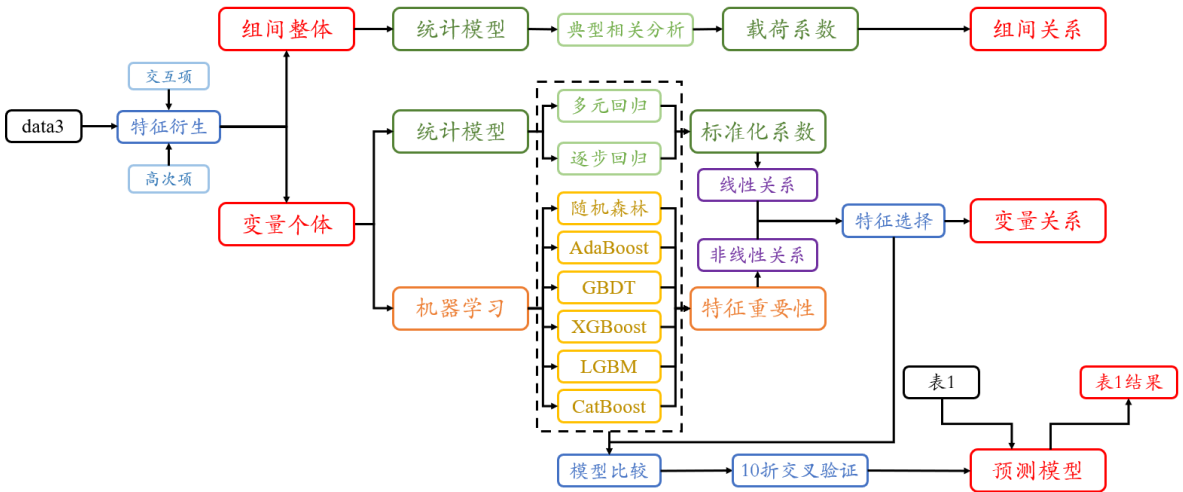


图 5 问题二的流程图

在本节中，我们先对题目附件 data3 的数据进行统计检验和特征衍生。为探究“工艺参数”和“结构变量”两组变量整体间的关系，我们使用典型相关分析，根据载荷系数等信息来进行研究。为探究“工艺参数”和“结构变量”共五个特征之间的关系，我们不仅使用了经典统计的标准化逐步回归方法，利用标准化系数只管判断变量的线性关系，还使用了可解释性较强的机器学习集成树模型，利用特征重要性来判断特征之间的非线性关系。在进行特征选择和模型的性能比较之后，我们选择最优的模型和最好的特征进行拟合，采用十折交叉验证的方法对模型进行评估，最后对题目表 1 的数据进行预测。

5.2.1 数据观测与特征衍生

对工艺参数和结构变量的五个特征，分别绘制相关图、三维散点图和散点图矩阵，如图 4 所示。根据节 5.1.1 中 K-S 检验的结论，所有的特征都不符合正态分布，因此采用 Spearman 秩相关系数来表示接受距离、热风速度、厚度、孔隙率及压缩回弹性之间的相关性。图 4(左)是五个特征的相关图，其中蓝色表示正相关，红色表示负相关，且颜色越深，饼图的颜色占据的面积越大，表示相关性越强。由相关图所知，各指标之间的相关性均较弱。三维散点图中明显的非线性关系也证明了这一点。

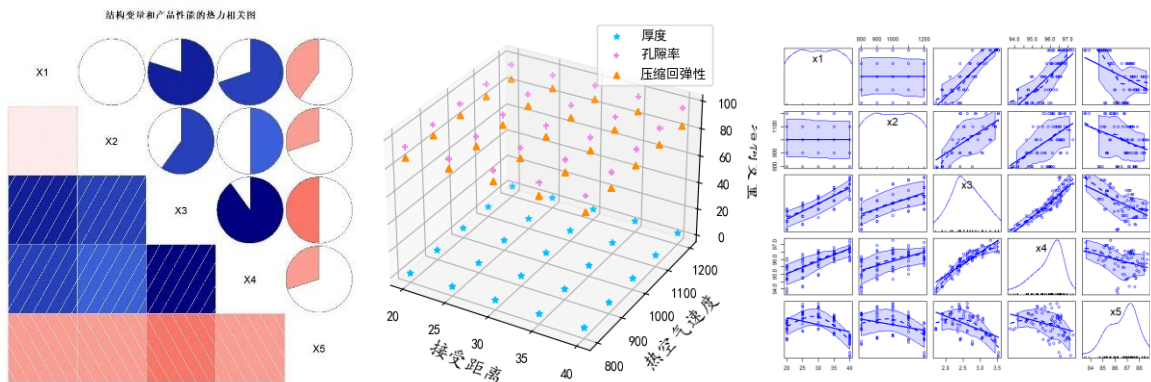


图 6 工艺参数及结构变量之间的非线性关系图示

根据图 4(右)，特征 x_1 和 x_5 在二维平面下的拟合曲线明显是曲线关系，而非直线。因此，特征之间可能存在潜在的多项式关系。利用这一点，我们对原始数据做特征衍生，加入能表达非线性关系的高次项和交互项，所得到的两组特征为

$x_1, x_2, x_1^2, x_2^2, x_1^3, x_2^3, x_1:x_2$, 和 $x_3, x_4, x_5, x_3^2, x_4^2, x_5^2, x_3^3, x_4^3, x_5^3, x_3:x_4, x_4:x_5, x_3:x_5, x_3:x_4:x_5$ 。

5.2.2 整体分析：典型相关分析

为了探究工艺参数与结构变量之间的关系，我们首先决定从两组变量总体的相关性出发，对这两组数据进行典型相关分析。典型相关分析采用主成分的思想浓缩信息，用变量组内的观测值线性组合形成的典型变量 U, V 代替原始变量，通过对综合变量之间的相关性分析，反映两组原始变量之间的关系。此处，我们可提取出两对典型变量如下：

$$\begin{cases} U_1 = -0.049x_1 - 0.001x_2 \\ V_1 = -0.449x_3 + 0.002x_4 + 0.045x_5 \end{cases} \quad (4)$$

$$\begin{cases} U_2 = -0.054x_1 + 0.001x_2 \\ V_2 = 0.682x_3 - 0.144x_4 + 0.203x_5 \end{cases} \quad (5)$$

上述两个典型相关对的分析结果如下表 2，即工艺参数与结构变量这两组间具有显著典型相关关系。

表 2 典型相关系数及显著性

典型相关系数	Wilks'lambda	F	p
0.999	0.002	205.671	<0.001**
0.679	0.539	11.136	<0.001**

为了表示典型变量 U, V 与 5 组指标之间的相关性，我们采用典型载荷系数对其进行表示，载荷系数的绝对值越大即典型变量与指标间相关性越强，结果如表 3 所示。

表 3 典型载荷系数

特征	典型变量 U_1	典型变量 U_2	典型变量 V_1	典型变量 V_2
x_1	-0.826	-0.563	-0.816	-0.048
x_2	-0.563	0.826	-0.556	0.070
x_3	-0.988	-0.001	-1.000	-0.016
x_4	-0.893	0.020	-0.908	0.236
x_5	0.513	-0.038	0.519	-0.444

在分析了工艺参数与结构变量两组变量之间的关系后，我们接下来分析变量个体间的线性关系与非线性关系。

5.2.3 个体分析：统计模型

在统计模型中，能有效反映变量之间的关系的方法是相关分析与回归分析。利用回归分析中，标准化回归系数能直观体现因变量对自变量的影响大小的特点，我们以标准化回归模型作为基础，添加逐步回归的步骤，并以特征衍生出的变量作为自变量，工艺参数和结构变量的原始变量分别作为因变量，以最小二乘法拟合多元回归模型。

回归方程的显著性检验结果如表 4 所示，表中，虽然所有的模型都通过了 F 检验，但是利用工艺参数预测结构变量的模型拟合较好，而用结构变量预测工艺参数的模型的 R^2 较低。这是因为工艺参数的设置在二维空间中呈网格状、离散性的分布，在样本量较少的情况下，回归方程很难预测变量间的关系。

表 4 回归分析的模型检验

因变量	R^2	AIC	F 统计量	P-value
x_1	0.7116	132.6	34.1	<0.0001

x_2	0.3082	190.2	32.52	<0.0001
x_3	0.9998	-408.8	65220	<0.0001
x_4	0.8913	113.2	59.4	<0.0001
x_5	0.9644	-26.2	473.4	<0.0001

对回归系数进行显著性检验，利用逐步回归法，逐步剔除最不显著的变量，直到所有的变量都能通过 t 检验为止，最终的结果如表 5 所示。

表 5 回归分析的参数检验

因变量	特征	参数估计	标准误差	T 统计量	P-value	显著性
x_1	截距	-0.2183	0.1019	-2.142	0.0357	*
	x_3	0.6362	0.1911	3.330	0.0014	**
	x_4	0.3326	0.1970	1.690	0.0956	.
	$x_3 \times x_4$	0.2029	0.0888	2.286	0.0253	*
	$x_3 \times x_5$	-0.1907	0.0847	-2.252	0.0275	*
	$x_3 \times x_4 \times x_5$	0.2179	0.0817	2.659	0.0097	**
x_2	x_3	0.5552	0.0967	5.742	<0.0001	***
x_3	截距	0.0221	0.0034	6.483	<0.0001	***
	x_1	0.8169	0.0018	466.564	<0.0001	***
	x_2	0.5552	0.0018	317.062	<0.0001	***
	x_1^2	0.1203	0.0021	57.094	<0.0001	***
	x_2^2	-0.1427	0.0021	-67.746	<0.0001	***
	$x_1 \times x_2$	0.0135	0.0018	7.670	<0.0001	***
x_4	截距	0.3108	0.0774	4.014	0.0001	***
	x_1	0.7301	0.0397	18.393	<0.0001	***
	x_2	0.5219	0.0397	13.150	<0.0001	***
	x_1^2	-0.0953	0.0477	-1.995	0.0499	*
	x_2^2	-0.2198	0.0477	-4.601	<0.0001	***
	$x_1 \times x_2$	-0.2166	0.0399	-5.422	<0.0001	***
x_5	截距	1.3625	0.0440	30.95	<0.0001	***
	x_1	-0.4026	0.0226	-17.84	<0.0001	***
	x_2	-0.3203	0.0226	-14.19	<0.0001	***
	x_1^2	-0.8631	0.0272	-31.78	<0.0001	***
	x_2^2	-0.5178	0.0272	-19.07	<0.0001	***

表中：显著性和 P-value 的关系为：0:‘***’, 0.001:‘**’, 0.01:‘*’, 0.05:‘.’, 0.1:‘’

根据表 4 的标准化回归系数，所得到的回归方程如式(6)-(10)所示。

$$x_1 = -0.2183 + 0.6362x_3 + 0.3326x_4 + 0.2029x_3 \times x_4 - 0.1907x_3 \times x_5 + 0.2179x_3 \times x_4 \times x_5 \quad (6)$$

$$x_2 = 0.552x_3 \quad (7)$$

$$x_3 = 0.0221 + 0.8169x_1 + 0.5552x_2 + 0.1203x_1^2 - 0.1427x_2^2 + 0.0135x_1 \times x_2 \quad (8)$$

$$x_4 = 0.3108 + 0.7301x_1 + 0.5219x_2 - 0.0953x_1^2 - 0.2198x_2^2 - 0.2166x_1 \times x_2 \quad (9)$$

$$x_5 = 1.3625 - 0.4026x_1 - 0.3203x_2 - 0.8631x_1^2 - 0.5178x_2^2 \quad (10)$$

最后，为验证模型的稳定性，还需要对回归方程的假设进行验证。多元线性回归分析具有自变量独立性、残差独立性、残差方差齐性、残差正态性、自变量与因变量存在线性关系五个假设。由于使用非线性回归，第五条不满足；由于已经明确自变量存在相关，且添加了交互性。因此只需要检验残差的独立性、方差齐性和正态性。

分别绘制五个模型的残差图、残差 Q-Q 图、位置尺度图和残差杠杆图，如图 7 所示。第一行的残差图与拟合图反映了残差的独立性，一般来说，既然自变量与因变量相关，那么残差与预测值就应该无任何关联。一个优秀的回归模型中，除了白噪声，模型应该包含全部的系统方差；第二行是残差正态 Q-Q 图，该图对残差的正态性进行了检验，即具有显著正态性；第三行是位置尺度图，位置尺度图中红线上下点随机分布，则说明模型满足方差齐性的假设；第四行是残差与杠杆图。根据图像，可以看出回归分析中有没有离群点、强影响点或异常点。

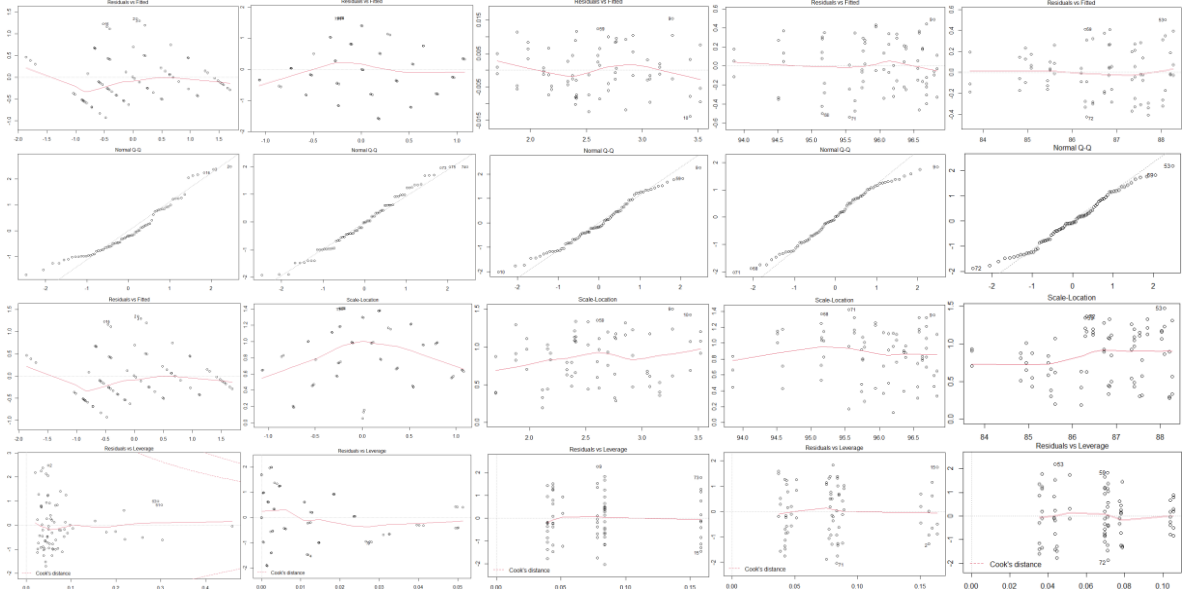


图 7 回归分析的模型检验

5.2.4 个体分析：机器学习模型

统计模型具备可解释性较强的优点，但是往往对较为复杂的数据你和能力较差。为了提升模型的拟合和预测性能，我们在进行了统计模型计算的基础上，同时利用机器学习领域中，可解释性较强的集成树模型对数据进行回归分析。以树作为基学习器的集成模型中，最具代表性的是随机森林算法以及 AdaBoost 算法，分别代表了 Bagging 和 Boosting 两种集成方式。GBDT 是在 AdaBoost 上的改进，XGBoost 又是在 GBDT 上更加深入的改进。同时，中国微软亚研院所提出的 LightGBM 算法以及俄罗斯公司 Yandex 所提出的 CatBoost 算法也在多个监督学习任务上展现了非常优秀的性能。因此，我们就选用随机森林、AdaBoost、GBDT、XGBoost、LightGBM 和 CatBoost 进行拟合。

随机森林^[3]是 Leo Breiman 和 Adele Cutler 提出的一种集成树算法，该术语是 1995 年由贝尔实验室的 Tin Kam Ho 所提出的随机决策森林而来的，并被注册为商标。该算法采用自助抽样法，随机特征子集和平均投票来进行预测的由许多决策树组成的集合模型，可以降低过拟合的风险。

AdaBoost^[4]是 Yoav Freund 和 Robert E. Schapire 在 1995 年的 the Second European Conference on Computational Learning Theory 首次提出的一种集成学习模型。它通过基学习器的线性组合来最小化损失函数，在模型训练的过程中通过提升分错数据点的权重来定位模型的不足。使得误差率低的弱分类器在最终分类器中占的权重较大，否则较小。

GBDT^[5]是 Freidman 提出的一种梯度提升树算法，与 AdaBoost 不同的是，GBDT 是通过计算负梯度来识别问题和改进模型。GBDT 每一步残差计算增大了被分类错误

样本的权重，对分类正确的样本的权重趋近于零，使得树能专注于那些被错分的样本。为了更精确地定位模型的不足，GBDT 可以使用更多种类的目标函数，解决了部分损失函数对于异常值鲁棒性较差的问题。

XGBoost^[6]是美国卡内基梅隆大学华裔学者陈天奇教授所提出的集成学习模型。与 AdaBoost 和 GBDT 不同的是，XGBoost 在计算中不仅使用一阶导数，而且对目标函数进行二阶泰勒展开，提升了计算效率和拟合精度。为方便计算，XGBoost 将数据格式转换为叶节点的方式，不仅能较快的计算损失，而且可以以增益为基础输出模型的特征重要性，直观的体现每一个自变量对预测因变量的重要程度。

LightGBM^[7]是软微旗下的 Distributed Machine Learning Toolkit (DMKT) 项目，由 2014 年首届阿里巴巴大数据竞赛获胜者之一柯国霖主持开发。相比于 xgboost 的 level-wise 生长策略，lightgbm 使用了 leaf-wise 树生长策略，控制模型过拟合的同时提高了模型的可解释性。为了提升模型运行的性能，lightgbm 还采用了基于梯度的单边采样算法，针对特征的互斥特征捆绑算法，主要解决的是速度和内存的问题。

CatBoost^[8]是俄罗斯的搜索巨头 Yandex 在 2017 年开源的机器学习库。与 XgBoost 和 Lightgbm 相比，CatBoost 嵌入了自动将类别型特征处理为数值型特征的创新算法。为了丰富特征维度，CatBoost 利用了特征之间的联系进行类别特征组合。为了避免梯度估计的偏差，解决预测偏移的问题，CatBoost 采用排序提升的方法对抗训练集中的噪声点。此外，Catboost 采用了完全对称树作为基模型，降低模型过拟合的风险。

表 6 集成树模型的拟合性能度量

自变量	模型	R^2	MAE	RMSE
x_1	随机森林	0.979716	0.657778	1.007708
	AdaBoost	0.957407	1.059244	1.459335
	GBDT	0.999999	0.004410	0.005462
	XGBoost	0.999999	0.000725	0.001059
	LightGBM	0.992508	0.391594	0.612031
	CatBoost	0.999094	0.165692	0.212839
x_2	随机森林	0.943198	26.76000	33.70534
	AdaBoost	0.833728	51.15604	57.66657
	GBDT	0.999996	0.218287	0.278276
	XGBoost	0.999999	0.000844	0.001237
	LightGBM	0.929685	28.26417	37.50061
	CatBoost	0.998665	4.107329	5.166817
x_3	随机森林	0.999637	0.007591	0.008929
	AdaBoost	0.984468	0.048425	0.058461
	GBDT	0.999862	0.004428	0.005509
	XGBoost	0.999861	0.004453	0.005532
	LightGBM	0.944364	0.079233	0.110644
	CatBoost	0.999862	0.004431	0.005509
x_4	随机森林	0.922804	0.193672	0.226360
	AdaBoost	0.911302	0.210856	0.242639
	GBDT	0.923211	0.191958	0.225764
	XGBoost	0.923211	0.192023	0.225764
	LightGBM	0.848649	0.254036	0.316955
	CatBoost	0.923211	0.191931	0.225764

x_5	随机森林	0.974319	0.154532	0.193681
	AdaBoost	0.923846	0.266226	0.333524
	GBDT	0.974989	0.151666	0.191139
	XGBoost	0.974989	0.151564	0.191137
	LightGBM	0.690122	0.498655	0.672786
	CatBoost	0.974989	0.151576	0.191137

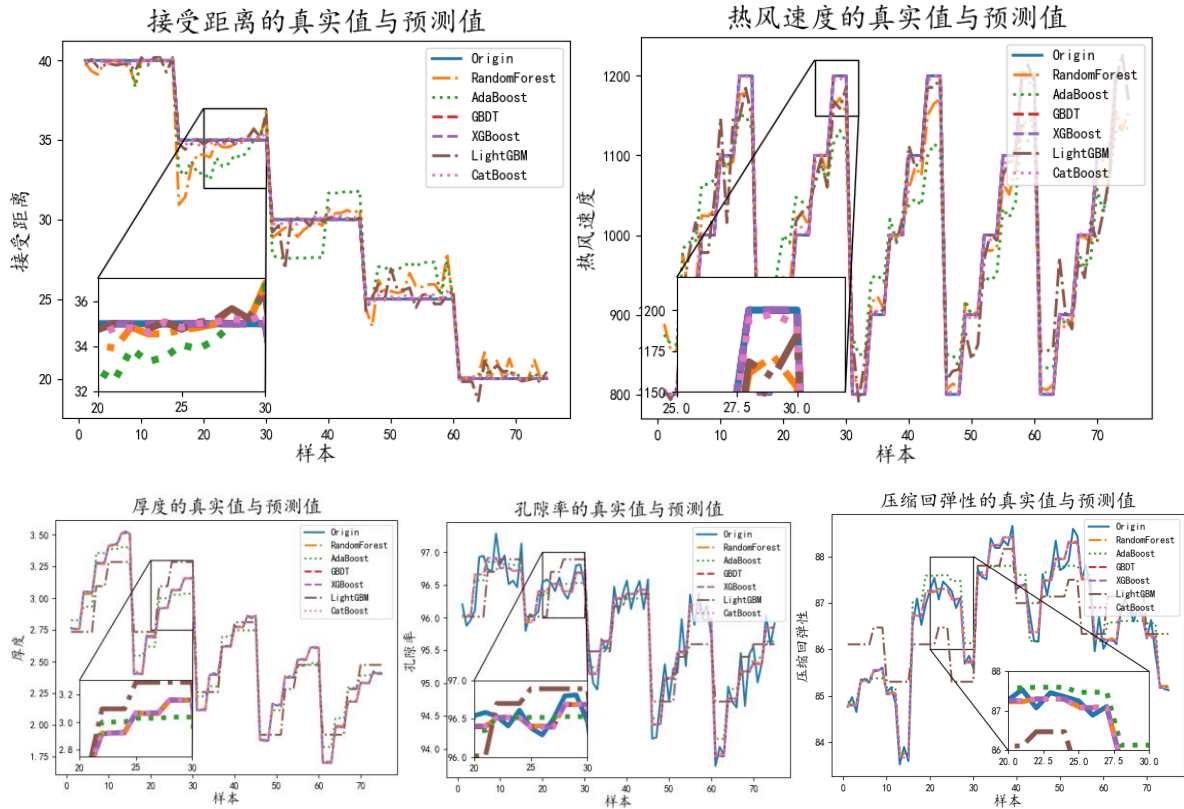


图 8 集成模型的预测值和真实值比较

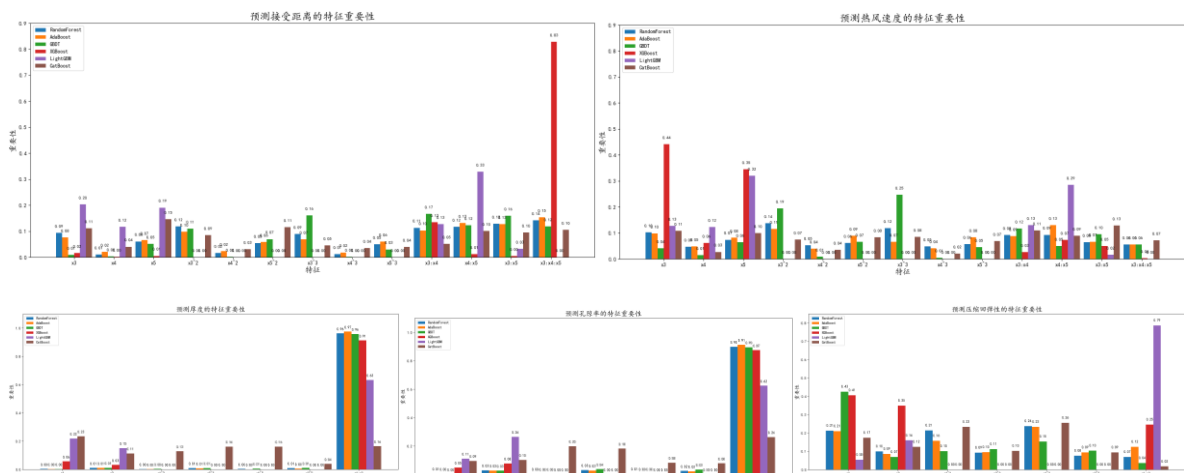


图 9 集成树模型输出的特征重要性

根据图 9，显然，不同模型尽管性能度量类似，但是对各个特征的侧重也各有不同的。例如，XGBoost 算法、随机森林等模型会特别侧重某一个或多个特征，而 CatBoost

模型在每一个特征上的重要性几乎相同。这是由 CatBoost 的结构决定了，CatBoost 模型的每一棵树都要求绝对对称，因此研究变量之间的关系时，可以将 CatBoost 剔除，防止造成困扰。当预测接受距离时， $x_3:x_4:x_5$ 是最为重要的特征；预测热风速度时， x_3 和 x_5 较为重要；预测厚度时， $x_1:x_2$ 最为重要。当预测孔隙率时， $x_1:x_2$ 最为重要。预测压缩回弹性率时， x_1 、 x_2 和 $x_1:x_2$ 较为重要。

5.2.5 模型预测

由于本题数据量较少，只有 75 个样本。若直接拆分训练集与测试集，分得的数据都会很少。为了更好的检验模型的性能，我们采用十折交叉验证法，将数据集整体分为 10 个等大的子数据集，每次选取一个作为测试集，其余的作为训练集进行训练模型。每次训练后，保留模型在测试集上的 R^2 ，MAE 和 MSE，最终输出十次预测的 R^2 ，MAE 和 MSE 的均值作为性能度量指标。

表 7 十折交叉验证的性能度量

因变量	模型	R^2	MAE	MSE
x_3	CatBoost	0.999861	0.005549	0.000038
x_4	CatBoost	0.906443	0.258359	0.106563
x_5	XGBoost	0.942343	0.284774	0.126745

显然，CatBoost 和 XGBoost 的拟合能力都较强，使用它们对题目中表 1 的数据进行预测，所得的结果如表 8 所示。

表 8 问题 2 的结果

接受距离 (cm)	热风速度 (r/min)	厚度 (mm)	孔隙率 (%)	压缩回弹性率 (%)
38	850	2.759	96.017	85.345
33	950	2.696	96.405	87.282
28	1150	2.756	96.359	86.258
23	1250	2.579	96.012	86.223
38	1250	3.517	96.723	83.669
33	1150	3.066	96.393	86.180
28	950	2.397	95.630	88.375
23	850	1.880	94.368	87.851

5.3 问题三模型的建立与求解

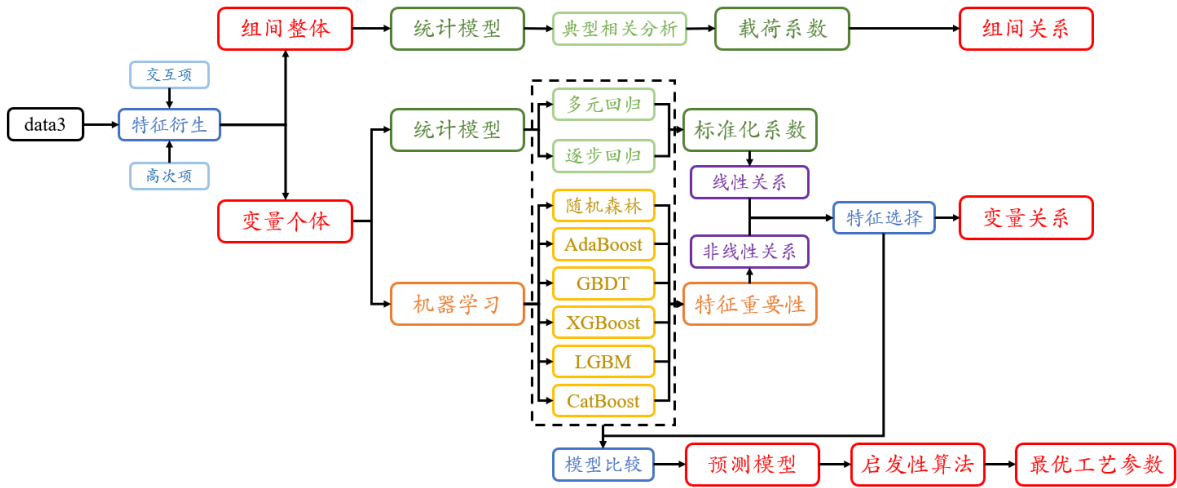


图 10 问题三的流程图

与第二问类似，为探究“结构变量”与“产品性能”之间的关系，以及“结构变量”之间、“产品性能”之间的关系，我们延续第二问的模型，同样分整体和个体进行建模和求解。在得到由“工艺参数”到三个“结构变量”，再到“产品性能”中的“过滤效率”的最佳拟合模型后，将之带入启发性算法进行寻优，寻找到最优的工艺参数。在寻优时，我们不仅使用机器学习模型作为目标函数，也使用回归方程作为补充，两两比较，选择最优的结果。

5.3.1 数据观测与特征衍生

由于结构变量及产品性能各指标均不符合正态分布，与节 5.2.1 一样，我们采用 Spearman 秩相关系数来表征厚度、孔隙率、压缩回弹性、过滤阻力、过滤效率、透气性之间的线性相关性，并用热力相关图、散点图矩阵和三维散点图进行可视化变量之间的关系。图(左)的相关图体现变量之间存在一定的相关性，但是图(右)的散点图矩阵以及图的三维散点图显示出，变量之间的关系并未简单的线性关系。

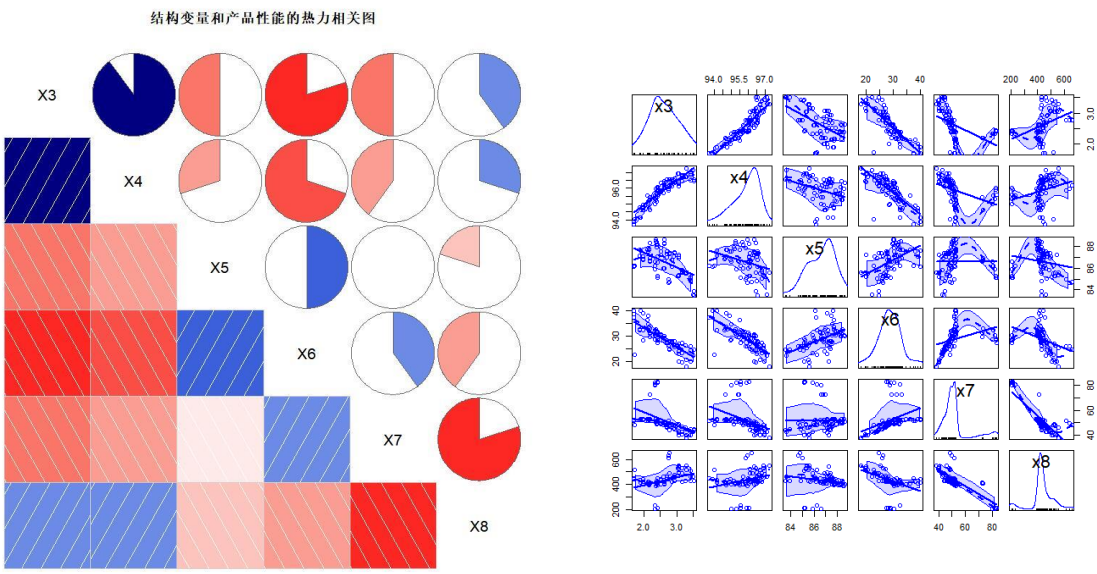


图 11 结构变量和产品性能的相关图和散点图矩阵

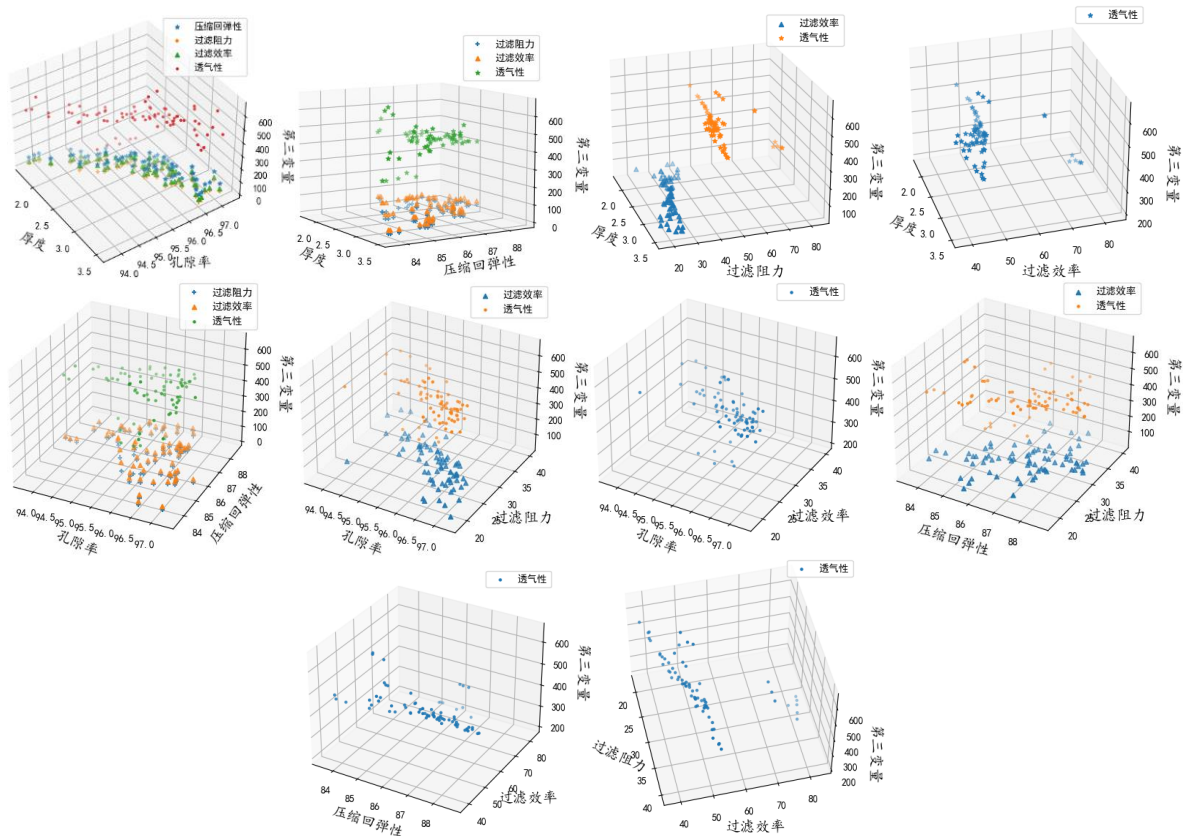


图 12 结构变量和产品性能的三维散点图

与节 5.2.1 相同，我们对数据进行特征衍生，得到 $x_3, x_4, x_5, x_3^2, x_4^2, x_5^2, x_3^3, x_4^3, x_5^3, x_3 \cdot x_4, x_3 \cdot x_5, x_4 \cdot x_5, x_3 \cdot x_4 \cdot x_5, x_6, x_7, x_8, x_6^2, x_7^2, x_8^2, x_6^3, x_7^3, x_8^3, x_6 \cdot x_7, x_6 \cdot x_8, x_7 \cdot x_8, x_6 \cdot x_7 \cdot x_8$ 两组特征。

5.3.2 整体与组内分析：典型相关分析

本部分将从结构变量与产品性能两组变量总体的相关性出发，对这两组数据进行典型相关分析。此处，我们可提取出三对典型变量如下：

$$\begin{cases} U_1 = -0.621x_3 - 0.081x_4 + 0.097x_5 \\ V_1 = -0.020x_6 - 0.032x_7 - 0.001x_8 \end{cases} \quad (11)$$

$$\begin{cases} U_2 = 0.022x_3 - 0.088x_4 - 0.158x_5 \\ V_2 = -0.192x_6 + 0.146x_7 - 0.002x_8 \end{cases} \quad (12)$$

$$\begin{cases} U_3 = 0.797x_3 - 0.718x_4 + 0.099x_5 \\ V_3 = -0.121x_6 + 0.107x_7 - 0.004x_8 \end{cases} \quad (13)$$

上述三个典型相关对的分析结果如下表 9，其中一对呈现出显著性，即工艺参数与结构变量这两组间具有显著典型相关关系。

表 9 典型相关系数及显著性

典型相关系数	Wilks'lambda	F	p
0.896	0.150	7.681	<0.001**
0.461	0.763	1.807	0.142
0.175	0.969	0.822	0.373

为了表示典型变量 U, V 与 8 组指标之间的相关性，我们采用典型载荷系数对其进

行表示，结果如表 10 所示。

表 10 典型载荷系数

特征	典型变量 U_1	典型变量 U_2	典型变量 U_3	典型变量 V_1	典型变量 V_2	典型变量 V_3
x_3	-0.992	0.116	0.057	-0.837	-0.066	0.006
x_4	-0.909	0.154	-0.388	-0.767	0.086	-0.043
x_5	0.413	-0.794	-0.447	0.349	-0.447	-0.049
x_6	0.776	-0.220	-0.006	0.919	-0.391	-0.052
x_7	0.521	0.317	-0.061	0.617	0.564	-0.549
x_8	-0.355	-0.032	0.100	-0.420	0.058	0.906

在分析了结构变量与产品性能两组变量之间的关系后，我们接下来分析变量个体间的线性关系与非线性关系。

5.3.3 个体分析：统计模型

与 5.2.3 类似，为反映结构变量和产品性能之间的关系，首先以标准化回归模型作为基础，添加逐步回归的步骤，并以特征衍生出的变量作为自变量，结构变量和产品性能的原始变量分别作为因变量，以最小二乘法拟合多元回归模型。

回归方程的显著性检验结果如表 8 所示，表中，虽然所有的模型都通过了 F 检验，但是利用模型拟合结构变量及产品性能的 R^2 均较低。这是因为结构变量及产品性能的设置二维空间中均呈网格状、离散性的分布，在样本量较少的情况下，回归方程很难预测变量间的关系。

表 11 回归分析的模型检验

因变量	R^2	AIC	F 统计量	P-value
x_3	0.8842	74.1	48.87	<0.0001
x_4	0.7353	132.1	22.91	<0.0001
x_5	0.7239	137.3	18.93	<0.0001
x_6	0.7569	123.7	29.81	<0.0001
x_7	0.7297	129.7	26.22	<0.0001
x_8	0.4966	170.3	17.51	<0.0001

对回归系数进行显著性检验，利用逐步回归法，逐步剔除最不显著的变量，直到所有的变量都能通过 t 检验为止，最终的结果如表 12 所示。

表 12 回归分析的参数检验

因变量	特征	参数估计	标准误差	T 统计量	P-value	显著性
x_3	截距	-0.76747	0.08899	-8.624	<0.0001	***
	x_7	-2.55029	0.18990	-13.43	<0.0001	***
	x_8	-0.66312	0.16611	-3.992	0.0002	***
	x_6^2	-0.42075	0.10312	-4.080	0.0002	***
	x_7^2	1.32206	0.16758	7.889	<0.0001	***
	x_8^2	0.81821	0.16041	5.101	<0.0001	***
	x_6^3	0.11156	0.04064	2.745	0.0078	**
	x_8^3	-0.10971	0.05686	-1.929	0.0581	.
	$x_7: x_6$	0.66218	0.25719	2.575	0.0124	*
	$x_7: x_8$	1.67108	0.32896	5.080	<0.0001	***

	$x_3:x_4:x_5$	0.19336	0.11139	1.736	0.0874	.
	截距	-0.43612	0.11248	-3.877	0.0002	***
	x_7	-2.10847	0.24612	-8.567	<0.0001	***
	x_8	-0.46671	0.19115	-2.442	0.1731	*
	x_6^2	-0.71461	0.12776	-5.593	<0.0001	***
x_4	x_7^2	1.07391	0.17252	6.225	<0.0001	***
	x_6^3	0.16524	0.05553	2.976	0.0041	**
	$x_6:x_8$	-0.93322	0.29684	-3.144	0.0025	**
	$x_7:x_8$	0.49039	0.14776	3.319	0.0015	**
	$x_6:x_7:x_8$	0.32843	0.12874	2.551	0.0131	*
	截距	0.33383	0.08800	3.794	0.0003	***
	x_6	1.00183	0.17535	5.713	<0.0001	***
	x_8	-0.42700	0.24841	-1.719	0.0903	.
	x_6^2	-0.97684	0.13499	-7.237	<0.0001	***
	x_8^2	-0.67695	0.13456	-5.031	<0.0001	***
x_5	x_6^3	0.18766	0.06134	3.059	0.0032	**
	x_7^3	-0.15506	0.05337	-2.906	0.0050	**
	$x_6:x_8$	-1.96663	0.31145	-6.314	<0.0001	***
	$x_7:x_8$	-0.79448	0.25751	-3.085	0.0030	**
	$x_6:x_7:x_8$	0.76992	0.13174	5.844	<0.0001	***
	截距	0.21151	0.11417	1.853	0.0683	.
	x_3	-0.81752	0.15604	-5.239	<0.0001	***
	x_5	0.14416	0.08182	1.762	0.0826	.
	x_3^2	-0.48658	0.12674	-3.839	<0.0001	***
x_6	x_5^2	-0.17354	0.08397	-2.607	0.0426	*
	x_3^3	0.17507	0.09313	1.880	0.0645	.
	x_4^3	-0.18750	0.04644	-4.038	<0.0001	***
	$x_3:x_5$	-0.54575	0.16387	-3.330	0.0014	**
	x_3	-1.83671	0.16881	-10.880	<0.0001	***
	x_5	-0.62849	0.09152	-6.867	<0.0001	***
	x_4^2	0.20054	0.07287	2.752	0.0076	**
x_7	x_5^2	0.29293	0.07196	4.071	0.0001	***
	x_3^3	0.75080	0.09976	7.526	<0.0001	***
	$x_3:x_5$	1.06455	0.14105	7.547	<0.0001	***
	$x_3:x_4:x_5$	0.21673	0.08866	2.445	0.0171	*
	x_3	1.7851	0.2212	8.068	<0.0001	***
x_8	x_3^3	-0.8490	0.1272	-6.675	<0.0001	***
	$x_3:x_5$	-0.8013	0.1728	-4.637	<0.0001	***
	$x_3:x_4$	-0.3116	0.1067	-2.921	0.00468	**

表中：显著性和 P-value 的关系为：0:***, 0.001:**, 0.01:*, 0.05:., 0.1:'

根据表 12 的标准化回归系数，所得到的回归方程如式(14)-(19)所示。

$$\begin{aligned}
 x_3 = & -0.76747 - 2.55029x_7 - 0.66312x_8 - 0.42075x_6^2 + 1.32206x_7^2 \\
 & + 0.81821x_8^2 + 0.11156x_6^3 - 0.10971x_8^3 + 0.66218x_7 \times x_6 \\
 & + 1.67108x_7 \times + 0.19336x_3 * x_4 \times x_5
 \end{aligned} \tag{14}$$

$$x_4 = -0.43612 - 0.210847x_7 - 0.46671x_8 - 0.71461x_6^2 + 1.07391x_7^2 + 0.16524x_6^3 - 0.93322x_6 \times x_8 + 0.49039x_7 \times x_8 + 0.32843x_6 \times x_7 \times x_8 \quad (15)$$

$$x_5 = 0.33383 + 1.00183x_6 - 0.42700x_8 - 0.97684x_6^2 - 0.67695x_8^2 + 0.18766x_6^3 - 0.15506x_7^3 - 1.96663x_6 \times x_8 - 0.79448x_7 \times x_8 + 0.76992x_6 \times x_7 \times x_8 \quad (16)$$

$$x_6 = 0.21151 - 0.21752x_3 + 0.14416x_5 - 0.18658x_3^2 - 0.17354x_5^2 + 0.17507x_3^3 - 0.18750x_4^3 - 0.54575x_3 \times x_5 \quad (17)$$

$$x_7 = -1.83671x_3 - 0.62849x_5 + 0.20054x_4^2 + 0.29293x_5^2 + 0.75080x_3^3 + 1.06455x_3 \times x_5 + 0.21673x_3 \times x_4 \times x_5 \quad (18)$$

$$x_8 = 1.7851x_3 - 0.8490x_3^3 - 0.8013x_3 \times x_5 - 0.3116x_3 \times x_4 \quad (19)$$

与节 5.2.3 类似，检验假设的图像如图 13 所示。

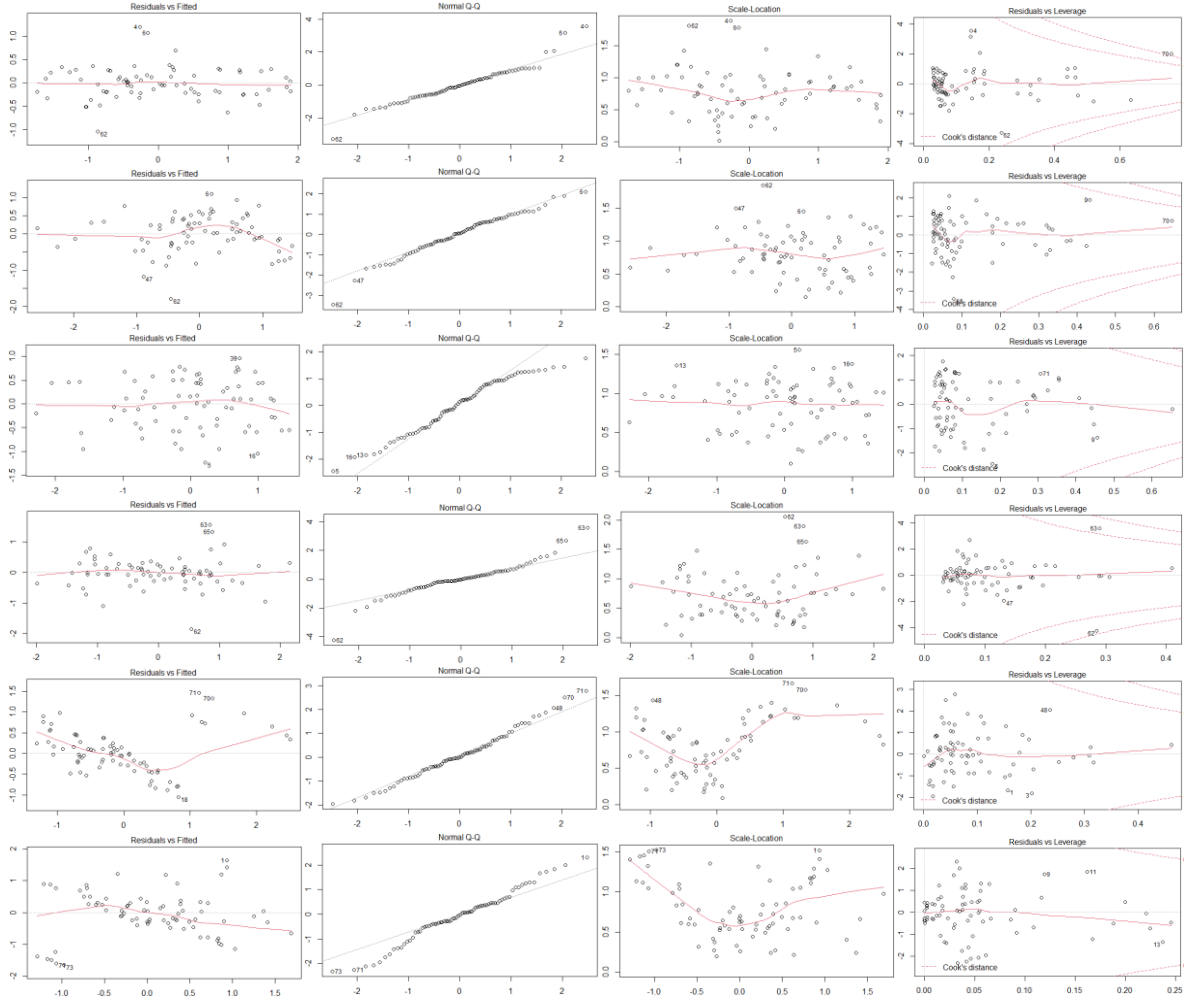


图 13 回归分析检验

另外，我们还需对结构变量之间、产品性能之间的关系进行分析，与本节结构变量和产品性能的回归模型思路相同。以标准化回归模型作为基础，添加逐步回归的步骤，并以结构变量特征衍生出的变量作为自变量，结构变量原始变量分别作为因变量，以最小二乘法拟合结构变量的多元回归模型。同理，建立产品性能的多元回归模型。回归方程的显著性检验结果如表 13 所示

表 13 回归分析的模型检验

因变量	R^2	AIC	F 统计量	P-value
x_3	0.8921	56.8	144.7	<0.0001

x_4	0.8898	54.4	290.7	<0.0001
x_5	0.4379	178.6	18.44	<0.0001
x_6	0.7865	110.0	50.84	<0.0001
x_7	0.8337	89.3	86.76	<0.0001
x_8	0.7917	106.2	66.52	<0.0001

对回归系数进行显著性检验，利用逐步回归法，逐步剔除最不显著的变量，直到所有的变量都能通过 t 检验为止，最终的结果如表 14 所示

表 14 回归分析的参数检验

因变量	特征	参数估计	标准误差	T 统计量	P-value	显著性
x_3	截距	-0.1082	0.0480	-2.254	0.0273	*
	x_4	-0.9668	0.0772	12.520	<0.0001	***
	x_5	-0.1638	0.0485	-3.372	0.0012	**
	x_4^3	-0.0525	0.0217	-2.412	0.0184	*
	$x_4 \cdot x_5$	-0.2189	0.0565	-3.868	0.0002	***
x_4	截距	0.2564	0.0518	4.994	<0.0001	***
	x_3	0.9258	0.0393	23.549	<0.0001	***
	x_3^2	-0.2599	0.0348	-7.464	<0.0001	***
x_5	截距	0.2853	0.1138	2.511	0.0143	*
	x_3	-0.4830	0.1108	-4.359	<0.0001	***
	x_4^2	0.4779	0.2541	1.881	0.0642	.
	$x_3 \cdot x_4$	-0.8552	0.2911	-2.938	0.0044	**
x_6	截距	0.9033	0.0879	10.278	<0.0001	***
	x_7	1.9976	0.1616	12.365	<0.0001	***
	x_7^2	-1.2610	0.1556	-8.1000	<0.001	***
	x_8^2	-0.6154	0.1694	-3.633	<0.0001	***
	x_8^3	0.1103	0.0528	2.089	0.0404	*
	$x_7 \cdot x_8$	-1.3138	0.3638	-3.612	<0.0001	***
x_7	截距	-0.1754	0.0641	-2.732	0.0080	**
	x_6	0.2773	0.0591	4.696	<0.0001	***
	x_8	-0.4772	0.0630	-7.527	<0.0001	***
	x_6^2	-0.0722	0.0348	-2.076	0.0416	*
	x_8^2	0.22497	0.0270	9.248	<0.0001	***
x_8	截距	0.4714	0.1000	4.711	<0.0001	***
	x_6	-0.9534	0.1285	-7.413	<0.0001	***
	x_7	1.0814	0.2620	4.127	<0.0001	***
	x_6^2	0.2830	0.0504	5.619	<0.0001	***
	x_7^2	-0.7691	0.1081	-7.039	<0.0001	***

根据标准化回归系数，所得到的结构变量的回归方程如下所示。

$$x_3 = -0.10818 + 0.96681x_4 - 0.16381x_5 - 0.05249x_4^3 - 0.21886x_4 \times x_5 \quad (20)$$

$$x_4 = 0.25645 + 0.92583x_3 - 0.25991x_3^2 \quad (21)$$

$$x_5 = 0.2853 - 0.4830x_3 + 0.4779x_4^2 - 0.8552x_3 \times x_4 \quad (22)$$

$$x_6 = 0.90329 + 1.99763x_7 - 1.26062x_7^2 - 0.61537x_8^3 + 0.11031x_8^3 - 1.31381x_7 \times x_8 \quad (23)$$

$$x_7 = -0.17514 + 0.27733x_6 - 0.47715x_8 - 0.07220x_6^2 + 0.24970x_8^2 \quad (24)$$

$$x_8 = 0.47143 - 0.95376x_6 + 1.08138x_7 + 0.28299x_6^2 - 0.76079x_7 \quad (25)$$

根据标准化系数，预测 x_3 时， x_4 的线性影响最大。预测 x_4 时， x_3 的线性影响最大。选标准化系数最高的就是 x_3 。预测 x_5 时， x_3 : x_4 的线性影响最大。预测 x_6 时， x_7 的线性影响最大。预测 x_7 时， x_8 的线性影响最大。预测 x_8 时， x_7 的线性影响最大。

5.3.4 个体分析：机器学习模型

与节 5.2.4 相同，使用六种集成树模型进行回归分析，模型的预测精度如表 15 所示。

表 15 集成树模型的拟合性能度量

自变量	模型	R^2	MAE	RMSE
x_3	随机森林	0.975072	0.043114	0.074062
	AdaBoost	0.975480	0.062307	0.073454
	GBDT	0.999999	0.000345	0.000459
	XGBoost	0.999999	0.000649	0.000853
	LightGBM	0.996687	0.020670	0.027074
	CatBoost	0.999641	0.007197	0.008890
x_4	随机森林	0.960374	0.116458	0.162180
	AdaBoost	0.934543	0.172063	0.208441
	GBDT	0.999993	0.001694	0.002097
	XGBoost	0.999999	0.000703	0.000958
	LightGBM	0.985503	0.075762	0.098094
	CatBoost	0.997511	0.033541	0.040627
x_5	随机森林	0.969258	0.166876	0.211912
	AdaBoost	0.909545	0.309057	0.363494
	GBDT	0.999988	0.003404	0.004255
	XGBoost	0.999999	0.000757	0.001022
	LightGBM	0.961093	0.155379	0.238394
	CatBoost	0.997168	0.051331	0.064316
x_6	随机森林	0.936282	0.735464	1.153939
	AdaBoost	0.921857	1.068854	1.277905
	GBDT	0.999979	0.016470	0.020630
	XGBoost	0.999999	0.000931	0.001288
	LightGBM	0.855269	1.163581	1.739133
	CatBoost	0.994276	0.277429	0.345864
x_7	随机森林	0.978282	0.864781	1.577143
	AdaBoost	0.983745	1.104179	1.364422
	GBDT	0.999997	0.01405	0.017889
	XGBoost	0.999999	0.001029	0.001339
	LightGBM	0.949475	1.649768	2.405563
	CatBoost	0.999053	0.249334	0.329275
x_8	随机森林	0.944367	13.70743	19.99231
	AdaBoost	0.933791	17.07784	21.81004

GBDT	0.999988	0.238063	0.283034
XGBoost	0.999999	0.000918	0.001255
LightGBM	0.903954	18.47930	26.26865
CatBoost	0.995988	4.196603	5.368263

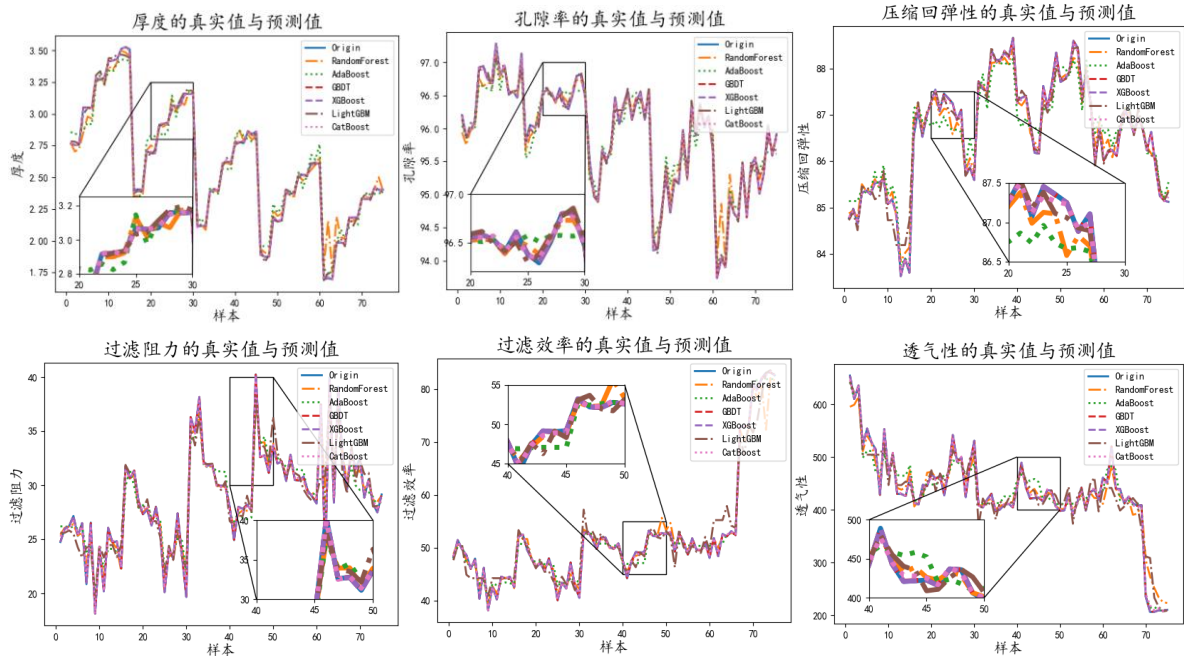


图 14 结构变量和产品性能的预测值与真实值

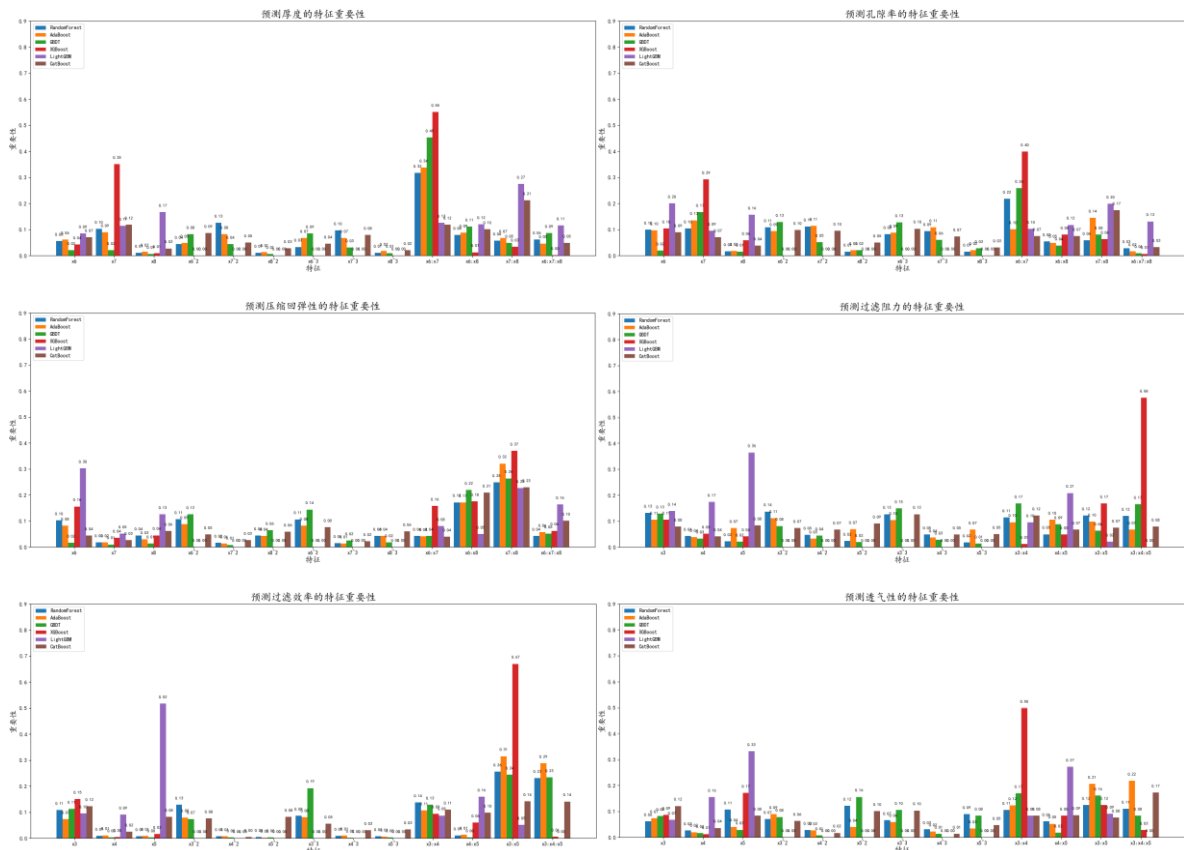


图 15 预测结构变量和产品新能的特征重要性

同样，在研究结构变量和产品性能之间的关系时，我们剔除不具有特征偏好性的模型 CatBoost 进行分析。根据图 15，当预测厚度时， $x_5:x_6$ 是最为重要的特征。当预测孔隙率时， x_7 和 $x_6:x_7$ 较为重要。当预测压缩回弹性率时， $x_7:x_8$ 最为重要， x_6 和 $x_6:x_7$ 较为重要。当预测过滤阻力时 $x_3:x_4:x_5$ 是最重要的特征，当预测过滤效率的时候， $x_3:x_5$ 是最重要的特征， $x_3:x_4:x_5$ 和 x_3 较为重要。当预测透气性时， $x_3:x_5$ 、 $x_3:x_4$ 、 x_5 较为重要。

此后，研究结构变量之间与产品性能之间的关系，省略去拟合精度表格，展示预测值与真实值的图像如图所示，可以看出拟合较好。

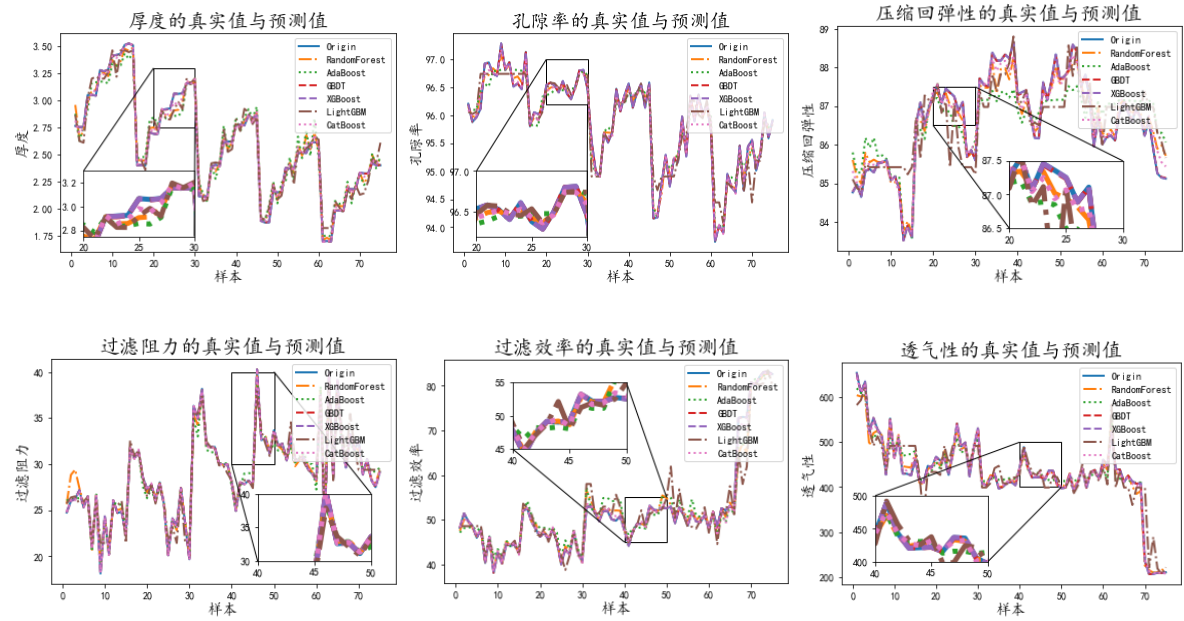


图 16 结构变量和产品性能的预测值与真实值（组间关系）

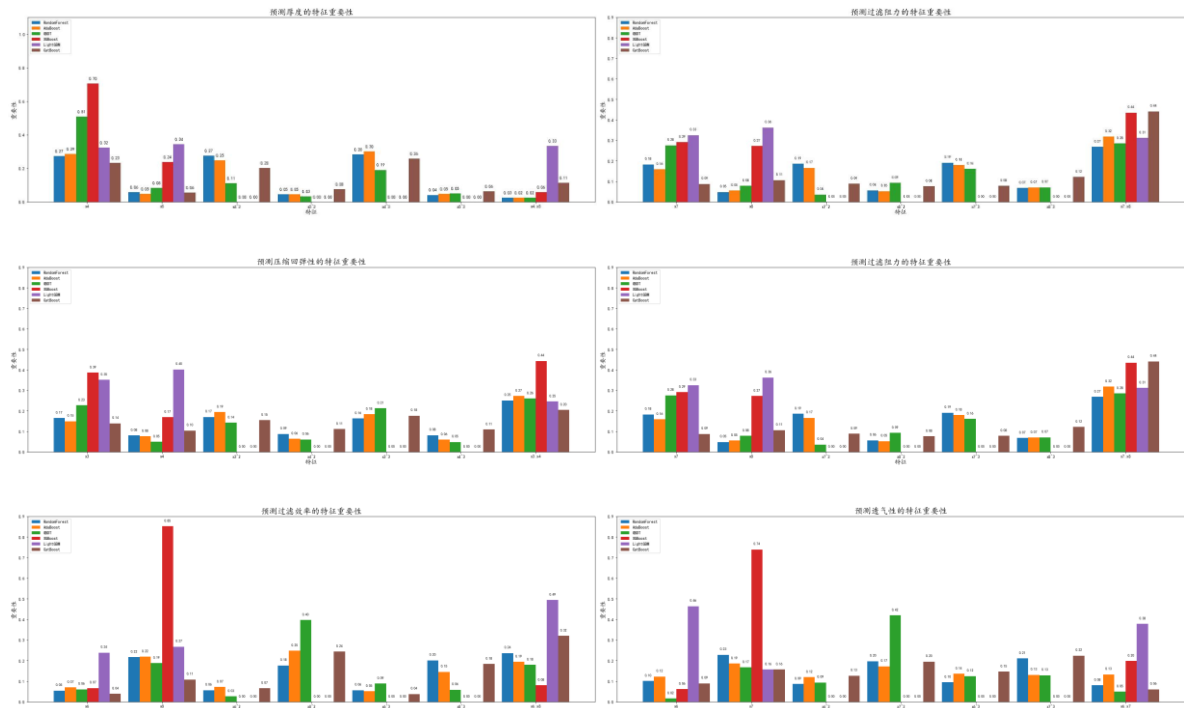


图 17 预测结构变量和产品新能的特征重要性（组间关系）

据图，预测厚度时， x_1 为最重要的特征。当预测孔隙率时， x_3 、 $x_3:x_5$ 是最重要的特征。当预测压缩回弹性率时， x_3 和 $x_3:x_4$ 是最为重要的特征， x_4 较为重要。当预测过滤阻力时， $x_7:x_8$ 最为重要， x_7 、 x_8 较为重要。当预测过滤效率时， x_8 最为重要， $x_6:x_8$ 较为重要。当预测透气性时， x_7 最为重要。

5.3.5 单目标寻优模型——增强精英保留的遗传算法 (SEGA)

在自然界中，生物周而复始的繁衍，基因随之重组、变异，使生物不断具备新的性状，以适应更加复杂多变的环境，从而实现进化。进化算法正是基于精简后的生物进化过程所提出的数学模型，其使用较为简单和抽象的编码方式来表达复杂的进化现象，并通过简化后的遗传过程来对较为复杂的解空间进行启发性的搜索，以尽可能地找到全局最优解。一般来说，进化算法都有较强的并行性，即不仅支持种群间的并行计算，也支持若干个体间的并行计算。

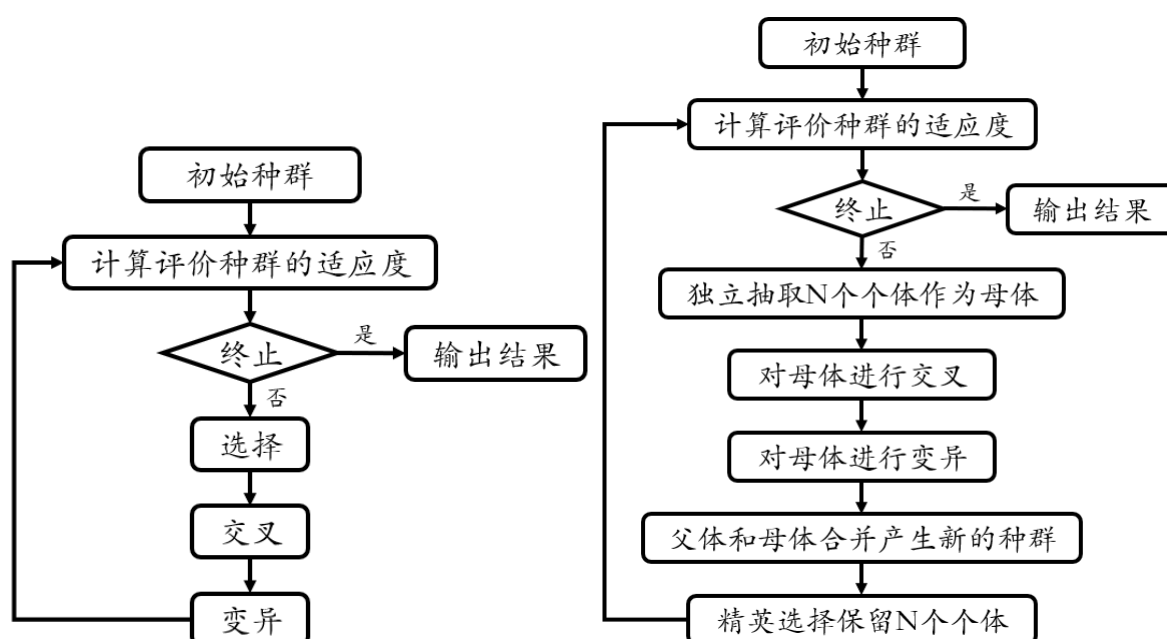


图 18 经典遗传算法和增强精英策略遗传算法的流程图

经典的遗传算法和增强精英保留的遗传算法的流程图如图 18 所示。经典遗传算法依靠染色体之间的交叉、染色体基因的变异和染色体的选择来完成进化过程。选择、重组和变异是进化算法的经典操作算子，对进化算法的改进一般都是对这三个经典算子进行改进。本文的增强精英保留的遗传算法正是对选择算子进行修正。经典的遗传算法只进行了一次选择，也就是将重组和变异后的个体无条件淘汰父本的个体。这样的做法存在一定的弊端，因为它有可能将较好的父本剔除，导致收敛速度较慢。为此，我们加入精英保留策略对遗传算法进行改进。具体的算法步骤为：

- 1) 根据编码的约束，初始化 N 个个体的种群
- 2) 对当前种群进行统计分析，记录最优个体和平均适应度
- 3) 如若满足终止准则，则停止，否则继续执行
- 4) 从当前种群独立的抽取 N 个母体
- 5) 对这 N 个母体进行交叉操作
- 6) 对这 N 个母体交叉后的个体进行变异
- 7) 将父代种群和交叉变异后的种群进行合并，得到 $2N$ 个个体的种群

8) 根据精英选择策略, 从 $2N$ 个个体中选择 N 个个体, 得到新一代种群

9) 返回第 2) 步

在算法进行上述步骤时, 我们设置较大的变异率和交叉率进行进化, 这是为了防止生成的新一代种群中存在过多的重复个体。

第三问的目标函数只有最大化过滤效率, 如式(26)所示。因为题目中并未设置参数的范围, 而接受距离、热风速度等参数明显不可能为负数, 因此约束条件如式(27)所示。

$$\text{Min } x_7 = F(x_3, x_4, x_5) = F(g(x_1, x_2), h(x_1, x_2), z(x_1, x_2)) \quad (26)$$

式中: F, G, g, h, z 分别表示以 x_7, x_6, x_3, x_4, x_5 为因变量的模型。为方便表示, 并未给出多次项和交互项特征。

$$s.t. \begin{cases} x_1 > 0 \\ x_2 > 0 \\ x_3 > 0 \\ x_4 > 0 \\ x_5 > 0 \end{cases} \quad (27)$$

设置 SEGA 的初始种群个数为 100, 最大迭代次数为 25。同时, 先设置式(26)中的 $g(x_1, x_2), h(x_1, x_2), z(x_1, x_2), F(x_3, x_4, x_5), G(x_3, x_4, x_5)$ 分别为第二问和第三问中所得到的拟合性能最佳的 CatBoost, CatBoost, XGBoost, XGBoost, XGBoost 模型。在经历变异、重组等过程后, 种群的变化如图 19 所示。可以看出, 函数的收敛极快, 在前几轮就寻找到了最大值 83.14685, 此时的接受距离为 0.2862cm, 热风速度为 8085.9375r/min。

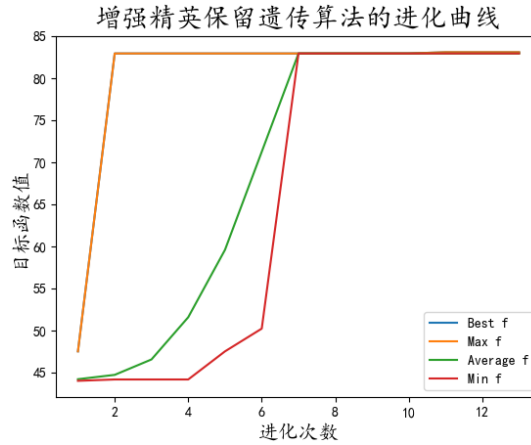


图 19 SEGA 的适应度进化曲线 (以机器学习模型作为目标函数)

最后, 为证明选择 CatBoost 与 XGBoost 模型作为函数的必要性和正确性, 同样再以第二问和第三问中得到的回归方程作为式(26)中的 $g(x_1, x_2), h(x_1, x_2), z(x_1, x_2), F(x_3, x_4, x_5), G(x_3, x_4, x_5)$ 。最终以回归方程为目标函数的 SEGA 的迭代过程如图 20 所示, 最优的目标函数为 48.8198, 但是接受距离和热风速度都为 0.0001。显然, 特征之间的线性关系非常不显著, 寻优的效果对比以机器学习为目标函数的模型差了近乎一半。

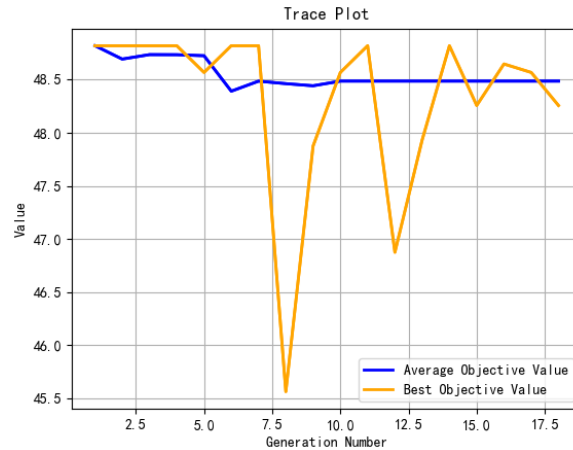


图 20 SEGA 的适应度进化曲线 (以回归方程作为目标函数)

5.4 问题四的建模与求解

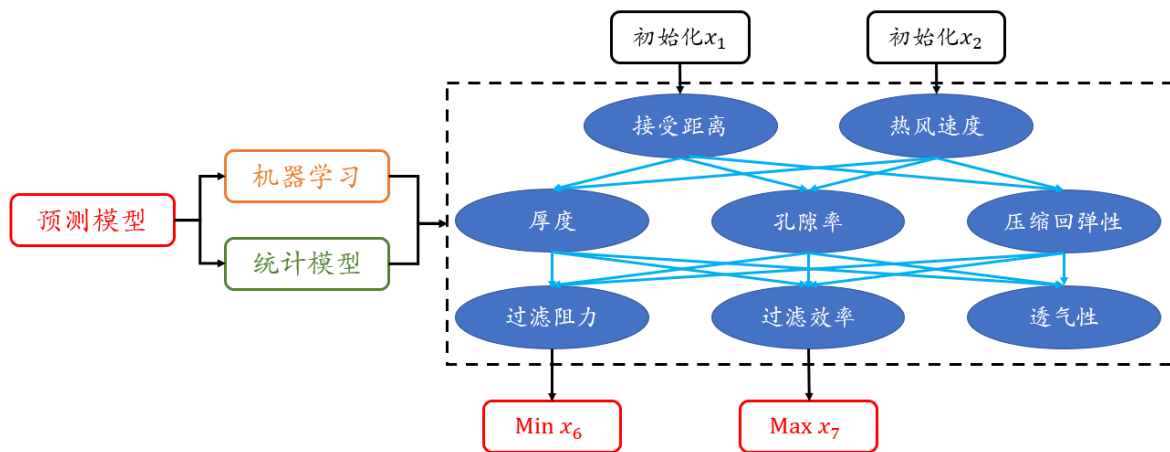


图 21 问题四的流程图

在本节中，由于回归分析得到的方程式较为复杂，机器学习模型的预测关系无法以公式的形式可视化，因此我们同样选用启发性算法进行建模。与第三问最后的寻优相同，我们在设置启发性算法中的最大化和最小化目标时，使用了拟合能力最强的集成树模型的预测方法建立算法。在寻优结束后，考虑到树模型无法外推的特征，我们使用回归方程进行二次寻优，对结果进行对比和补充。最终，最优的结果依旧是使用树模型的预测方法。

5.4.1 多目标寻优模型——NSGA-III算法

在求解单目标问题时，往往可以在所有候选解的集合中选出唯一最好的解。但是，在多目标优化中，各个目标函数之间可能存在冲突，最优的解集并不一定只有一个。一般而言，若对一个多目标优化问题 $\min f(x)$ ，对于任意子目标函数 $f_i(x)$ ，都有 $f_i(X) \leq f_i(X^*)$ ，且至少存在一个子目标函数 $f_i(x)$ 使得 $f_i(X) < f_i(X^*)$ ，则称 X^* 为强 Pareto 最优解。而若所有的子目标函数 $f_i(x)$ 都使得 $f_i(X) < f_i(X^*)$ ，则称 X^* 为弱 Pareto 最优解。以双目标寻优为例，设横坐标为 f_1 ，纵坐标为 f_2 ，绘制图像如 22 所示。图中，强 Pareto 解落在红色实线上，而弱 Pareto 解落在蓝色虚线上

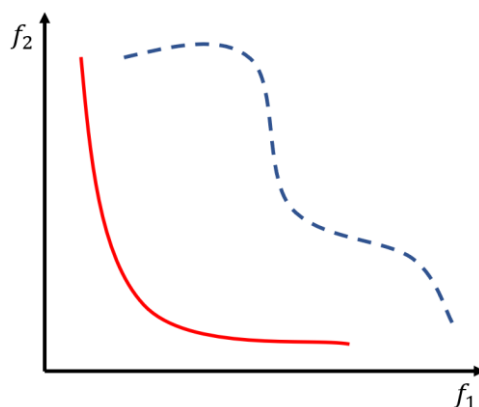


图 22 强 Pareto 解和弱 Pareto 解

1995 年, K Deb 等首次提出非支配遗传算法(Non-dominated Sorting Genetic Algorithms, NSGA)。非支配遗传算法是以多目标规划的 Pareto 最优概念为基础, 并在遗传算法上进行改进所得到的。与遗传算法不同的是, NSGA 算法对种群的个体进行了快速非支配排序, 增大了个体被保留的概率, 其余的染色体交叉、选择、变异等操作与标准遗传算法并无差别。经过几十年的测试, 已经有多位学者证明, NSGA 算法在多目标规划方面比遗传算法的性能更好。

在多目标规划中, 在节 5.3.5 中使用过的精英策略是一种常见的优化方法。其旨在通过提高较优秀个体的保留概率, 加快程序的运行速度和寻优效率。而 NSGA 并未使用这种方法。若设置目标函数个数为 n , 种群数量为 N , 则 NSGA 的时间复杂度为 $O(nN^3)$ 。基于此, K Deb^[9]等人在 2002 年对 NSGA 作出修改, 提出一种全新的 NSGA-II 算法。NSGA-II 不仅使用精英策略, 而且采用拥挤度作为选择优秀个体的标准, 保证了种群中的个体多样性, 且将时间复杂度降低为 $O(nN^2)$ 。2013 年, 在 NSGA-II 的框架基础上, 为在处理四个或以上的目标函数的问题上取得更好的性能, K Deb^[10]等提出了一种基于参考点的多目标进化算法, 即 NSGA-III。该算法强调非支配, 但也更偏向接近一组提供的参考点的种群成员。与 EMO 算法等方法进行比较, NSGA-III 对几乎所有问题都产生了令人满意的结果。一般来说, 基于 Pareto 的多目标进化算法的算法流程都较为统一, 如图 23 所示。

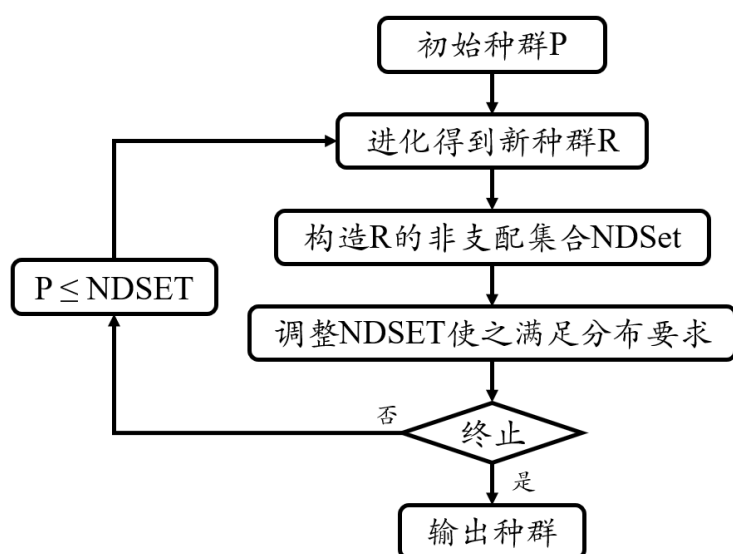


图 23 基于 Pareto 的多目标进化算法的流程图

在对比了多种算法的性能基础上, 我们主要选择使用 NSGA-III 进行规划和求解。

NSGA-III依托参考点进行工作，而不再是 NSGA- II 所依托的拥挤度。参考点的目的就是获取种群的个体与相应参考点之间的映射关系，即距离，进而使得种群向着更加接近参考点的方向进化，并让参考点的分布更加均匀。NSGA-III的伪代码如下表 16 所示。

表 16 NSGA-III的伪代码

Algorithm: NSGA-III
输入: 参考点 Z^s 或期望点 Z^a , 父代 P_t
输出: 子代 P_{t+1}
1: $S_t = \emptyset, i = 1$, 即设置存储集合
2: $Q_t = \text{交叉} + \text{变异}(P_t)$, 即利用交叉与变异产生新的群体
3: $R_t = P_t \cup Q_t$, 即获取父代集合
4: $(F_1, F_2, \dots) = \text{非支配排序}(R_t)$, 即利用非支配排序作等级划分
5: repeat
6: $S_t = S_t \cup F_i \ \& \ i = i + 1$, 即不断将等级划分后的个体排序, 低等级的存入 S_t 之中, 直到 S_t 的个数大于种群数 N
7: until $ S_t \geq N$
8: 最后一个前沿假设: $F_l = F_i$
9: if $ S_t = N$ then
10: $P_{t+1} = S_t$, break , 即若 $ S_t = N$, 则下一轮迭代种群 $P_{t+1} = S_t$, 否则开始基于参考点的操作
11: else
12: $P_{t+1} = \bigcup_{j=1}^{l-1} F_j$
13: $F_l: K = N - P_{t+1} $
14: 标准化目标函数 f^n 并且创造参考点集合 (S_t, Z^r, Z^s, Z^a)
15: 将种群 S_t 中的每一个个体 s 和一个参考点组合, 创建映射关系 $[\pi(s), d(s)]$, 其中, $\pi(s)$ 是距离种群个体最近的参考点, $d(s)$ 是参考点和种群个体之间的距离。
16: 计算参考点的生态位计数 $j \in Z^r: \rho_j = \sum_{s \in S_t/F_l} (\pi(s) = j)$
17: 选择生态位较少的参考点, 并从 F_l 前沿中寻找 K 个种群个体。
18: end if

根据题目, 目标函数共有两个, 即最大化过滤效率的同时最小化过滤阻力。为方便绘图表示, 将最大化的公式添加负号, 转为最小化。因此, 我们的目标函数如式(28)所示。由接受距离不可能大于 100cm, 热风速度不大于 2000r/min, 厚度不大于 3mm 且压缩回弹性不低于 85%, 约束条件如式(29)所示。

$$\begin{aligned} \text{Min } x_7 &= -F(x_3, x_4, x_5) = -F(g(x_1, x_2), h(x_1, x_2), z(x_1, x_2)) \\ \text{Min } x_6 &= G(x_3, x_4, x_5) = G(g(x_1, x_2), h(x_1, x_2), z(x_1, x_2)) \end{aligned} \quad (28)$$

式中: F, G, g, h, z 分别表示以 x_7, x_6, x_3, x_4, x_5 为因变量的模型。为方便表示, 并未给出多次项和交互项特征。

$$s. t. \begin{cases} 0 < x_1 < 100 \\ 0 < x_2 < 2000 \\ 0 < x_3 \leq 3 \\ x_4 > 0 \\ x_5 \geq 85 \end{cases} \quad (29)$$

5.4.2 模型求解

设置 NSGA-III 算法的种群个数为 50，迭代次数为 100。同时，设置式(9)中的 $g(x_1, x_2), h(x_1, x_2), z(x_1, x_2), F(x_3, x_4, x_5), G(x_3, x_4, x_5)$ 分别为第二问和第三问中所得到的拟合性能最佳的 CatBoost, CatBoost, XGBoost, XGBoost, XGBoost 模型。在经历变异、重组等过程后，种群的变化如图 24 所示。在图中，左上角是初始化的种群，即父本的分布。右上角是进化 5 次后的种群，可以看到种群已经有明显的移动趋势。左下角和右下角分别是进化 50 次和 100 次后的种群。此时所有的个体几乎都拥挤在 3 个点上，一方面，这是因为机器学习的拟合性能虽然极佳，但是树模型的外推能力较差，在样本较少的情况下，无法向外衍生；另一方面，这也说明满足约束条件的同时，还能保证目标函数性能的点较少。

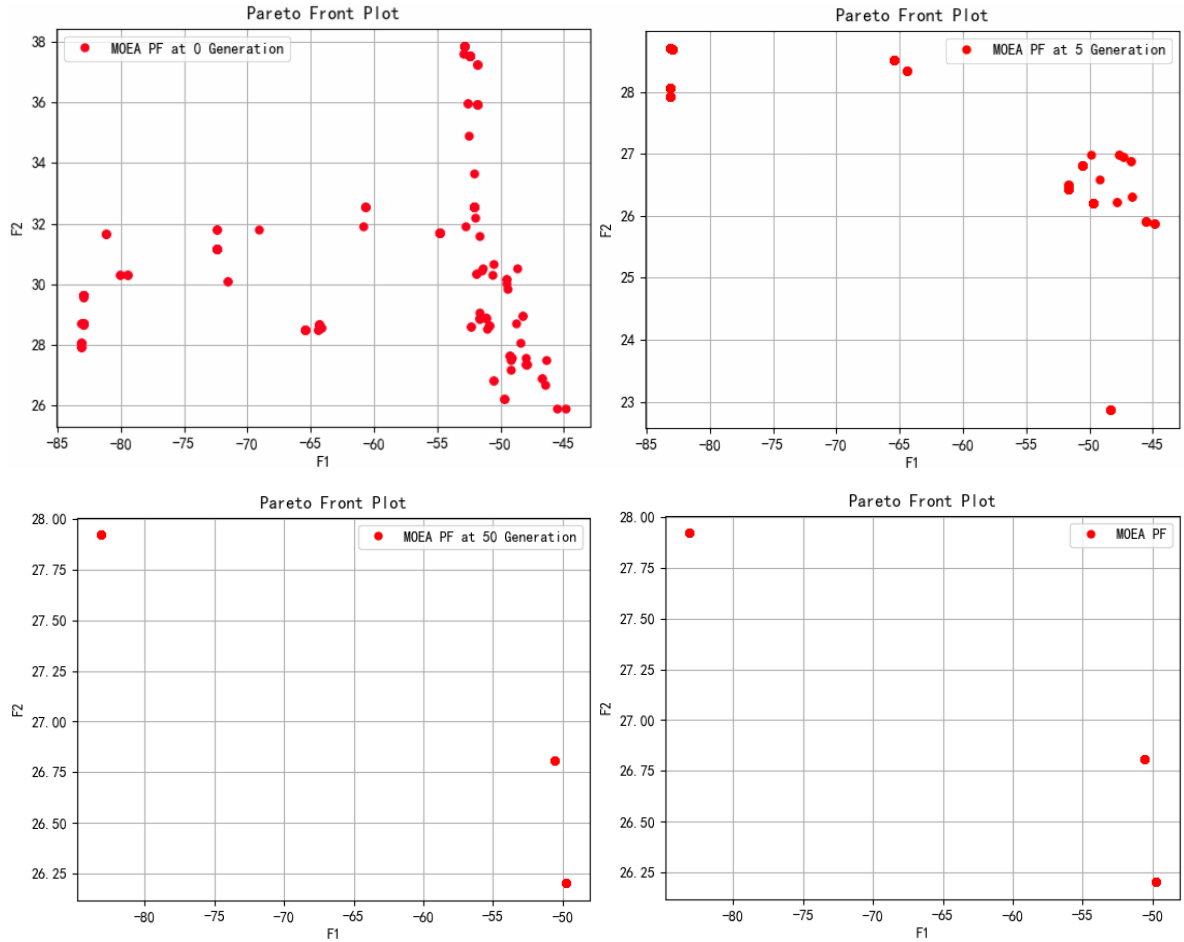


图 24 NSGA-III 算法的寻优过程

图 24 中的五个点的坐标，从左到右分别为 $(-83.17, 27.92)$, $(-50.57, 26.81)$ 和 $(-49.73, 26.20)$ 。参考强 Pareto 解和弱 Pareto 解分布，可以发现 $(-83.17, 27.92)$ 和 $(-49.73, 26.20)$ 类似于强 Pareto 解曲线的最上端和最下端，因此最优的解可能是他们俩中的一个。对比横纵坐标的范围，可以发现在 f_2 仅仅增加 1.52 的情况下， f_1 减少了 33.44，变化幅度为 f_2 的 2786.67%。因此，我们选择左上角的点作为最优解。此时过滤效率为 83.17%，

过滤阻力位 27.92Pa。处在此最优解的工艺性能参数共 17 个点，如表所示，接受距离处在区间[11.32, 18.28]内，均值为 13.41，而热风速度处在[1192.32, 1964.71]之间，均值为 1666.05。

表 17 满足最优解的工艺参数

样本	接受距离 cm	热风速度 r/min
1	12.942578997802734	1656.127864074707
2	11.667421300539662	1850.625222935505
3	11.667421300539662	1850.625222935505
4	11.670223505749608	1838.0185542778843
5	18.275273392397196	1192.3243567590634
6	11.670223505749608	1838.0185542778843
7	11.322752527184013	1964.7146419927758
8	17.7723686957491	1195.6241066882906
9	11.462575435191406	1917.7831497643563
10	12.942578997802734	1656.127864074707
11	11.667421300539662	1845.2536905248683
12	18.275273392397196	1192.3243567590634
13	11.322752527184013	1964.7146419927758
14	11.682757623962654	1838.0185542778843
15	11.667421300539662	1800.9470863021513
16	14.353185828610435	1525.9988576496223
17	17.683865218269286	1195.6306050569328

最后，为证明选择 CatBoost 与 XGBoost 模型作为函数的必要性和正确性，同样再以第二问和第三问中得到的回归方程作为式(27)中的 $g(x_1, x_2), h(x_1, x_2), z(x_1, x_2), F(x_3, x_4, x_5), G(x_3, x_4, x_5)$ 。算法迭代失败，无法求解，没有可以满足条件的种群。这也证明了特征之间的非线性关系显著，简单的线性关系无法表示特征之间复杂的关系，使用机器学习模型作为预测函数是正确且准确的。

六、模型的评价、缺点与推广

6.1 模型的优点

1. 在探究变量关系之前，我们利用统计检验和相关分析等方法，优先判断变量之间的线性关系的可能性。发现变量之间可能存在的交互性和曲线关系后，我们进行特征衍生，添加了多项式高次项和交互项因子。
2. 在探究工艺参数、结构变量和产品性能之间的关系时，我们不仅对整体的“变量组”之间的关系进行研究，而且对“组内变量”的个体之间的关系进行研究，充分考虑了特征之间所有可能的潜在关系。
3. 在探究变量个体关系时，我们将关系分为非线性和线性进行分别讨论。研究线性关系时，我们利用统计模型的多元标准化逐步回归法，以标准化系数作为指标直观反映线性关系；研究非线性关系时，我们利用可解释性较强的集成树模型，利用拟合能力更强的树模型所输出的特征重要性作为指标反映非线性的关系。
4. 在进行单目标和双目标寻优时，我们没有使用较为简单的进化算法，而是对

基础的遗传算法进行改进。针对选择、交叉和变异算子，分别选用增强保留的遗传算法进行单目标寻优，以及基于参考点的 NSGA-III 算法进行多目标寻优。寻优的效率更快，时间复杂度更小。

5. 使用进化算法进行寻优时，我们不仅选用了拟合能力更强、表达变量非线性关系的机器学习模型作为目标函数，而且对比了拟合能力更强较弱，表达线性关系的回归方程作为目标函数。将两者的结果进行比较，突出变量间非线性关系更强的特点，以及选用机器学习模型的必要性和正确性。

6.2 模型的缺点

1. 在利用回归模型寻找变量之间的关系时，由于时间原因，只考虑了简单的线性模型，而未考虑可能存在的指数、对数等非线性关系。

6.3 模型的推广

1. 本文所给的数据较为复杂，但数据样本量较少，故而在模型的拟合上，机器学习的性能明显大于经典统计模型。当面对实际情况中更为复杂的数据，应当更加重视回归方程的构建，利用指数函数、幂函数、对数函数等方法添加非线性关系
2. 近年来机器学习和强化学习的不断发展，强化学习模型在寻优和决策中表现出了较好的性能。因此，在实际应用中，不仅可以使启发性的进化算法，而且可以考虑强化学习模型等方法。

七、参考文献

- [1]. 韩玲,胡梦缘,马英博,郝栋连.医用非织造口罩材料及其新技术的研究现状[J].西安工程大学学报,2020,34(02):20-25.DOI:10.13338/j.issn.1674-649x.2020.02.003
- [2]. 程可为,刘亚,于雯,赵义侠.新型熔喷非织造材料研究进展[J].纺织导报,2021(12):61-66.DOI:10.16481/j.cnki.ctl.2021.12.015
- [3]. Breiman L. Random forests[J]. Machine learning, 2001, 45(1): 5-32.
- [4]. Freund Y, Schapire R E. Experiments with a new boosting algorithm[C]//icml. 1996, 96: 148-156.
- [5]. Friedman J H. Greedy function approximation: a gradient boosting machine[J]. Annals of statistics, 2001: 1189-1232.
- [6]. Chen T, Guestrin C. Xgboost: A scalable tree boosting system[C]//Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining. 2016: 785-794.
- [7]. Ke G, Meng Q, Finley T, et al. Lightgbm: A highly efficient gradient boosting decision tree[J]. Advances in neural information processing systems, 2017, 30.
- [8]. Dorogush A V, Ershov V, Gulin A. CatBoost: gradient boosting with categorical features support[J]. arXiv preprint arXiv:1810.11363, 2018.
- [9]. Deb K, Pratap A, Agarwal S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. IEEE transactions on evolutionary computation, 2002, 6(2): 182-197.
- [10]. Deb K, Jain H. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints[J]. IEEE transactions on evolutionary computation, 2013, 18(4): 577-601.

附录

Python

```
import pandas as pd
import numpy as np
import seaborn as sns
from matplotlib import pyplot as plt
from matplotlib import cm
from pylab import *
from matplotlib.font_manager import FontProperties
from matplotlib.collections import LineCollection
from mpl_toolkits.axes_grid1.inset_locator import mark_inset
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False

df = pd.read_csv("D:/3、竞赛/2022.08.04 “华数杯” 全国大学生数学建模竞赛
/data3.csv")

""" ===== 问题二 ===== """
""" ===== 正态性检验 ===== """
from scipy.stats import kstest
print("x1 的正态性检验的结果为: " + str(kstest(df["x1"], cdf = "norm")))
print("x2 的正态性检验的结果为: " + str(kstest(df["x2"], cdf = "norm")))
print("x3 的正态性检验的结果为: " + str(kstest(df["x3"], cdf = "norm")))
print("x4 的正态性检验的结果为: " + str(kstest(df["x4"], cdf = "norm")))
print("x5 的正态性检验的结果为: " + str(kstest(df["x5"], cdf = "norm")))
print("x6 的正态性检验的结果为: " + str(kstest(df["x6"], cdf = "norm")))
print("x7 的正态性检验的结果为: " + str(kstest(df["x7"], cdf = "norm")))
print("x8 的正态性检验的结果为: " + str(kstest(df["x8"], cdf = "norm")))

""" ===== 三维散点图 ===== """
p = plt.subplot(projection = "3d")
p.scatter(df["x1"], df["x2"], df["x3"],
          c = "deepskyblue", marker='*',
          label = "厚度")
p.scatter(df["x1"], df["x2"], df["x4"],
          c = "violet", marker='+',
          label = "孔隙率")
p.scatter(df["x1"], df["x2"], df["x5"],
          c = "darkorange", marker='^',
          label = "压缩回弹性")
p.set_xlabel("接受距离", fontdict={'family': 'KaiTi', 'size': 15},
```

```

rotation = -25)
p.set_ylabel("热空气速度", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("结构变量", fontdict={'family': 'KaiTi', 'size': 15},
rotation = -92)
plt.legend()

""" ===== spearman 相关热力图 ===== """
spearman_data = pd.DataFrame(df.iloc[:, :5]).corr(method='spearman')
ax = sns.heatmap(spearman_data,
                  vmax = 1,
                  cmap = 'coolwarm',
                  annot = True)
ax.set_title('热力相关图 (Spearman)', fontsize = 15)
plt.show()

from sklearn import metrics

""" ===== dataq2 ===== """
dataq2 = pd.DataFrame()
dataq2["x1"] = df["x1"]
dataq2["x2"] = df["x2"]
dataq2["x1^2"] = np.array(df["x1"]) * np.array(df["x1"])
dataq2["x2^2"] = np.array(df["x2"]) * np.array(df["x2"])
dataq2["x1^3"] = np.array(df["x1"]) * np.array(df["x1"]) *
np.array(df["x1"])
dataq2["x2^3"] = np.array(df["x2"]) * np.array(df["x2"]) *
np.array(df["x2"])
dataq2["x1x2"] = np.array(df["x1"]) * np.array(df["x2"])
dataq2["x3"] = df["x3"]
dataq2["x4"] = df["x4"]
dataq2["x5"] = df["x5"]

dataq2_ins = pd.DataFrame()
dataq2_ins["x3"] = df["x3"]
dataq2_ins["x4"] = df["x4"]
dataq2_ins["x5"] = df["x5"]
dataq2_ins["x3^2"] = np.array(df["x3"]) * np.array(df["x3"])
dataq2_ins["x4^2"] = np.array(df["x4"]) * np.array(df["x4"])
dataq2_ins["x5^2"] = np.array(df["x5"]) * np.array(df["x5"])
dataq2_ins["x3^3"] = np.array(df["x3"]) * np.array(df["x3"]) *
np.array(df["x3"])
dataq2_ins["x4^3"] = np.array(df["x4"]) * np.array(df["x4"]) *
np.array(df["x4"])

```

```

dataq2_ins["x5^3"] = np.array(df["x5"]) * np.array(df["x5"]) *
np.array(df["x5"])
dataq2_ins["x3x4"] = np.array(df["x3"]) * np.array(df["x4"])
dataq2_ins["x4x5"] = np.array(df["x4"]) * np.array(df["x5"])
dataq2_ins["x3x5"] = np.array(df["x3"]) * np.array(df["x5"])
dataq2_ins["x3x4x5"] = np.array(df["x3"]) * np.array(df["x4"]) *
np.array(df["x5"])
dataq2_ins["x1"] = df["x1"]
dataq2_ins["x2"] = df["x2"]

d2 = pd.concat([dataq2.iloc[:, 0:7], dataq2_ins.iloc[:, 0:13]], axis =
1)
d2.to_csv("D:/3、竞赛/2022.08.04 “华数杯”全国大学生数学建模竞赛
/dataq2.csv")

""" ===== 核密度曲线 ===== """
plt.figure()
plt.subplot(2, 4, 1)
sns.kdeplot(df["x1"], color = "red", label = "接受距离", shade =
True).set(xlabel = None, ylabel = None)
plt.legend()
plt.subplot(2, 4, 2)
sns.kdeplot(df["x2"], color = "blue", label = "热空气速度", shade =
True).set(xlabel = None, ylabel = None)
plt.legend()
plt.subplot(2, 4, 3)
sns.kdeplot(df["x3"], color = "green", label = "厚度", shade =
True).set(xlabel = None, ylabel = None)
plt.legend()
plt.subplot(2, 4, 4)
sns.kdeplot(df["x4"], color = "violet", label = "孔隙率", shade =
True).set(xlabel = None, ylabel = None)
plt.legend()
plt.subplot(2, 4, 5)
sns.kdeplot(df["x5"], color = "orange", label = "压缩回弹性", shade =
True).set(xlabel = None, ylabel = None)
plt.legend()
plt.subplot(2, 4, 6)
sns.kdeplot(df["x6"], color = "deepskyblue", label = "过滤阻力", shade =
True).set(xlabel = None, ylabel = None)
plt.legend()
plt.subplot(2, 4, 7)
sns.kdeplot(df["x7"], color = "chocolate", label = "过滤效率", shade =
True).set(xlabel = None, ylabel = None)

```

```

plt.legend()
plt.subplot(2, 4, 8)
sns.kdeplot(df["x8"], color = "darkorchid", label = "透气性", shade =
True).set(xlabel = None, ylabel = None)
plt.legend()

""" ===== 随机森林 ===== """
def RF(train_x, train_y, n_estimators = 300, random_state = 317):
    from sklearn.ensemble import RandomForestRegressor
    model = RandomForestRegressor(n_estimators = n_estimators,
                                random_state = random_state)

    model.fit(train_x, train_y)
    return model

""" ===== AdaBoost ===== """
def AdaBoost(train_x, train_y, n_estimators = 300,
             learning_rate = 0.1, random_state = 317):
    from sklearn.ensemble import AdaBoostRegressor
    model = AdaBoostRegressor(n_estimators = n_estimators,
                              learning_rate = learning_rate,
                              random_state = random_state)

    model.fit(train_x, train_y)
    return model

""" ===== GBDT ===== """
def GBDT(train_x, train_y, learning_rate = 0.1,
         n_estimators = 300, random_state = 317):
    from sklearn.ensemble import GradientBoostingRegressor
    model = GradientBoostingRegressor(learning_rate = learning_rate,
                                      n_estimators = n_estimators,
                                      random_state = random_state)

    model.fit(train_x, train_y)
    return model

""" ===== XGB ===== """
def XGBoost(train_x, train_y, n_estimators = 300, seed = 317):
    import xgboost as xgb
    from xgboost import plot_importance
    model = xgb.XGBRegressor(silent = 0,
                             n_estimators = n_estimators,
                             seed = seed)

    model.fit(train_x, train_y,
              eval_metric = ["mae", "rmse"],
              eval_set = [(train_x, train_y)],

```

```

        verbose = True)

    """ === 性能度量 === """
    results = model.evals_result()
    epochs = n_estimators
    x_axis = range(0, epochs)
    plt.figure()
    plt.plot(x_axis, results['validation_0']['mae'],
             label = "MAE is %5f" %results['validation_0']['mae'][-1])
    plt.plot(x_axis, results['validation_0']['rmse'],
             label = "RMSE is %5f" %results['validation_0']['rmse'][-1])
    plt.ylabel("Score", fontsize = 15)
    plt.xlabel("Epochs", fontsize = 15)
    plt.legend()

    """ ===== 特征重要性 ===== """
    plot_importance(model)
    return model

    """ ===== light ===== """
def LGBM(train_x, train_y, n_estimators = 3000, random_state = 317):
    import lightgbm as lgb
    model = lgb.LGBMRegressor(n_estimators = n_estimators,
                              random_state = random_state)
    model.fit(train_x, train_y,
              eval_metric = ["mae", "RMSE"],
              eval_set = [(train_x, train_y)])
    lgb.plot_importance(model, max_num_features = 30)
    return model

    """ ===== cat ===== """
def CatBoost(train_x, train_y):
    import catboost as cat
    train_set = cat.Pool(data = train_x,
                          label = train_y)
    model = cat.CatBoostRegressor(iterations = 300,
                                   random_seed = 317)

    model.fit(train_set)
    model.plot_tree(tree_idx = 0, pool = train_set)
    return model

    """ ===== x1 ===== """
from sklearn import metrics

```

```

model_rf_x1 = RF(dataq2_ins.iloc[:, 0:13], dataq2_ins["x1"])
pre_rf_x1 = model_rf_x1.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x1"]),
pre_rf_x1)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x1"]), pre_rf_x1)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x1"]),
pre_rf_x1))))
model_ada_x1 = AdaBoost(dataq2_ins.iloc[:, 0:13], dataq2_ins["x1"])
pre_ada_x1 = model_ada_x1.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x1"]),
pre_ada_x1)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x1"]),
pre_ada_x1)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x1"]),
pre_ada_x1))))
model_gbdx1 = GBDT(dataq2_ins.iloc[:, 0:13], dataq2_ins["x1"])
pre_gbdx1 = model_gbdx1.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x1"]),
pre_gbdx1)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x1"]),
pre_gbdx1)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x1"]),
pre_gbdx1))))
model_xgb_x1 = XGBoost(dataq2_ins.iloc[:, 0:13], dataq2_ins["x1"])
pre_xgb_x1 = model_xgb_x1.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x1"]),
pre_xgb_x1)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x1"]),
pre_xgb_x1)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x1"]),
pre_xgb_x1))))
model_lgbm_x1 = LGBM(dataq2_ins.iloc[:, 0:13], dataq2_ins["x1"])
pre_lgbm_x1 = model_lgbm_x1.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x1"]),
pre_lgbm_x1)))
print("MAE:      " +

```

```

str(metrics.mean_absolute_error(np.array(dataq2_ins["x1"]),
pre_lgbm_x1)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x1"]),
pre_lgbm_x1))))
model_cat_x1 = CatBoost(dataq2_ins.iloc[:, 0:13], dataq2_ins["x1"])
pre_cat_x1 = model_cat_x1.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x1"]),
pre_cat_x1)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x1"]),
pre_cat_x1)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x1"]),
pre_cat_x1))))

""" ===== x2 ===== """
model_rf_x2 = RF(dataq2_ins.iloc[:, 0:13], dataq2_ins["x2"])
pre_rf_x2 = model_rf_x2.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x2"]),
pre_rf_x2)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x2"]), pre_rf_x2)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x2"]),
pre_rf_x2))))
model_ada_x2 = AdaBoost(dataq2_ins.iloc[:, 0:13], dataq2_ins["x2"])
pre_ada_x2 = model_ada_x2.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x2"]),
pre_ada_x2)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x2"]),
pre_ada_x2)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x2"]),
pre_ada_x2))))
model_gbdx2 = GBDT(dataq2_ins.iloc[:, 0:13], dataq2_ins["x2"])
pre_gbdx2 = model_gbdx2.predict(dataq2_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq2_ins["x2"]),
pre_gbdx2)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x2"]),
pre_gbdx2)))
print("RMSE:                                " +

```

```

str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x2"]),
pre_gbdx2))))
model_xgb_x2 = XGBoost(dataq2_ins.iloc[:, 0:13], dataq2_ins["x2"])
pre_xgb_x2 = model_xgb_x2.predict(dataq2_ins.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq2_ins["x2"]),
pre_xgb_x2)))
print("MAE: " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x2"]),
pre_xgb_x2)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x2"]),
pre_xgb_x2))))
model_lgbm_x2 = LGBM(dataq2_ins.iloc[:, 0:13], dataq2_ins["x2"])
pre_lgbm_x2 = model_lgbm_x2.predict(dataq2_ins.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq2_ins["x2"]),
pre_lgbm_x2)))
print("MAE: " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x2"]),
pre_lgbm_x2)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x2"]),
pre_lgbm_x2))))
model_cat_x2 = CatBoost(dataq2_ins.iloc[:, 0:13], dataq2_ins["x2"])
pre_cat_x2 = model_cat_x2.predict(dataq2_ins.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq2_ins["x2"]),
pre_cat_x2)))
print("MAE: " +
str(metrics.mean_absolute_error(np.array(dataq2_ins["x2"]),
pre_cat_x2)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2_ins["x2"]),
pre_cat_x2))))

""" ===== x3 ===== """
model_rf_x3 = RF(dataq2.iloc[:, 0:7], dataq2["x3"])
pre_rf_x3 = model_rf_x3.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x3"]), pre_rf_x3)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x3"]),
pre_rf_x3)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x3"]),
pre_rf_x3))))
model_ada_x3 = AdaBoost(dataq2.iloc[:, 0:7], dataq2["x3"])
pre_ada_x3 = model_ada_x3.predict(dataq2.iloc[:, 0:7])

```

```

print("R2:      " + str(metrics.r2_score(np.array(dataq2["x3"]),
pre_ada_x3)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x3"]),
pre_ada_x3)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x3"]),
pre_ada_x3))))
model_gbdx3 = GBDT(dataq2.iloc[:, 0:7], dataq2["x3"])
pre_gbdx3 = model_gbdx3.predict(dataq2.iloc[:, 0:7])
print("R2:      " + str(metrics.r2_score(np.array(dataq2["x3"]),
pre_gbdx3)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x3"]),
pre_gbdx3)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x3"]),
pre_gbdx3))))
model_xgbx3 = XGBoost(dataq2.iloc[:, 0:7], dataq2["x3"])
pre_xgbx3 = model_xgbx3.predict(dataq2.iloc[:, 0:7])
print("R2:      " + str(metrics.r2_score(np.array(dataq2["x3"]),
pre_xgbx3)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x3"]),
pre_xgbx3)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x3"]),
pre_xgbx3))))
model_lgbx3 = LGBM(dataq2.iloc[:, 0:7], dataq2["x3"])
pre_lgbx3 = model_lgbx3.predict(dataq2.iloc[:, 0:7])
print("R2:      " + str(metrics.r2_score(np.array(dataq2["x3"]),
pre_lgbx3)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x3"]),
pre_lgbx3)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x3"]),
pre_lgbx3))))
model_catx3 = CatBoost(dataq2.iloc[:, 0:7], dataq2["x3"])
pre_catx3 = model_catx3.predict(dataq2.iloc[:, 0:7])
print("R2:      " + str(metrics.r2_score(np.array(dataq2["x3"]),
pre_catx3)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x3"]),
pre_catx3)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x3"]),
pre_catx3))))

```

```

""" ===== x4 ===== """
model_rf_x4 = RF(dataq2.iloc[:, 0:7], dataq2["x4"])
pre_rf_x4 = model_rf_x4.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x4"]), pre_rf_x4)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x4"]),
pre_rf_x4)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x4"]),
pre_rf_x4))))
model_ada_x4 = AdaBoost(dataq2.iloc[:, 0:7], dataq2["x4"])
pre_ada_x4 = model_ada_x4.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x4"]),
pre_ada_x4)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x4"]),
pre_ada_x4)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x4"]),
pre_ada_x4))))
model_gbdt_x4 = GBDT(dataq2.iloc[:, 0:7], dataq2["x4"])
pre_gbdt_x4 = model_gbdt_x4.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x4"]),
pre_gbdt_x4)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x4"]),
pre_gbdt_x4)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x4"]),
pre_gbdt_x4))))
model_xgb_x4 = XGBoost(dataq2.iloc[:, 0:7], dataq2["x4"])
pre_xgb_x4 = model_xgb_x4.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x4"]),
pre_xgb_x4)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x4"]),
pre_xgb_x4)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x4"]),
pre_xgb_x4))))
model_lgbm_x4 = LGBM(dataq2.iloc[:, 0:7], dataq2["x4"])
pre_lgbm_x4 = model_lgbm_x4.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x4"]),
pre_lgbm_x4)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x4"]),
pre_lgbm_x4)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x4"]),

```

```

pre_lgbm_x4)))
model_cat_x4 = CatBoost(dataq2.iloc[:, 0:7], dataq2["x4"])
pre_cat_x4 = model_cat_x4.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x4"]),
pre_cat_x4)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x4"]),
pre_cat_x4)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x4"]),
pre_cat_x4))))

""" ===== x5 ===== """
model_rf_x5 = RF(dataq2.iloc[:, 0:7], dataq2["x5"])
pre_rf_x5 = model_rf_x5.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x5"]), pre_rf_x5)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x5"]),
pre_rf_x5)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x5"]),
pre_rf_x5))))
model_ada_x5 = AdaBoost(dataq2.iloc[:, 0:7], dataq2["x5"])
pre_ada_x5 = model_ada_x5.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x5"]),
pre_ada_x5)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x5"]),
pre_ada_x5)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x5"]),
pre_ada_x5))))
model_gbdx5 = GBDT(dataq2.iloc[:, 0:7], dataq2["x5"])
pre_gbdx5 = model_gbdx5.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x5"]),
pre_gbdx5)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x5"]),
pre_gbdx5)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x5"]),
pre_gbdx5))))
model_xgb_x5 = XGBoost(dataq2.iloc[:, 0:7], dataq2["x5"])
pre_xgb_x5 = model_xgb_x5.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x5"]),
pre_xgb_x5)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x5"]),
pre_xgb_x5)))

```

```

print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x5"]),
pre_xgb_x5))))
model_lgbm_x5 = LGBM(dataq2.iloc[:, 0:7], dataq2["x5"])
pre_lgbm_x5 = model_lgbm_x5.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x5"]),
pre_lgbm_x5)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x5"]),
pre_lgbm_x5)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x5"]),
pre_lgbm_x5))))
model_cat_x5 = CatBoost(dataq2.iloc[:, 0:7], dataq2["x5"])
pre_cat_x5 = model_cat_x5.predict(dataq2.iloc[:, 0:7])
print("R2: " + str(metrics.r2_score(np.array(dataq2["x5"]),
pre_cat_x5)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq2["x5"]),
pre_cat_x5)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq2["x5"]),
pre_cat_x5))))

""" ===== 预测值和正确值的比较 ===== """
ax = plt.subplot(1, 1, 1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x1"], label = "Origin", linestyle = "-", linewidth =
2)
ax.plot(x_axis, pre_rf_x1, label = "RandomForest", linestyle = "-.",
linewidth = 2)
ax.plot(x_axis, pre_ada_x1, label = "AdaBoost", linestyle = ":", linewidth
= 2)
ax.plot(x_axis, pre_gbdx1, label = "GBDT", linestyle = "--", linewidth
= 2)
ax.plot(x_axis, pre_xgb_x1, label = "XGBoost", linestyle = "--", linewidth
= 2)
ax.plot(x_axis, pre_lgbm_x1, label = "LightGBM", linestyle = "-.",
linewidth = 2)
ax.plot(x_axis, pre_cat_x1, label = "CatBoost", linestyle = ":", linewidth
= 2)
plt.ylabel("接受距离", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad =
2)
plt.title("接受距离的真实值与预测值", fontdict={'family': 'KaiTi',
'size': 20})

```

```

plt.legend(loc = "upper right")
axins = ax.inset_axes((0.07, 0.07, 0.33, 0.3))
axins.plot(x_axis, df["x1"], linestyle = "-", linewidth = 5)
axins.plot(x_axis, pre_rf_x1, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x1, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx1, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x1, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x1, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x1, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 20
xlim1 = 30
# Y 轴的显示范围
ylim0 = 32
ylim1 = 37
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 2, loc2 = 4)

ax = plt.subplot(1, 1, 1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x2"], label = "Origin", linestyle = "-", linewidth = 2)
ax.plot(x_axis, pre_rf_x2, label = "RandomForest", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x2, label = "AdaBoost", linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx2, label = "GBDT", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x2, label = "XGBoost", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x2, label = "LightGBM", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x2, label = "CatBoost", linestyle = ":", linewidth = 2)
plt.ylabel("热风速度", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("热风速度的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})

```

```

plt.legend(loc = "upper right")
axins = ax.inset_axes((0.07, 0.07, 0.33, 0.3))
axins.plot(x_axis, df["x2"], linestyle = "-", linewidth = 5)
axins.plot(x_axis, pre_rf_x2, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x2, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx2, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x2, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x2, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x2, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 25
xlim1 = 32
# Y 轴的显示范围
ylim0 = 1150
ylim1 = 1220
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 2, loc2 = 4)

ax = plt.subplot(1, 1, 1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x3"], label = "Origin", linestyle = "-", linewidth = 2)
ax.plot(x_axis, pre_rf_x3, label = "RandomForest", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x3, label = "AdaBoost", linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx3, label = "GBDT", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x3, label = "XGBoost", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x3, label = "LightGBM", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x3, label = "CatBoost", linestyle = ":", linewidth = 2)
plt.ylabel("厚度", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("厚度的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})

```

```

plt.legend(loc = "upper right")
axins = ax.inset_axes((0.07, 0.07, 0.33, 0.3))
axins.plot(x_axis, df["x3"], linestyle = "-", linewidth = 5)
axins.plot(x_axis, pre_rf_x3, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x3, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx3, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x3, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x3, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x3, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 20
xlim1 = 30
# Y 轴的显示范围
ylim0 = 2.75
ylim1 = 3.30
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 2, loc2 = 4)

ax = plt.subplot(1, 1, 1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x4"], label = "Origin", linestyle = "-", linewidth = 2)
ax.plot(x_axis, pre_rf_x4, label = "RandomForest", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x4, label = "AdaBoost", linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx4, label = "GBDT", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x4, label = "XGBoost", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x4, label = "LightGBM", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x4, label = "CatBoost", linestyle = ":", linewidth = 2)
plt.ylabel("孔隙率", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("孔隙率的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})

```

```

plt.legend(loc = "upper right")
axins = ax.inset_axes((0.08, 0.08, 0.33, 0.3))
axins.plot(x_axis, df["x4"], linestyle = "-", linewidth = 5)
axins.plot(x_axis, pre_rf_x4, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x4, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx4, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x4, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x4, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x4, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 20
xlim1 = 30
# Y 轴的显示范围
ylim0 = 96
ylim1 = 97
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 2, loc2 = 1)

ax = plt.subplot(1, 1, 1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x5"], label = "Origin", linestyle = "-", linewidth = 2)
ax.plot(x_axis, pre_rf_x5, label = "RandomForest", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x5, label = "AdaBoost", linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx5, label = "GBDT", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x5, label = "XGBoost", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x5, label = "LightGBM", linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x5, label = "CatBoost", linestyle = ":", linewidth = 2)
plt.ylabel("压缩回弹性", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("压缩回弹性的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})

```

```

plt.legend(loc = "upper right")
axins = ax.inset_axes((0.5, 0.1, 0.4, 0.3))
axins.plot(x_axis, df["x5"], linestyle = "-", linewidth = 5)
axins.plot(x_axis, pre_rf_x5, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x5, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx5, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x5, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x5, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x5, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 20
xlim1 = 30
# Y 轴的显示范围
ylim0 = 86
ylim1 = 88
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 3, loc2 = 1)

""" ===== 特征重要性 ===== """
rf_imp = model_rf_x1.feature_importances_ /
sum(model_rf_x1.feature_importances_)
ada_imp = model_ada_x1.feature_importances_ /
sum(model_ada_x1.feature_importances_)
gbdx_imp = model_gbdx_x1.feature_importances_ /
sum(model_gbdx_x1.feature_importances_)
xgb_imp = model_xgb_x1.feature_importances_ /
sum(model_xgb_x1.feature_importances_)
lgbm_imp = model_lgbm_x1.feature_importances_ /
sum(model_lgbm_x1.feature_importances_)
cat_imp = model_cat_x1.feature_importances_ /
sum(model_cat_x1.feature_importances_)
labels = ["x3", "x4", "x5", "x3^2", "x4^2", "x5^2", "x3^3", "x4^3",
"x5^3",
"x3:x4", "x4:x5", "x3:x5", "x3:x4:x5"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,
20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")

```

```

r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbd_t_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测接受距离的特征重要性",
             fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbd_t_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x2.feature_importances_          /
sum(model_rf_x2.feature_importances_)
ada_imp          =          model_ada_x2.feature_importances_          /
sum(model_ada_x2.feature_importances_)
gbd_t_imp        =          model_gbd_t_x2.feature_importances_          /
sum(model_gbd_t_x2.feature_importances_)
xgb_imp          =          model_xgb_x2.feature_importances_          /
sum(model_xgb_x2.feature_importances_)
lgbm_imp         =          model_lgbm_x2.feature_importances_          /
sum(model_lgbm_x2.feature_importances_)
cat_imp          =          model_cat_x2.feature_importances_          /

```

```

sum(model_cat_x2.feature_importances_)
labels = ["x3", "x4", "x5", "x3^2", "x4^2", "x5^2", "x3^3", "x4^3",
"x5^3",
        "x3:x4", "x4:x5", "x3:x5", "x3:x4:x5"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,
20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbd_t_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测热风速度的特征重要性",
            fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbd_t_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x3.feature_importances_
sum(model_rf_x3.feature_importances_)

```

```

ada_imp          =          model_ada_x3.feature_importances_          /
sum(model_ada_x3.feature_importances_)
gbdt_imp         =          model_gbdx3.feature_importances_          /
sum(model_gbdx3.feature_importances_)
xgb_imp          =          model_xgb_x3.feature_importances_          /
sum(model_xgb_x3.feature_importances_)
lgbm_imp         =          model_lgbm_x3.feature_importances_          /
sum(model_lgbm_x3.feature_importances_)
cat_imp          =          model_cat_x3.feature_importances_          /
sum(model_cat_x3.feature_importances_)
labels = ["x1", "x2", "x1^2", "x2^2", "x1^3", "x2^3", "x1:x2"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbdt_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 1.1)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测厚度的特征重要性",
             fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x - 0.5*w, gbdt_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 2.5*w, cat_imp):

```

```

plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x4.feature_importances_          /
sum(model_rf_x4.feature_importances_)
ada_imp          =          model_ada_x4.feature_importances_          /
sum(model_ada_x4.feature_importances_)
gbdt_imp         =          model_gbdt_x4.feature_importances_          /
sum(model_gbdt_x4.feature_importances_)
xgb_imp          =          model_xgb_x4.feature_importances_          /
sum(model_xgb_x4.feature_importances_)
lgbm_imp         =          model_lgbm_x4.feature_importances_          /
sum(model_lgbm_x4.feature_importances_)
cat_imp          =          model_cat_x4.feature_importances_          /
sum(model_cat_x4.feature_importances_)
labels = ["x1", "x2", "x1^2", "x2^2", "x1^3", "x2^3", "x1:x2"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbdt_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 1.1)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测孔隙率的特征重要性",
             fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x - 0.5*w, gbdt_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize

```

```

= 10)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x5.feature_importances_          /
sum(model_rf_x5.feature_importances_)
ada_imp          =          model_ada_x5.feature_importances_          /
sum(model_ada_x5.feature_importances_)
gbdt_imp          =          model_gbdx5.feature_importances_          /
sum(model_gbdx5.feature_importances_)
xgb_imp          =          model_xgb_x5.feature_importances_          /
sum(model_xgb_x5.feature_importances_)
lgbm_imp          =          model_lgbm_x5.feature_importances_          /
sum(model_lgbm_x5.feature_importances_)
cat_imp          =          model_cat_x5.feature_importances_          /
sum(model_cat_x5.feature_importances_)
labels = ["x1", "x2", "x1^2", "x2^2", "x1^3", "x2^3", "x1:x2"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbdt_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.85)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测压缩回弹性的特征重要性",
              fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize

```

```

= 10)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x - 0.5*w, gbdt_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 10)
ax.legend(loc = "upper left")

""" ===== 预测 ===== """
""" ===== 10 折交叉验证 ===== """
def CrossVal(data_x, data_y, model_def, k = 10):
    from sklearn import metrics
    import random
    random.seed(317)
    """ ===== 数据集分割 ===== """
    index = [i for i in range(len(data_y))]
    random.shuffle(index)
    data_x = np.array(data_x)[index]
    data_y = np.array(data_y)[index]
    n = len(data_y) // k
    r2 = []
    mae = []
    mse = []
    for i in range(k):
        print("%d 折交叉验证的第" % k + str(i + 1) + "次" )
        val_x = data_x[n * i: n * (i+1)]
        val_y = data_y[n * i: n * (i+1)]
        if i == 0:
            train_x = data_x[n * (i + 1): ]
            train_y = data_y[n * (i + 1): ]
        elif i == k - 1:
            train_x = data_x[: n * i]
            train_y = data_y[: n * i]

```

```

else:
    train_x = np.array(pd.concat([pd.DataFrame(data_x[: n * i]),
                                   pd.DataFrame(data_x[n * (i +
1): ])],
                                   ignore_index = True))
    train_y = np.array(pd.concat([pd.DataFrame(data_y[: n * i]),
                                   pd.DataFrame(data_y[n * (i +
1): ])],
                                   ignore_index = True))
    """ ===== 模型评估 ===== """
    model = model_def(train_x, train_y)
    pred_val = model.predict(val_x)

    r2.append(metrics.r2_score(val_y, pred_val))
    mae.append(metrics.mean_absolute_error(val_y, pred_val))
    mse.append(metrics.mean_squared_error(val_y, pred_val))

    return np.average(r2), np.average(mae), np.average(mse)

CrossVal(df.iloc[:, 0:7], df.iloc[:, -1], XGBoost)
CrossVal(df.iloc[:, 0:7], df.iloc[:, -1], CatBoost)

```

```

""" ===== 问题三 ===== """
""" ===== 三维散点图 ===== """
p = plt.subplot(projection = "3d")
p.scatter(df["x3"], df["x4"], df["x5"], marker='*', label = "压缩回弹性")
p.scatter(df["x3"], df["x4"], df["x6"], marker='+', label = "过滤阻力")
p.scatter(df["x3"], df["x4"], df["x7"], marker='^', label = "过滤效率")
p.scatter(df["x3"], df["x4"], df["x8"], marker='.', label = "透气性")
p.set_xlabel("厚度", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)
p.set_ylabel("孔隙率", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x3"], df["x5"], df["x6"], marker='+', label = "过滤阻力")
p.scatter(df["x3"], df["x5"], df["x7"], marker='^', label = "过滤效率")
p.scatter(df["x3"], df["x5"], df["x8"], marker='*', label = "透气性")
p.set_xlabel("厚度", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)
p.set_ylabel("压缩回弹性", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x3"], df["x6"], df["x7"], marker='^', label = "过滤效率")
p.scatter(df["x3"], df["x7"], df["x8"], marker='*', label = "透气性")
p.set_xlabel("厚度", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)
p.set_ylabel("过滤阻力", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -92)
plt.legend()

```

```

p = plt.subplot(projection = "3d")
p.scatter(df["x3"], df["x7"], df["x8"], marker='*', label = "透气性")
p.set_xlabel("厚度", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)
p.set_ylabel("过滤效率", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x4"], df["x5"], df["x6"], marker='+', label = "过滤阻力")
p.scatter(df["x4"], df["x5"], df["x7"], marker='^', label = "过滤效率")
p.scatter(df["x4"], df["x5"], df["x8"], marker='.', label = "透气性")
p.set_xlabel("孔隙率", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)
p.set_ylabel("压缩回弹性", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x4"], df["x6"], df["x7"], marker='^', label = "过滤效率")
p.scatter(df["x4"], df["x6"], df["x8"], marker='.', label = "透气性")
p.set_xlabel("孔隙率", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)
p.set_ylabel("过滤阻力", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x4"], df["x7"], df["x8"], marker='.', label = "透气性")
p.set_xlabel("孔隙率", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)
p.set_ylabel("过滤效率", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x5"], df["x6"], df["x7"], marker='^', label = "过滤效率")
p.scatter(df["x5"], df["x6"], df["x8"], marker='.', label = "透气性")
p.set_xlabel("压缩回弹性", fontdict={'family': 'KaiTi', 'size': 15}, rotation = -25)

```

```

p.set_ylabel("过滤阻力", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15},
rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x5"], df["x7"], df["x8"], marker='.', label = "透气性")
p.set_xlabel("压缩回弹性", fontdict={'family': 'KaiTi', 'size': 15},
rotation = -25)
p.set_ylabel("过滤效率", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15},
rotation = -92)
plt.legend()

p = plt.subplot(projection = "3d")
p.scatter(df["x6"], df["x7"], df["x8"], marker='.', label = "透气性")
p.set_xlabel("过滤阻力", fontdict={'family': 'KaiTi', 'size': 15},
rotation = -25)
p.set_ylabel("过滤效率", fontdict={'family': 'KaiTi', 'size': 15})
p.set_zlabel("第三变量", fontdict={'family': 'KaiTi', 'size': 15},
rotation = -92)
plt.legend()

""" ===== spearman 相关热力图 ===== """
spearman_data = pd.DataFrame(df.iloc[:, 2:]).corr(method='spearman')
ax = sns.heatmap(spearman_data,
                  vmax = 1,
                  cmap = 'coolwarm',
                  annot = True)
ax.set_title('热力相关图 (Spearman)', fontsize = 15)
plt.show()

""" ===== dataq3 ===== """
dataq3 = pd.DataFrame()
dataq3["x3"] = df["x3"]
dataq3["x4"] = df["x4"]
dataq3["x5"] = df["x5"]
dataq3["x3^2"] = np.array(df["x3"]) * np.array(df["x3"])
dataq3["x4^2"] = np.array(df["x4"]) * np.array(df["x4"])
dataq3["x5^2"] = np.array(df["x5"]) * np.array(df["x5"])
dataq3["x3^3"] = np.array(df["x3"]) * np.array(df["x3"]) *
np.array(df["x3"])
dataq3["x4^3"] = np.array(df["x4"]) * np.array(df["x4"]) *

```

```

np.array(df["x4"])
dataq3["x5^3"] = np.array(df["x5"]) * np.array(df["x5"]) *
np.array(df["x5"])
dataq3["x3x4"] = np.array(df["x3"]) * np.array(df["x4"])
dataq3["x4x5"] = np.array(df["x4"]) * np.array(df["x5"])
dataq3["x3x5"] = np.array(df["x3"]) * np.array(df["x5"])
dataq3["x3x4x5"] = np.array(df["x3"]) * np.array(df["x4"]) *
np.array(df["x5"])
dataq3["x6"] = df["x6"]
dataq3["x7"] = df["x7"]
dataq3["x8"] = df["x8"]

dataq3_ins = pd.DataFrame()
dataq3_ins["x6"] = df["x6"]
dataq3_ins["x7"] = df["x7"]
dataq3_ins["x8"] = df["x8"]
dataq3_ins["x6^2"] = np.array(df["x6"]) * np.array(df["x6"])
dataq3_ins["x7^2"] = np.array(df["x7"]) * np.array(df["x7"])
dataq3_ins["x8^2"] = np.array(df["x8"]) * np.array(df["x8"])
dataq3_ins["x6^3"] = np.array(df["x6"]) * np.array(df["x6"]) *
np.array(df["x6"])
dataq3_ins["x7^3"] = np.array(df["x7"]) * np.array(df["x7"]) *
np.array(df["x7"])
dataq3_ins["x8^3"] = np.array(df["x8"]) * np.array(df["x8"]) *
np.array(df["x8"])
dataq3_ins["x6x7"] = np.array(df["x6"]) * np.array(df["x7"])
dataq3_ins["x6x8"] = np.array(df["x6"]) * np.array(df["x8"])
dataq3_ins["x7x8"] = np.array(df["x7"]) * np.array(df["x8"])
dataq3_ins["x6x7x8"] = np.array(df["x6"]) * np.array(df["x7"]) *
np.array(df["x8"])
dataq3_ins["x3"] = df["x3"]
dataq3_ins["x4"] = df["x4"]
dataq3_ins["x5"] = df["x5"]

d3 = pd.concat([dataq3.iloc[:, 0:13], dataq3_ins.iloc[:, 0:13]], axis =
1)
d3.to_csv("D:/3、竞赛/2022.08.04 “华数杯” 全国大学生数学建模竞赛
/dataq3.csv")

""" ===== x3 ===== """
from sklearn import metrics
model_rf_x3 = RF(dataq3_ins.iloc[:, 0:13], dataq3_ins["x3"])
pre_rf_x3 = model_rf_x3.predict(dataq3_ins.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3_ins["x3"]),

```

```

pre_rf_x3)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x3"]), pre_rf_x3)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x3"]),
pre_rf_x3))))
model_ada_x3 = AdaBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x3"])
pre_ada_x3 = model_ada_x3.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x3"]),
pre_ada_x3)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x3"]),
pre_ada_x3)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x3"]),
pre_ada_x3))))
model_gbdx3 = GBDT(dataq3_ins.iloc[:, 0:13], dataq3_ins["x3"])
pre_gbdx3 = model_gbdx3.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x3"]),
pre_gbdx3)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x3"]),
pre_gbdx3)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x3"]),
pre_gbdx3))))
model_xgb_x3 = XGBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x3"])
pre_xgb_x3 = model_xgb_x3.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x3"]),
pre_xgb_x3)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x3"]),
pre_xgb_x3)))
print("RMSE:                                " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x3"]),
pre_xgb_x3))))
model_lgbm_x3 = LGBM(dataq3_ins.iloc[:, 0:13], dataq3_ins["x3"])
pre_lgbm_x3 = model_lgbm_x3.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x3"]),
pre_lgbm_x3)))
print("MAE:                                " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x3"]),
pre_lgbm_x3)))
print("RMSE:                                " +

```

```

str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x3"]),
pre_lgbm_x3))))
model_cat_x3 = CatBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x3"])
pre_cat_x3 = model_cat_x3.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x3"]),
pre_cat_x3)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x3"]),
pre_cat_x3)))
print("RMSE:     " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x3"]),
pre_cat_x3))))

""" ===== x4 ===== """
from sklearn import metrics
model_rf_x4 = RF(dataq3_ins.iloc[:, 0:13], dataq3_ins["x4"])
pre_rf_x4 = model_rf_x4.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x4"]),
pre_rf_x4)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x4"]), pre_rf_x4)))
print("RMSE:     " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x4"]),
pre_rf_x4))))
model_ada_x4 = AdaBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x4"])
pre_ada_x4 = model_ada_x4.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x4"]),
pre_ada_x4)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x4"]),
pre_ada_x4)))
print("RMSE:     " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x4"]),
pre_ada_x4))))
model_gbdx4 = GBDT(dataq3_ins.iloc[:, 0:13], dataq3_ins["x4"])
pre_gbdx4 = model_gbdx4.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x4"]),
pre_gbdx4)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x4"]),
pre_gbdx4)))
print("RMSE:     " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x4"]),
pre_gbdx4))))

```

```

model_xgb_x4 = XGBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x4"])
pre_xgb_x4 = model_xgb_x4.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x4"]),
pre_xgb_x4)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x4"]),
pre_xgb_x4)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x4"]),
pre_xgb_x4))))
model_lgbm_x4 = LGBM(dataq3_ins.iloc[:, 0:13], dataq3_ins["x4"])
pre_lgbm_x4 = model_lgbm_x4.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x4"]),
pre_lgbm_x4)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x4"]),
pre_lgbm_x4)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x4"]),
pre_lgbm_x4))))
model_cat_x4 = CatBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x4"])
pre_cat_x4 = model_cat_x4.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x4"]),
pre_cat_x4)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x4"]),
pre_cat_x4)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x4"]),
pre_cat_x4))))

""" ===== x5 ===== """
from sklearn import metrics
model_rf_x5 = RF(dataq3_ins.iloc[:, 0:13], dataq3_ins["x5"])
pre_rf_x5 = model_rf_x5.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x5"]),
pre_rf_x5)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x5"]), pre_rf_x5)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x5"]),
pre_rf_x5))))
model_ada_x5 = AdaBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x5"])
pre_ada_x5 = model_ada_x5.predict(dataq3_ins.iloc[:, 0:13])

```

```

print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x5"]),
pre_ada_x5)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x5"]),
pre_ada_x5)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x5"]),
pre_ada_x5))))
model_gbdx5 = GBDT(dataq3_ins.iloc[:, 0:13], dataq3_ins["x5"])
pre_gbdx5 = model_gbdx5.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x5"]),
pre_gbdx5)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x5"]),
pre_gbdx5)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x5"]),
pre_gbdx5))))
model_xgb_x5 = XGBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x5"])
pre_xgb_x5 = model_xgb_x5.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x5"]),
pre_xgb_x5)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x5"]),
pre_xgb_x5)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x5"]),
pre_xgb_x5))))
model_lgbm_x5 = LGBM(dataq3_ins.iloc[:, 0:13], dataq3_ins["x5"])
pre_lgbm_x5 = model_lgbm_x5.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x5"]),
pre_lgbm_x5)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x5"]),
pre_lgbm_x5)))
print("RMSE:      " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x5"]),
pre_lgbm_x5))))
model_cat_x5 = CatBoost(dataq3_ins.iloc[:, 0:13], dataq3_ins["x5"])
pre_cat_x5 = model_cat_x5.predict(dataq3_ins.iloc[:, 0:13])
print("R2:      " + str(metrics.r2_score(np.array(dataq3_ins["x5"]),
pre_cat_x5)))
print("MAE:      " +
str(metrics.mean_absolute_error(np.array(dataq3_ins["x5"]),

```

```

pre_cat_x5)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3_ins["x5"]),
pre_cat_x5))))

""" ===== x6 ===== """
from sklearn import metrics
model_rf_x6 = RF(dataq3.iloc[:, 0:13], dataq3["x6"])
pre_rf_x6 = model_rf_x6.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x6"]), pre_rf_x6)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x6"]),
pre_rf_x6)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x6"]),
pre_rf_x6))))
model_ada_x6 = AdaBoost(dataq3.iloc[:, 0:13], dataq3["x6"])
pre_ada_x6 = model_ada_x6.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x6"]),
pre_ada_x6)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x6"]),
pre_ada_x6)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x6"]),
pre_ada_x6))))
model_gbdx6 = GBDT(dataq3.iloc[:, 0:13], dataq3["x6"])
pre_gbdx6 = model_gbdx6.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x6"]),
pre_gbdx6)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x6"]),
pre_gbdx6)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x6"]),
pre_gbdx6))))
model_xgb_x6 = XGBoost(dataq3.iloc[:, 0:13], dataq3["x6"])
pre_xgb_x6 = model_xgb_x6.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x6"]),
pre_xgb_x6)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x6"]),
pre_xgb_x6)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x6"]),
pre_xgb_x6))))
model_lgbm_x6 = LGBM(dataq3.iloc[:, 0:13], dataq3["x6"])
pre_lgbm_x6 = model_lgbm_x6.predict(dataq3.iloc[:, 0:13])

```

```

print("R2: " + str(metrics.r2_score(np.array(dataq3["x6"]),
pre_lgbm_x6)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x6"]),
pre_lgbm_x6)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x6"]),
pre_lgbm_x6))))
model_cat_x6 = CatBoost(dataq3.iloc[:, 0:13], dataq3["x6"])
pre_cat_x6 = model_cat_x6.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x6"]),
pre_cat_x6)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x6"]),
pre_cat_x6)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x6"]),
pre_cat_x6))))

""" ===== x7 ===== """
from sklearn import metrics
model_rf_x7 = RF(dataq3.iloc[:, 0:13], dataq3["x7"])
pre_rf_x7 = model_rf_x7.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x7"]), pre_rf_x7)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x7"]),
pre_rf_x7)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x7"]),
pre_rf_x7))))
model_ada_x7 = AdaBoost(dataq3.iloc[:, 0:13], dataq3["x7"])
pre_ada_x7 = model_ada_x7.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x7"]),
pre_ada_x7)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x7"]),
pre_ada_x7)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x7"]),
pre_ada_x7))))
model_gbdx7 = GBDT(dataq3.iloc[:, 0:13], dataq3["x7"])
pre_gbdx7 = model_gbdx7.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x7"]),
pre_gbdx7)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x7"]),
pre_gbdx7)))
print("RMSE: " +

```

```

str(sqrt(metrics.mean_squared_error(np.array(dataq3["x7"]),
pre_gbdx7))))
model_xgb_x7 = XGBoost(dataq3.iloc[:, 0:13], dataq3["x7"])
pre_xgb_x7 = model_xgb_x7.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x7"]),
pre_xgb_x7)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x7"]),
pre_xgb_x7)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x7"]),
pre_xgb_x7))))
model_lgbm_x7 = LGBM(dataq3.iloc[:, 0:13], dataq3["x7"])
pre_lgbm_x7 = model_lgbm_x7.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x7"]),
pre_lgbm_x7)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x7"]),
pre_lgbm_x7)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x7"]),
pre_lgbm_x7))))
model_cat_x7 = CatBoost(dataq3.iloc[:, 0:13], dataq3["x7"])
pre_cat_x7 = model_cat_x7.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x7"]),
pre_cat_x7)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x7"]),
pre_cat_x7)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x7"]),
pre_cat_x7))))

""" ===== x8 ===== """
from sklearn import metrics
model_rf_x8 = RF(dataq3.iloc[:, 0:13], dataq3["x8"])
pre_rf_x8 = model_rf_x8.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x8"]), pre_rf_x8)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x8"]),
pre_rf_x8)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x8"]),
pre_rf_x8))))
model_ada_x8 = AdaBoost(dataq3.iloc[:, 0:13], dataq3["x8"])
pre_ada_x8 = model_ada_x8.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x8"]),

```

```

pre_ada_x8)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x8"]),
pre_ada_x8)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x8"]),
pre_ada_x8))))
model_gbdx8 = GBDT(dataq3.iloc[:, 0:13], dataq3["x8"])
pre_gbdx8 = model_gbdx8.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x8"]),
pre_gbdx8)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x8"]),
pre_gbdx8)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x8"]),
pre_gbdx8))))
model_xgbx8 = XGBoost(dataq3.iloc[:, 0:13], dataq3["x8"])
pre_xgbx8 = model_xgbx8.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x8"]),
pre_xgbx8)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x8"]),
pre_xgbx8)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x8"]),
pre_xgbx8))))
model_lgbx8 = LGBM(dataq3.iloc[:, 0:13], dataq3["x8"])
pre_lgbx8 = model_lgbx8.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x8"]),
pre_lgbx8)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x8"]),
pre_lgbx8)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x8"]),
pre_lgbx8))))
model_catx8 = CatBoost(dataq3.iloc[:, 0:13], dataq3["x8"])
pre_catx8 = model_catx8.predict(dataq3.iloc[:, 0:13])
print("R2: " + str(metrics.r2_score(np.array(dataq3["x8"]),
pre_catx8)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(dataq3["x8"]),
pre_catx8)))
print("RMSE: " +
str(sqrt(metrics.mean_squared_error(np.array(dataq3["x8"]),
pre_catx8))))

""" ===== 预测值和正确值的比较 ===== """

```

```

ax = plt.subplot(1,1,1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x3"], label = "Origin", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_rf_x3, label = "RandomForest",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x3, label = "AdaBoost",linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx3, label = "GBDT",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x3, label = "XGBoost",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x3, label = "LightGBM",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x3, label = "CatBoost",linestyle = ":", linewidth = 2)
plt.ylabel("厚度", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("厚度的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})
plt.legend(loc = "upper right")
axins = ax.inset_axes((0.07, 0.07, 0.33, 0.3))
axins.plot(x_axis, df["x3"], linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_rf_x3, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x3, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx3, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x3, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x3, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x3, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 20
xlim1 = 30
# Y 轴的显示范围
ylim0 = 2.8
ylim1 = 3.25
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 2, loc2 = 4)

```

```

ax = plt.subplot(1,1,1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x4"], label = "Origin", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_rf_x4, label = "RandomForest",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x4, label = "AdaBoost",linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx4, label = "GBDT",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x4, label = "XGBoost",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x4, label = "LightGBM",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x4, label = "CatBoost",linestyle = ":", linewidth = 2)
plt.ylabel("孔隙率", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("孔隙率的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})
plt.legend(loc = "upper right")
axins = ax.inset_axes((0.07, 0.07, 0.33, 0.3))
axins.plot(x_axis, df["x4"], linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_rf_x4, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x4, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx4, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x4, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x4, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x4, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 20
xlim1 = 30
# Y 轴的显示范围
ylim0 = 96.2
ylim1 = 97
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 2, loc2 = 4)

```

```

ax = plt.subplot(1,1,1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x5"], label = "Origin", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_rf_x5, label = "RandomForest",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x5, label = "AdaBoost",linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx_x5, label = "GBDT",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x5, label = "XGBoost",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x5, label = "LightGBM",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x5, label = "CatBoost",linestyle = ":", linewidth = 2)
plt.ylabel("压缩回弹性", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("压缩回弹性的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})
plt.legend(loc = "upper right")
axins = ax.inset_axes((0.5, 0.1, 0.33, 0.3))
axins.plot(x_axis, df["x5"], linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_rf_x5, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x5, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx_x5, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x5, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x5, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x5, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 20
xlim1 = 30
# Y 轴的显示范围
ylim0 = 86.5
ylim1 = 87.5
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 1, loc2 = 3)

```

```

ax = plt.subplot(1,1,1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x6"], label = "Origin", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_rf_x6, label = "RandomForest",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x6, label = "AdaBoost",linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx6, label = "GBDT",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x6, label = "XGBoost",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x6, label = "LightGBM",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x6, label = "CatBoost",linestyle = ":", linewidth = 2)
plt.ylabel("过滤阻力", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("过滤阻力的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})
plt.legend(loc = "upper right")
axins = ax.inset_axes((0.6, 0.1, 0.33, 0.3))
axins.plot(x_axis, df["x6"], linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_rf_x6, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x6, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx6, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x6, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x6, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x6, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 40
xlim1 = 50
# Y 轴的显示范围
ylim0 = 30
ylim1 = 40
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 1, loc2 = 3)

```

```

ax = plt.subplot(1,1,1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x7"], label = "Origin", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_rf_x7, label = "RandomForest",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x7, label = "AdaBoost",linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx7, label = "GBDT",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x7, label = "XGBoost",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x7, label = "LightGBM",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x7, label = "CatBoost",linestyle = ":", linewidth = 2)
plt.ylabel("过滤效率", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("过滤效率的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})
plt.legend(loc = "upper right")
axins = ax.inset_axes((0.2, 0.6, 0.33, 0.3))
axins.plot(x_axis, df["x7"], linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_rf_x7, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x7, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx7, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x7, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x7, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x7, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 40
xlim1 = 50
# Y 轴的显示范围
ylim0 = 45
ylim1 = 55
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 1, loc2 = 3)

```

```

ax = plt.subplot(1,1,1)
x_axis = range(1, 76)
ax.plot(x_axis, df["x8"], label = "Origin", linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_rf_x8, label = "RandomForest",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_ada_x8, label = "AdaBoost",linestyle = ":", linewidth = 2)
ax.plot(x_axis, pre_gbdx8, label = "GBDT",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_xgb_x8, label = "XGBoost",linestyle = "--", linewidth = 2)
ax.plot(x_axis, pre_lgbm_x8, label = "LightGBM",linestyle = "-.", linewidth = 2)
ax.plot(x_axis, pre_cat_x8, label = "CatBoost",linestyle = ":", linewidth = 2)
plt.ylabel("透气性", fontdict={'family': 'KaiTi', 'size': 15})
plt.xlabel("样本", fontdict={'family': 'KaiTi', 'size': 15}, labelpad = 2)
plt.title("透气性的真实值与预测值", fontdict={'family': 'KaiTi', 'size': 20})
plt.legend(loc = "upper right")
axins = ax.inset_axes((0.1, 0.1, 0.33, 0.3))
axins.plot(x_axis, df["x8"], linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_rf_x8, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_ada_x8, linestyle = ":", linewidth = 5)
axins.plot(x_axis, pre_gbdx8, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_xgb_x8, linestyle = "--", linewidth = 5)
axins.plot(x_axis, pre_lgbm_x8, linestyle = "-.", linewidth = 5)
axins.plot(x_axis, pre_cat_x8, linestyle = ":", linewidth = 5)
# 设置放大区间
zone_left = 10
zone_right = 50
# X 轴的显示范围
xlim0 = 40
xlim1 = 50
# Y 轴的显示范围
ylim0 = 400
ylim1 = 500
# 调整子坐标系的显示范围
axins.set_xlim(xlim0, xlim1)
axins.set_ylim(ylim0, ylim1)
mark_inset(ax, axins, loc1 = 2, loc2 = 4)

```

```

""" ===== 特征重要性 ===== """
rf_imp          =          model_rf_x3.feature_importances_          /
sum(model_rf_x3.feature_importances_)
ada_imp          =          model_ada_x3.feature_importances_          /
sum(model_ada_x3.feature_importances_)
gbdt_imp         =          model_gbdt_x3.feature_importances_        /
sum(model_gbdt_x3.feature_importances_)
xgb_imp          =          model_xgb_x3.feature_importances_          /
sum(model_xgb_x3.feature_importances_)
lgbm_imp         =          model_lgbm_x3.feature_importances_         /
sum(model_lgbm_x3.feature_importances_)
cat_imp          =          model_cat_x3.feature_importances_          /
sum(model_cat_x3.feature_importances_)
labels = ["x6", "x7", "x8", "x6^2", "x7^2", "x8^2", "x6^3", "x7^3",
"x8^3",
        "x6:x7", "x6:x8", "x7:x8", "x6:x7:x8"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,
20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbdt_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测厚度的特征重要性",
            fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbdt_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 0.5*w, xgb_imp):

```

```

plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x4.feature_importances_          /
sum(model_rf_x4.feature_importances_)
ada_imp          =          model_ada_x4.feature_importances_          /
sum(model_ada_x4.feature_importances_)
gbdt_imp         =          model_gbdx4.feature_importances_          /
sum(model_gbdx4.feature_importances_)
xgb_imp          =          model_xgb_x4.feature_importances_          /
sum(model_xgb_x4.feature_importances_)
lgbm_imp         =          model_lgbm_x4.feature_importances_          /
sum(model_lgbm_x4.feature_importances_)
cat_imp          =          model_cat_x4.feature_importances_          /
sum(model_cat_x4.feature_importances_)
labels = ["x6", "x7", "x8", "x6^2", "x7^2", "x8^2", "x6^3", "x7^3",
"x8^3",
        "x6:x7", "x6:x8", "x7:x8", "x6:x7:x8"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,
20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbdt_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测孔隙率的特征重要性",
            fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):

```

```

plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbd_t_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x5.feature_importances_          /
sum(model_rf_x5.feature_importances_)
ada_imp          =          model_ada_x5.feature_importances_          /
sum(model_ada_x5.feature_importances_)
gbdt_imp         =          model_gbd_t_x5.feature_importances_       /
sum(model_gbd_t_x5.feature_importances_)
xgb_imp          =          model_xgb_x5.feature_importances_         /
sum(model_xgb_x5.feature_importances_)
lgbm_imp         =          model_lgbm_x5.feature_importances_        /
sum(model_lgbm_x5.feature_importances_)
cat_imp          =          model_cat_x5.feature_importances_         /
sum(model_cat_x5.feature_importances_)
labels = ["x6", "x7", "x8", "x6^2", "x7^2", "x8^2", "x6^3", "x7^3",
"x8^3",
        "x6:x7", "x6:x8", "x7:x8", "x6:x7:x8"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,
20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbd_t_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")

```

```

ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测压缩回弹性的特征重要性",
             fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbd_t_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x6.feature_importances_          /
sum(model_rf_x6.feature_importances_)
ada_imp          =          model_ada_x6.feature_importances_          /
sum(model_ada_x6.feature_importances_)
gbdt_imp         =          model_gbd_t_x6.feature_importances_       /
sum(model_gbd_t_x6.feature_importances_)
xgb_imp          =          model_xgb_x6.feature_importances_          /
sum(model_xgb_x6.feature_importances_)
lgbm_imp         =          model_lgbm_x6.feature_importances_         /
sum(model_lgbm_x6.feature_importances_)
cat_imp          =          model_cat_x6.feature_importances_          /
sum(model_cat_x6.feature_importances_)
labels = ["x3", "x4", "x5", "x3^2", "x4^2", "x5^2", "x3^3", "x4^3",
"x5^3",
        "x3:x4", "x4:x5", "x3:x5", "x3:x4:x5"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,

```

```

20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbd_t_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测过滤阻力的特征重要性",
            fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbd_t_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
ax.legend(loc = "upper left")

rf_imp          =          model_rf_x7.feature_importances_          /
sum(model_rf_x7.feature_importances_)
ada_imp          =          model_ada_x7.feature_importances_          /
sum(model_ada_x7.feature_importances_)
gbd_t_imp        =          model_gbd_t_x7.feature_importances_          /
sum(model_gbd_t_x7.feature_importances_)
xgb_imp          =          model_xgb_x7.feature_importances_          /

```

```

sum(model_xgb_x7.feature_importances_)
lgbm_imp      =      model_lgbm_x7.feature_importances_      /
sum(model_lgbm_x7.feature_importances_)
cat_imp       =      model_cat_x7.feature_importances_       /
sum(model_cat_x7.feature_importances_)
labels = ["x3", "x4", "x5", "x3^2", "x4^2", "x5^2", "x3^3", "x4^3",
"x5^3",
        "x3:x4", "x4:x5", "x3:x5", "x3:x4:x5"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,
20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbd_t_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测过滤效率的特征重要性",
        fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbd_t_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)

```

```

ax.legend(loc = "upper left")

rf_imp          =          model_rf_x8.feature_importances_          /
sum(model_rf_x8.feature_importances_)
ada_imp          =          model_ada_x8.feature_importances_          /
sum(model_ada_x8.feature_importances_)
gbdt_imp         =          model_gbdx8.feature_importances_         /
sum(model_gbdx8.feature_importances_)
xgb_imp          =          model_xgb_x8.feature_importances_          /
sum(model_xgb_x8.feature_importances_)
lgbm_imp         =          model_lgbm_x8.feature_importances_         /
sum(model_lgbm_x8.feature_importances_)
cat_imp          =          model_cat_x8.feature_importances_          /
sum(model_cat_x8.feature_importances_)
labels = ["x3", "x4", "x5", "x3^2", "x4^2", "x5^2", "x3^3", "x4^3",
"x5^3",
          "x3:x4", "x4:x5", "x3:x5", "x3:x4:x5"]
x = np.array([0.9, 2.9, 4.9, 6.9, 8.9, 10.9, 12.9, 14.9, 16.9, 18.9,
20.9, 22.9, 24.9])
w = 0.3
fig, ax = plt.subplots(figsize=(24,8))
r1 = ax.bar(x - 2.5*w, rf_imp, w, label = "RandomForest")
r2 = ax.bar(x - 1.5*w, ada_imp, w, label = "AdaBoost")
r3 = ax.bar(x - 0.5*w, gbdt_imp, w, label = "GBDT")
r4 = ax.bar(x + 0.5*w, xgb_imp, w, label = "XGBoost")
r5 = ax.bar(x + 1.5*w, lgbm_imp, w, label = "LightGBM")
r6 = ax.bar(x + 2.5*w, cat_imp, w, label = "CatBoost")
ax.set_ylabel("重要性", fontdict={'family': 'KaiTi', 'size': 15})
ax.set_xlabel("特征", fontdict={'family': 'KaiTi', 'size': 15}, labelpad
= 2)
ax.set_ylim(0, 0.9)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontdict={'family': 'KaiTi', 'size': 10})
ax.set_title("预测透气性的特征重要性",
             fontdict={'family': 'KaiTi', 'size': 20})
for a, b in zip(x - 2.5*w, rf_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 1.5*w, ada_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x - 0.5*w, gbdt_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)

```

```

for a, b in zip(x + 0.5*w, xgb_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 1.5*w, lgbm_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
for a, b in zip(x + 2.5*w, cat_imp):
    plt.text(a, b + 0.02, '%.2f' % b, ha='center', va='bottom', fontsize
= 8)
ax.legend(loc = "upper left")


""" ===== BP ===== """
""" ===== 数据集 ===== """
from sklearn.model_selection import train_test_split
train_x3, test_x3, train_y3, test_y3 = train_test_split(dataq2.iloc[:,
0:7],
                                                    dataq2.iloc[:,
7],
                                                    test_size =
0.2,
                                                    random_state =
317,
                                                    shuffle = True)
train_x4, test_x4, train_y4, test_y4 = train_test_split(dataq2.iloc[:,
0:7],
                                                    dataq2.iloc[:,
8],
                                                    test_size =
0.2,
                                                    random_state =
317,
                                                    shuffle = True)
train_x5, test_x5, train_y5, test_y5 = train_test_split(dataq2.iloc[:,
0:7],
                                                    dataq2.iloc[:,
-1],
                                                    test_size =

```

```

0.2,
                                                    random_state =
317,
                                                    shuffle = True)
train_x6, test_x6, train_y6, test_y6 = train_test_split(dataq3.iloc[:,
0:13],
                                                    dataq3.iloc[:,
-3],
                                                    test_size =
0.2,
                                                    random_state =
317,
                                                    shuffle = True)
train_x7, test_x7, train_y7, test_y7 = train_test_split(dataq3.iloc[:,
0:13],
                                                    dataq3.iloc[:,
-2],
                                                    test_size =
0.2,
                                                    random_state =
317,
                                                    shuffle = True)

""" ===== 标准化 ===== """
meanx3 = train_x3.mean(axis = 0)
train_x3 -= meanx3
stdx3 = train_x3.std(axis = 0)
train_x3 /= stdx3
test_x3 -= meanx3
test_x3 /= stdx3

meanx4 = train_x4.mean(axis = 0)
train_x4 -= meanx4
stdx4 = train_x4.std(axis = 0)
train_x4 /= stdx4
test_x4 -= meanx4
test_x4 /= stdx4

meanx5 = train_x5.mean(axis = 0)
train_x5 -= meanx5
stdx5 = train_x5.std(axis = 0)
train_x5 /= stdx5
test_x5 -= meanx5
test_x5 /= stdx5

```

```

meanx6 = train_x6.mean(axis = 0)
train_x6 -= meanx6
stdx6 = train_x6.std(axis = 0)
train_x6 /= stdx6
test_x6 -= meanx6
test_x6 /= stdx6

meanx7 = train_x7.mean(axis = 0)
train_x7 -= meanx7
stdx7 = train_x7.std(axis = 0)
train_x7 /= stdx7
test_x7 -= meanx7
test_x7 /= stdx7

""" ===== 建模 ===== """
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import RMSprop
import tensorflow as tf
def bp(train_x, train_y, test_x, test_y):
    model = Sequential()
    model.add(Dense(units = 2048, activation = "relu",
                    input_shape = (train_x.shape[1], )))
    model.add(Dropout(0.1))
    model.add(Dense(units = 1024, activation = "relu"))
    model.add(Dropout(0.1))
    model.add(Dense(units = 1))
    model.compile(optimizer = RMSprop(learning_rate = 1e-4),
                  loss = "mse",
                  metrics = ["mae"])
    history = model.fit(train_x, train_y,
                        steps_per_epoch = 20,
                        epochs = 500,
                        batch_size = 1,
                        validation_data = (test_x, test_y))
    history_dict = history.history
    epochs = len(history_dict["loss"])
    x_axis = range(1, epochs + 1)
    plt.figure()
    plt.subplot(1, 2, 1)
    plt.plot(x_axis, history_dict['loss'],
             label = "Train's MSE is %4f" %history_dict['loss'][-1])
    plt.plot(x_axis, history_dict['val_loss'],

```

```

        label = "Validation's MSE is %4f" %history_dict['val_loss'][-1],
        linestyle=':', linewidth = 5)
    plt.ylabel("MSE", fontsize = 15)
    plt.xlabel("Epochs", fontsize = 15)
    plt.legend(loc = "upper right")
    plt.subplot(1, 2, 2)
    plt.plot(x_axis, history_dict['mae'],
             label = "Train's MAE is %4f" %history_dict['mae'][-1])
    plt.plot(x_axis, history_dict['val_mae'],
             label = "Validation's MAE is %4f" %history_dict['val_mae'][-
1],
             linestyle=':', linewidth = 5)
    plt.ylabel("MAE", fontsize = 15)
    plt.xlabel("Epochs", fontsize = 15)
    plt.legend(loc = "upper right")
    plt.show()
    return model

model_bp_x3 = bp(train_x3, train_y3, test_x3, test_y3)
model_bp_x4 = bp(train_x4, train_y4, test_x4, test_y4)
model_bp_x5 = bp(train_x5, train_y5, test_x5, test_y5)
model_bp_x6 = bp(train_x6, train_y6, test_x6, test_y6)
model_bp_x7 = bp(train_x7, train_y7, test_x7, test_y7)

pred_bp_x3 = model_bp_x3.predict(test_x3)
print("R2: " + str(metrics.r2_score(test_y3, pred_bp_x3)))
print("MAE: " + str(metrics.mean_absolute_error(test_y3, pred_bp_x3)))
print("RMSE: " + str(sqrt(metrics.mean_squared_error(test_y3,
pred_bp_x3))))
pred_bp_x4 = model_bp_x4.predict(test_x4)
print("R2: " + str(metrics.r2_score(test_y4, pred_bp_x4)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(test_y4,
pred_bp_x4))))
print("RMSE: " + str(sqrt(metrics.mean_squared_error(np.array(test_y4,
pred_bp_x4)))))
pred_bp_x5 = model_bp_x5.predict(test_x5)
print("R2: " + str(metrics.r2_score(test_y5, pred_bp_x5)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(test_y5,
pred_bp_x5))))
print("RMSE: " + str(sqrt(metrics.mean_squared_error(np.array(test_y5,
pred_bp_x5)))))
pred_bp_x6 = model_bp_x6.predict(test_x6)
print("R2: " + str(metrics.r2_score(test_y6, pred_bp_x6)))

```

```

print("MAE: " + str(metrics.mean_absolute_error(np.array(test_y6,
pred_bp_x6))))
print("RMSE: " + str(sqrt(metrics.mean_squared_error(np.array(test_y6,
pred_bp_x6)))))
pred_bp_x7 = model_bp_x7.predict(test_x7)
print("R2: " + str(metrics.r2_score(test_y7, pred_bp_x7)))
print("MAE: " + str(metrics.mean_absolute_error(np.array(test_y7,
pred_bp_x7))))
print("RMSE: " + str(sqrt(metrics.mean_squared_error(np.array(test_y7,
pred_bp_x7)))))

```

```

"""

```

```

max x7
s. t.
x1, x2, x3, x4, x5 > 0

```

```

model
predcit x7: XGB
predict x3\x4\x5: Cat/Cat/XGB
"""

```

```

""" ===== SEGA ===== """

```

```

model_cat_x3 = CatBoost(dataq2.iloc[:, 0:7], dataq2["x3"])
model_cat_x4 = CatBoost(dataq2.iloc[:, 0:7], dataq2["x4"])
model_xgb_x5 = XGBoost(dataq2.iloc[:, 0:7], dataq2["x5"])
model_xgb_x6 = XGBoost(dataq3.iloc[:, 0:13], dataq3["x6"])
model_xgb_x7 = XGBoost(dataq3.iloc[:, 0:13], dataq3["x7"])
import geatpy as ea
n = 100
class MyProblem(ea.Problem): # 继承 Problem 父类
    def __init__(self):

```

```

        name = 'MyProblem' # 初始化 name (函数名称, 可以随意设置)
        M = 1 # 初始化 M (目标维数)
        maxormins = [-1] # 初始化 maxormins (目标最小最大化标记列表, 1:
最小化该目标; -1: 最大化该目标)
        Dim = 2 # 初始化 Dim (决策变量维数)
        varTypes = [0] * Dim # 初始化 varTypes (决策变量的类型, 元素为 0
表示对应的变量是连续的; 1 表示是离散的)
        lb = [0] * Dim # 决策变量下界
        ub = [10**5] * Dim # 决策变量上界
        lbin = [0] * Dim # 决策变量下边界 (0 表示不包含该变量的下边界,
1 表示包含)
        ubin = [0] * Dim # 决策变量上边界 (0 表示不包含该变量的上边界,
1 表示包含)
        # 调用父类构造方法完成实例化
        ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
ub, lbin, ubin)
    def evalVars(self, Vars): # 目标函数
        x1 = Vars[:, [0]]
        x2 = Vars[:, [1]]
        a = [x1, x2, x1**2, x2**2, x1**3, x2**3, x1*x2]
        d = pd.DataFrame()
        d["x1"] = a[0].reshape(1, -1)[0]
        d["x2"] = a[1].reshape(1, -1)[0]
        d["x1^2"] = a[2].reshape(1, -1)[0]
        d["x2^2"] = a[3].reshape(1, -1)[0]
        d["x1^3"] = a[4].reshape(1, -1)[0]
        d["x2^3"] = a[5].reshape(1, -1)[0]
        d["x1x2"] = a[6].reshape(1, -1)[0]
        x3 = model_cat_x3.predict(d)
        x4 = model_cat_x4.predict(d)
        x5 = model_xgb_x5.predict(d)
        b = [x3, x4, x5, x3**2, x4**2, x5**2, x3**3, x4**3, x5**3, x3*x4,
x4*x5, x3*x5, x3*x4*x5]
        df = pd.DataFrame()
        df["x3"] = b[0].reshape(1, -1)[0]
        df["x4"] = b[1].reshape(1, -1)[0]
        df["x5"] = b[2].reshape(1, -1)[0]
        df["x3^2"] = b[3].reshape(1, -1)[0]
        df["x4^2"] = b[4].reshape(1, -1)[0]
        df["x5^2"] = b[5].reshape(1, -1)[0]
        df["x3^3"] = b[6].reshape(1, -1)[0]
        df["x4^3"] = b[7].reshape(1, -1)[0]
        df["x5^3"] = b[8].reshape(1, -1)[0]
        df["x3x4"] = b[9].reshape(1, -1)[0]

```

```

df["x4x5"] = b[10].reshape(1, -1)[0]
df["x3x5"] = b[11].reshape(1, -1)[0]
df["x3x4x5"] = b[12].reshape(1, -1)[0]
f = model_xgb_x7.predict(df).reshape(n, 1)
CV = np.hstack([-model_cat_x3.predict(d).reshape(n, 1),
                 -model_cat_x4.predict(d).reshape(n, 1),
                 -model_xgb_x5.predict(d).reshape(n, 1)
                ]) # 构建违反约束程度矩阵
    return f, CV
problem = MyProblem()
# 构建算法
algorithm = ea.soea_SEGA_templet(problem,
                                ea.Population(Encoding='RI', NIND=n),
                                MAXGEN=50, # 最大进化代数。
                                logTras=1, # 表示每隔多少代记录一次日志信息, 0 表示不记录。

                                trappedValue=1e-6, # 单目标优化陷入停滞的判断阈值。

                                maxTrappedCount=10) # 进化停滞计数器最大上限值。
algorithm.mutOper.Pm = 0.9 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res_sega = ea.optimize(algorithm,
                       verbose=True,
                       drawing=1,
                       outputMsg=True,
                       drawLog=True,
                       saveFlag=True)
print(res_sega)

x = [i for i in range(1, 14)]

f_opt = [47.52191, 82.998024, 82.998024, 82.998024, 82.998024, 82.998024,
82.998024, 82.998024, 82.998024,
        82.998024, 83.14685, 83.14685, 83.14685]
f_max = [47.52191, 82.998024, 82.998024, 82.998024, 82.998024,
82.998024, 82.998024, 82.998024, 82.998024,
        82.998024, 83.14685, 83.14685, 83.14685]
f_avg = [44.207508, 44.724834, 46.559826, 51.586254, 59.611027,
71.289894, 82.998024, 82.998024, 82.998024,
        82.998024, 82.99954, 83.000725, 83.00222]
f_min = [44.014805, 44.175644, 44.175644, 44.175644, 47.52191,
50.232525, 82.998024, 82.998024, 82.998024,

```

```

        , 82.998024, 82.998024, 82.998024, 82.998024]
plt.figure()
plt.plot(x, f_opt, label = "Best f")
plt.plot(x, f_max, label = "Max f")
plt.plot(x, f_avg, label = "Average f")
plt.plot(x, f_min, label = "Min f")
plt.xlabel("进化次数", fontdict={'family': 'KaiTi', 'size': 15})
plt.ylabel("目标函数值", fontdict={'family': 'KaiTi', 'size': 15})
plt.title("增强精英保留遗传算法的进化曲线", fontdict={'family': 'KaiTi',
'size': 20})
plt.legend()

m1 = np.mean(df["x1"])
s1 = np.std(df["x1"])
m2 = np.mean(df["x2"])
s2 = np.std(df["x2"])
m3 = np.mean(df["x3"])
s3 = np.std(df["x3"])
m4 = np.mean(df["x4"])
s4 = np.std(df["x4"])
m5 = np.mean(df["x5"])
s5 = np.std(df["x5"])
m6 = np.mean(df["x6"])
s6 = np.std(df["x6"])
m7 = np.mean(df["x7"])
s7 = np.std(df["x7"])
m8 = np.mean(df["x8"])
s8 = np.std(df["x8"])

n = 100
class MyProblem(ea.Problem): # 继承 Problem 父类
    def __init__(self):
        name = 'MyProblem' # 初始化 name (函数名称, 可以随意设置)
        M = 1 # 初始化 M (目标维数)
        maxormins = [-1] # 初始化 maxormins (目标最小最大化标记列表, 1:
最小化该目标; -1: 最大化该目标)
        Dim = 2 # 初始化 Dim (决策变量维数)
        varTypes = [0] * Dim # 初始化 varTypes (决策变量的类型, 元素为 0
表示对应的变量是连续的; 1 表示是离散的)

```

```

        lb = [0] * Dim # 决策变量下界
        ub = [10**5] * Dim # 决策变量上界
        lbin = [0] * Dim # 决策变量下边界 (0 表示不包含该变量的下边界,
1 表示包含)
        ubin = [0] * Dim # 决策变量上边界 (0 表示不包含该变量的上边界,
1 表示包含)
        # 调用父类构造方法完成实例化
        ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
ub, lbin, ubin)
        def evalVars(self, Vars): # 目标函数
            x1 = Vars[:, [0]]
            x2 = Vars[:, [1]]
            x3 = 0.0221+0.8169*x1+0.5552*x2+0.1203*(x1**2)-
0.1427*(x2**2)+0.0135*x1*x2
            x4 = 0.3108+0.7301*x1+0.5219*x2-0.0953*(x1**2)-0.2198*(x2**2)-
0.2166*x1*x2
            x5 = 1.3625-0.4026*x1-0.3203*x2-0.8631*(x1**2)-0.5178*(x2**2)
            f = -(1.8367*x3+0.6285*x5-0.2005*(x4**2)-0.2929*(x5**2)-
0.7508*(x3**3)-1.0645*x3*x5-0.2167*x3*x4*x5))*s7+m7
            CV = np.hstack([-x3, -x4, -x5
                ]) # 构建违反约束程度矩阵
            return f, CV
        problem = MyProblem()
        # 构建算法
        algorithm = ea.soea_SEGA_templet(problem,
            ea.Population(Encoding='RI', NIND=n),
            MAXGEN=25, # 最大进化代数。
            logTras=1, # 表示每隔多少代记录一次日
志信息, 0 表示不记录。
            trappedValue=1e-6, # 单目标优化陷入停
滞的判断阈值。
            maxTrappedCount=10) # 进化停滞计数器
最大上限值。
        algorithm.mutOper.Pm = 0.9 # 修改变异算子的变异概率
        algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
        # 求解
        res_sega_lm = ea.optimize(algorithm,
            verbose=True,
            drawing=1,
            outputMsg=True,
            drawLog=True,
            saveFlag=True)
        print(res_sega_lm)

```

```

n = 100
class MyProblem(ea.Problem): # 继承 Problem 父类
    def __init__(self):
        name = 'MyProblem' # 初始化 name (函数名称, 可以随意设置)
        M = 1 # 初始化 M (目标维数)
        maxormins = [-1] # 初始化 maxormins (目标最小最大化标记列表, 1:
最小化该目标; -1: 最大化该目标)
        Dim = 2 # 初始化 Dim (决策变量维数)
        varTypes = [0] * Dim # 初始化 varTypes (决策变量的类型, 元素为 0
表示对应的变量是连续的; 1 表示是离散的)
        lb = [0] * Dim # 决策变量下界
        ub = [10**5] * Dim # 决策变量上界
        lbin = [0] * Dim # 决策变量下边界 (0 表示不包含该变量的下边界,
1 表示包含)
        ubin = [0] * Dim # 决策变量上边界 (0 表示不包含该变量的上边界,
1 表示包含)
        # 调用父类构造方法完成实例化
        ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
ub, lbin, ubin)
    def evalVars(self, Vars): # 目标函数
        x1 = Vars[:, [0]]
        x2 = Vars[:, [1]]
        a = [x1, x2, x1**2, x2**2, x1**3, x2**3, x1*x2]
        d = pd.DataFrame()
        d["x1"] = a[0].reshape(1, -1)[0]
        d["x2"] = a[1].reshape(1, -1)[0]
        d["x1^2"] = a[2].reshape(1, -1)[0]
        d["x2^2"] = a[3].reshape(1, -1)[0]
        d["x1^3"] = a[4].reshape(1, -1)[0]
        d["x2^3"] = a[5].reshape(1, -1)[0]
        d["x1x2"] = a[6].reshape(1, -1)[0]
        d = (d - meanx3) / stdx3
        x3 = model_bp_x3.predict(d)
        x4 = model_bp_x4.predict(d)
        x5 = model_bp_x5.predict(d)
        b = [x3, x4, x5, x3**2, x4**2, x5**2, x3**3, x4**3, x5**3, x3*x4,
x4*x5, x3*x5, x3*x4*x5]
        df = pd.DataFrame()
        df["x3"] = b[0].reshape(1, -1)[0]
        df["x4"] = b[1].reshape(1, -1)[0]
        df["x5"] = b[2].reshape(1, -1)[0]
        df["x3^2"] = b[3].reshape(1, -1)[0]

```

```

df["x4^2"] = b[4].reshape(1, -1)[0]
df["x5^2"] = b[5].reshape(1, -1)[0]
df["x3^3"] = b[6].reshape(1, -1)[0]
df["x4^3"] = b[7].reshape(1, -1)[0]
df["x5^3"] = b[8].reshape(1, -1)[0]
df["x3x4"] = b[9].reshape(1, -1)[0]
df["x4x5"] = b[10].reshape(1, -1)[0]
df["x3x5"] = b[11].reshape(1, -1)[0]
df["x3x4x5"] = b[12].reshape(1, -1)[0]
df = (df - meanx6) / stdx6
f = model_bp_x7.predict(df)
CV = np.hstack([-model_bp_x3.predict(d),
                -model_bp_x4.predict(d),
                -model_bp_x5.predict(d),
                model_bp_x7.predict(df) - 100
                ]) # 构建违反约束程度矩阵
return f, CV
problem = MyProblem()
# 构建算法
algorithm = ea.soea_SEGA_templet(problem,
                                ea.Population(Encoding='RI', NIND=n),
                                MAXGEN=50, # 最大进化代数。
                                logTras=1, # 表示每隔多少代记录一次日志信息, 0 表示不记录。
                                trappedValue=1e-6, # 单目标优化陷入停滞的判断阈值。
                                maxTrappedCount=10) # 进化停滞计数器最大上限值。
algorithm.mutOper.Pm = 0.9 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res_sega_bp = ea.optimize(algorithm,
                          verbose=True,
                          drawing=1,
                          outputMsg=True,
                          drawLog=True,
                          saveFlag=True)
print(res_sega_bp)

```

""" ===== 问题四 ===== """

```

"""
max x7
min x6

s. t.
0 < x1 < 100
0 < x2 < 2000
0 < x3 <= 3
x5 >= 85

model
predcit x6\x7: XGB
predict x3\x4\x5: Cat/Cat/XGB
"""
""" ===== NSGA-2 ===== """

model_cat_x3 = CatBoost(dataq2.iloc[:, 0:7], dataq2["x3"])
model_cat_x4 = CatBoost(dataq2.iloc[:, 0:7], dataq2["x4"])
model_xgb_x5 = XGBoost(dataq2.iloc[:, 0:7], dataq2["x5"])
model_xgb_x6 = XGBoost(dataq3.iloc[:, 0:13], dataq3["x6"])
model_xgb_x7 = XGBoost(dataq3.iloc[:, 0:13], dataq3["x7"])

n = 50
import geatpy as ea
class MyProblem(ea.Problem): # 继承 Problem 父类
    def __init__(self):
        name = 'MyProblem' # 初始化 name (函数名称, 可以随意设置)
        M = 2 # 优化目标个数
        maxormins = [1, 1] # 初始化 maxormins (目标最小最大化标记列表,
1: 最小化该目标; -1: 最大化该目标)
        Dim = 2 # 初始化 Dim (决策变量维数)
        varTypes = [0, 0] # 初始化 varTypes (决策变量的类型, 0: 实数; 1:
整数)
        lb = [0, 0] # 决策变量下界
        ub = [100, 2000] # 决策变量上界
        lbin = [0, 0] # 决策变量下边界 (0 表示不包含该变量的下边界, 1 表
示包含)
        ubin = [0, 0] # 决策变量上边界 (0 表示不包含该变量的上边界, 1 表
示包含)
        # 调用父类构造方法完成实例化
        ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
ub, lbin, ubin)

```

```

def evalVars(self, Vars): # 目标函数 s
    x1 = Vars[:, [0]]
    x2 = Vars[:, [1]]
    a = [x1, x2, x1**2, x2**2, x1**3, x2**3, x1*x2]
    d = pd.DataFrame()
    d["x1"] = a[0].reshape(1, -1)[0]
    d["x2"] = a[1].reshape(1, -1)[0]
    d["x1^2"] = a[2].reshape(1, -1)[0]
    d["x2^2"] = a[3].reshape(1, -1)[0]
    d["x1^3"] = a[4].reshape(1, -1)[0]
    d["x2^3"] = a[5].reshape(1, -1)[0]
    d["x1x2"] = a[6].reshape(1, -1)[0]
    x3 = model_cat_x3.predict(d)
    x4 = model_cat_x4.predict(d)
    x5 = model_xgb_x5.predict(d)
    b = [x3, x4, x5, x3**2, x4**2, x5**2, x3**3, x4**3, x5**3, x3*x4,
x4*x5, x3*x5, x3*x4*x5]
    df = pd.DataFrame()
    df["x3"] = b[0].reshape(1, -1)[0]
    df["x4"] = b[1].reshape(1, -1)[0]
    df["x5"] = b[2].reshape(1, -1)[0]
    df["x3^2"] = b[3].reshape(1, -1)[0]
    df["x4^2"] = b[4].reshape(1, -1)[0]
    df["x5^2"] = b[5].reshape(1, -1)[0]
    df["x3^3"] = b[6].reshape(1, -1)[0]
    df["x4^3"] = b[7].reshape(1, -1)[0]
    df["x5^3"] = b[8].reshape(1, -1)[0]
    df["x3x4"] = b[9].reshape(1, -1)[0]
    df["x4x5"] = b[10].reshape(1, -1)[0]
    df["x3x5"] = b[11].reshape(1, -1)[0]
    df["x3x4x5"] = b[12].reshape(1, -1)[0]
    f1 = -model_xgb_x7.predict(df).reshape(n, 1)
    f2 = model_xgb_x6.predict(df).reshape(n, 1)
    ObjV = np.hstack([f1, f2]) # 计算目标函数值矩阵
    CV = np.hstack([model_cat_x3.predict(d).reshape(n, 1) - 3,
                    85 - model_xgb_x5.predict(d).reshape(n, 1),
                    -model_cat_x3.predict(d).reshape(n, 1),
                    -model_cat_x4.predict(d).reshape(n, 1),
                    -model_xgb_x5.predict(d).reshape(n, 1)
                    ]) # 构建违反约束程度矩阵
    return ObjV, CV

# 实例化问题对象
problem = MyProblem()

```

```

# 构建算法
algorithm = ea.moea_NSGA2_templet(problem,
                                   ea.Population(Encoding='RI', NIND =
n),
                                   MAXGEN = 200, # 最大进化代数
                                   logTras = 0) # 表示每隔多少代记录一次

日志信息, 0 表示不记录。
algorithm.mutOper.Pm = 0.2 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res = ea.optimize(algorithm,
                  seed = 317,
                  verbose = True,
                  drawing = 2,
                  outputMsg = True,
                  drawLog = True,
                  saveFlag = True)

# 构建算法
algorithm = ea.moea_NSGA3_templet(problem,
                                   ea.Population(Encoding='RI', NIND =
n),
                                   MAXGEN = 100, # 最大进化代数
                                   logTras = 0) # 表示每隔多少代记录一次

日志信息, 0 表示不记录。
algorithm.mutOper.Pm = 0.2 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res = ea.optimize(algorithm,
                  seed = 317,
                  verbose = True,
                  drawing = 2,
                  outputMsg = True,
                  drawLog = True,
                  saveFlag = True)

x1 = [res["Vars"][0][0],
      res["Vars"][4][0],
      res["Vars"][6][0],
      res["Vars"][7][0],
      res["Vars"][10][0],
      res["Vars"][12][0],
      res["Vars"][13][0],

```

```

        res["Vars"][15][0],
        res["Vars"][17][0],
        res["Vars"][18][0],
        res["Vars"][19][0],
        res["Vars"][20][0],
        res["Vars"][21][0],
        res["Vars"][22][0],
        res["Vars"][23][0],
        res["Vars"][26][0],
        res["Vars"][27][0]]
x2 = [res["Vars"][0][1],
      res["Vars"][4][1],
      res["Vars"][6][1],
      res["Vars"][7][1],
      res["Vars"][10][1],
      res["Vars"][12][1],
      res["Vars"][13][1],
      res["Vars"][15][1],
      res["Vars"][17][1],
      res["Vars"][18][1],
      res["Vars"][19][1],
      res["Vars"][20][1],
      res["Vars"][21][1],
      res["Vars"][22][1],
      res["Vars"][23][1],
      res["Vars"][26][1],
      res["Vars"][27][1]]

m1 = np.mean(df["x1"])
s1 = np.std(df["x1"])
m2 = np.mean(df["x2"])
s2 = np.std(df["x2"])
m3 = np.mean(df["x3"])
s3 = np.std(df["x3"])
m4 = np.mean(df["x4"])
s4 = np.std(df["x4"])
m5 = np.mean(df["x5"])
s5 = np.std(df["x5"])
m6 = np.mean(df["x6"])
s6 = np.std(df["x6"])
m7 = np.mean(df["x7"])
s7 = np.std(df["x7"])

```

```

m8 = np.mean(df["x8"])
s8 = np.std(df["x8"])

df_std = pd.DataFrame()
df_std["x1"] = (df["x1"] - m1) / s1
df_std["x2"] = (df["x2"] - m2) / s2
df_std["x3"] = (df["x3"] - m3) / s3
df_std["x4"] = (df["x4"] - m4) / s4
df_std["x5"] = (df["x5"] - m5) / s5
df_std["x6"] = (df["x6"] - m6) / s6
df_std["x7"] = (df["x7"] - m7) / s7
df_std["x8"] = (df["x8"] - m8) / s8

n = 50
import geatpy as ea
class MyProblem(ea.Problem): # 继承 Problem 父类
    def __init__(self):
        name = 'MyProblem' # 初始化 name (函数名称, 可以随意设置)
        M = 2 # 优化目标个数
        maxormins = [1, 1] # 初始化 maxormins (目标最小最大化标记列表,
1: 最小化该目标; -1: 最大化该目标)
        Dim = 2 # 初始化 Dim (决策变量维数)
        varTypes = [0, 0] # 初始化 varTypes (决策变量的类型, 0: 实数; 1:
整数)
        lb = [0, 0] # 决策变量下界
        ub = [100, 2000] # 决策变量上界
        lbin = [0, 0] # 决策变量下边界 (0 表示不包含该变量的下边界, 1 表
示包含)
        ubin = [0, 0] # 决策变量上边界 (0 表示不包含该变量的上边界, 1 表
示包含)
        # 调用父类构造方法完成实例化
        ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
ub, lbin, ubin)

    def evalVars(self, Vars): # 目标函数
        x1 = Vars[:, [0]]
        x2 = Vars[:, [1]]
        x3 = 0.0221+0.8169*x1+0.5552*x2+0.1203*(x1**2)-
0.1427*(x2**2)+0.0135*x1*x2
        x4 = 0.3108+0.7301*x1+0.5219*x2-0.0953*(x1**2)-0.2198*(x2**2)-
0.2166*x1*x2
        x5 = 1.3625-0.4026*x1-0.3203*x2-0.8631*(x1**2)-0.5178*(x2**2)
        f1 = 1.8367*x3+0.6285*x5-0.2005*(x4**2)-0.2929*(x5**2)-
0.7508*(x3**3)-1.0645*x3*x5-0.2167*x3*x4*x5

```

```

        f2 = 0.2115-0.8175*x3+0.1441*x5-0.4866*(x3**2)-
0.1735*(x5**2)+0.1751*(x3**3)-0.1875*(x4**3)-0.5458*x3*x5
        ObjV = np.hstack([f1, f2]) # 计算目标函数值矩阵
        CV = np.hstack([x3 - 3,
                        85 - x4,
                        -x3,
                        -x4,
                        -x5
                        ]) # 构建违反约束程度矩阵
        return ObjV, CV

# 实例化问题对象
problem = MyProblem()
# 构建算法
algorithm = ea.moea_NSGA2_templet(problem,
                                ea.Population(Encoding='RI', NIND =
n),
                                MAXGEN = 50, # 最大进化代数
                                logTras = 0) # 表示每隔多少代记录一次
日志信息, 0 表示不记录。
algorithm.mutOper.Pm = 0.2 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res = ea.optimize(algorithm,
                  seed = 317,
                  verbose = True,
                  drawing = 1,
                  outputMsg = True,
                  drawLog = True,
                  saveFlag = False)

# 构建算法
algorithm = ea.moea_NSGA3_templet(problem,
                                ea.Population(Encoding='RI', NIND =
n),
                                MAXGEN = 100, # 最大进化代数
                                logTras = 0) # 表示每隔多少代记录一次
日志信息, 0 表示不记录。
algorithm.mutOper.Pm = 0.2 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res = ea.optimize(algorithm,
                  seed = 317,
                  verbose = True,
                  drawing = 2,

```

```

        outputMsg = True,
        drawLog = True,
        saveFlag = True)

n = 50
import geatpy as ea
class MyProblem(ea.Problem): # 继承 Problem 父类
    def __init__(self):
        name = 'MyProblem' # 初始化 name (函数名称, 可以随意设置)
        M = 2 # 优化目标个数
        maxormins = [1, 1] # 初始化 maxormins (目标最小最大化标记列表,
1: 最小化该目标; -1: 最大化该目标)
        Dim = 2 # 初始化 Dim (决策变量维数)
        varTypes = [0, 0] # 初始化 varTypes (决策变量的类型, 0: 实数; 1:
整数)
        lb = [0, 0] # 决策变量下界
        ub = [100, 2000] # 决策变量上界
        lbin = [0, 0] # 决策变量下边界 (0 表示不包含该变量的下边界, 1 表
示包含)
        ubin = [0, 0] # 决策变量上边界 (0 表示不包含该变量的上边界, 1 表
示包含)
        # 调用父类构造方法完成实例化
        ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
ub, lbin, ubin)

    def evalVars(self, Vars): # 目标函数 s
        x1 = Vars[:, [0]]
        x2 = Vars[:, [1]]
        a = [x1, x2, x1**2, x2**2, x1**3, x2**3, x1*x2]
        d = pd.DataFrame()
        d["x1"] = a[0].reshape(1, -1)[0]
        d["x2"] = a[1].reshape(1, -1)[0]
        d["x1^2"] = a[2].reshape(1, -1)[0]
        d["x2^2"] = a[3].reshape(1, -1)[0]
        d["x1^3"] = a[4].reshape(1, -1)[0]
        d["x2^3"] = a[5].reshape(1, -1)[0]
        d["x1x2"] = a[6].reshape(1, -1)[0]
        d = (d - meanx3) / stdx3
        x3 = model_cat_x3.predict(d)
        x4 = model_cat_x4.predict(d)
        x5 = model_xgb_x5.predict(d)
        b = [x3, x4, x5, x3**2, x4**2, x5**2, x3**3, x4**3, x5**3, x3*x4,

```

```

x4*x5, x3*x5, x3*x4*x5]
    df = pd.DataFrame()
    df["x3"] = b[0].reshape(1, -1)[0]
    df["x4"] = b[1].reshape(1, -1)[0]
    df["x5"] = b[2].reshape(1, -1)[0]
    df["x3^2"] = b[3].reshape(1, -1)[0]
    df["x4^2"] = b[4].reshape(1, -1)[0]
    df["x5^2"] = b[5].reshape(1, -1)[0]
    df["x3^3"] = b[6].reshape(1, -1)[0]
    df["x4^3"] = b[7].reshape(1, -1)[0]
    df["x5^3"] = b[8].reshape(1, -1)[0]
    df["x3x4"] = b[9].reshape(1, -1)[0]
    df["x4x5"] = b[10].reshape(1, -1)[0]
    df["x3x5"] = b[11].reshape(1, -1)[0]
    df["x3x4x5"] = b[12].reshape(1, -1)[0]
    df = (df - meanx6) / stdx6
    f1 = -model_xgb_x7.predict(df).reshape(n, 1)
    f2 = model_xgb_x6.predict(df).reshape(n, 1)
    ObjV = np.hstack([f1, f2]) # 计算目标函数值矩阵
    CV = np.hstack([model_cat_x3.predict(d).reshape(n, 1) - 3,
                    85 - model_xgb_x5.predict(d).reshape(n, 1),
                    -model_cat_x3.predict(d).reshape(n, 1),
                    -model_cat_x4.predict(d).reshape(n, 1),
                    -model_xgb_x5.predict(d).reshape(n, 1)
                    ]) # 构建违反约束程度矩阵
    return ObjV, CV

# 实例化问题对象
problem = MyProblem()
# 构建算法
algorithm = ea.moea_NSGA2_templet(problem,
                                   ea.Population(Encoding='RI', NIND =
n),
                                   MAXGEN = 200, # 最大进化代数
                                   logTras = 0) # 表示每隔多少代记录一次
日志信息, 0 表示不记录。
algorithm.mutOper.Pm = 0.2 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res = ea.optimize(algorithm,
                  seed = 317,
                  verbose = True,
                  drawing = 2,
                  outputMsg = True,

```

```

        drawLog = True,
        saveFlag = True)

# 构建算法
algorithm = ea.moea_NSGA3_templet(problem,
                                   ea.Population(Encoding='RI', NIND =
n),
                                   MAXGEN = 100, # 最大进化代数
                                   logTras = 0) # 表示每隔多少代记录一次
日志信息, 0 表示不记录。
algorithm.mutOper.Pm = 0.2 # 修改变异算子的变异概率
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
# 求解
res = ea.optimize(algorithm,
                  seed = 317,
                  verbose = True,
                  drawing = 2,
                  outputMsg = True,
                  drawLog = True,
                  saveFlag = True)

n = 100
class MyProblem(ea.Problem): # 继承 Problem 父类
    def __init__(self):
        name = 'MyProblem' # 初始化 name (函数名称, 可以随意设置)
        M = 1 # 初始化 M (目标维数)
        maxormins = [-1] # 初始化 maxormins (目标最小最大化标记列表, 1:
最小化该目标; -1: 最大化该目标)
        Dim = 2 # 初始化 Dim (决策变量维数)
        varTypes = [0] * Dim # 初始化 varTypes (决策变量的类型, 元素为 0
表示对应的变量是连续的; 1 表示是离散的)
        lb = [0] * Dim # 决策变量下界
        ub = [10**5] * Dim # 决策变量上界
        lbin = [0] * Dim # 决策变量下边界 (0 表示不包含该变量的下边界,
1 表示包含)
        ubin = [0] * Dim # 决策变量上边界 (0 表示不包含该变量的上边界,
1 表示包含)
        # 调用父类构造方法完成实例化
        ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
ub, lbin, ubin)
    def evalVars(self, Vars): # 目标函数
        x1 = Vars[:, [0]]

```

```

x2 = Vars[:, [1]]
a = [x1, x2, x1**2, x2**2, x1**3, x2**3, x1*x2]
d = pd.DataFrame()
d["x1"] = a[0].reshape(1, -1)[0]
d["x2"] = a[1].reshape(1, -1)[0]
d["x1^2"] = a[2].reshape(1, -1)[0]
d["x2^2"] = a[3].reshape(1, -1)[0]
d["x1^3"] = a[4].reshape(1, -1)[0]
d["x2^3"] = a[5].reshape(1, -1)[0]
d["x1x2"] = a[6].reshape(1, -1)[0]
d = (d - meanx3) / stdx3
x3 = model_bp_x3.predict(d)
x4 = model_bp_x4.predict(d)
x5 = model_bp_x5.predict(d)
b = [x3, x4, x5, x3**2, x4**2, x5**2, x3**3, x4**3, x5**3, x3*x4,
x4*x5, x3*x5, x3*x4*x5]
df = pd.DataFrame()
df["x3"] = b[0].reshape(1, -1)[0]
df["x4"] = b[1].reshape(1, -1)[0]
df["x5"] = b[2].reshape(1, -1)[0]
df["x3^2"] = b[3].reshape(1, -1)[0]
df["x4^2"] = b[4].reshape(1, -1)[0]
df["x5^2"] = b[5].reshape(1, -1)[0]
df["x3^3"] = b[6].reshape(1, -1)[0]
df["x4^3"] = b[7].reshape(1, -1)[0]
df["x5^3"] = b[8].reshape(1, -1)[0]
df["x3x4"] = b[9].reshape(1, -1)[0]
df["x4x5"] = b[10].reshape(1, -1)[0]
df["x3x5"] = b[11].reshape(1, -1)[0]
df["x3x4x5"] = b[12].reshape(1, -1)[0]
df = (df - meanx6) / stdx6
f = model_bp_x7.predict(df)
CV = np.hstack([-model_bp_x3.predict(d),
                -model_bp_x4.predict(d),
                -model_bp_x5.predict(d),
                model_bp_x7.predict(df) - 100
                ]) # 构建违反约束程度矩阵
return f, CV
problem = MyProblem()
# 构建算法
algorithm = ea.soea_SEGA_templet(problem,
                                ea.Population(Encoding='RI', NIND=n),
                                MAXGEN=50, # 最大进化代数。
                                logTras=1, # 表示每隔多少代记录一次日

```

志信息，0 表示不记录。

```
trappedValue=1e-6, # 单目标优化陷入停  
滞的判断阈值。
```

```
maxTrappedCount=10) # 进化停滞计数器  
最大上限值。
```

```
algorithm.mutOper.Pm = 0.9 # 修改变异算子的变异概率
```

```
algorithm.recOper.XOVR = 0.9 # 修改交叉算子的交叉概率
```

```
# 求解
```

```
res_sega_bp = ea.optimize(algorithm,  
                           verbose=True,  
                           drawing=1,  
                           outputMsg=True,  
                           drawLog=True,  
                           saveFlag=True)
```

```
print(res_sega_bp)
```

R

```
mydata = read.csv(file="D:/3、竞赛/2022.08.04 “华数杯” 全国大学生数学建模  
竞赛/data3.csv",
```

```
                  header = TRUE, sep = ",", encoding="UTF-8")
```

```
data = read.csv(file="D:/3、竞赛/2022.08.04 “华数杯” 全国大学生数学建模  
竞赛/data3.csv",
```

```
                header = TRUE, sep = ",", encoding="UTF-8")
```

```
library(car)
```

```
scatterplotMatrix(mydata[, 1:5], spread = FALSE, smoother.args =  
list(lty=2),
```

```
main = "散点图矩阵")
```

```
m1 = mean(mydata$x1)
```

```
m2 = mean(mydata$x2)
```

```
m3 = mean(mydata$x3)
```

```
m4 = mean(mydata$x4)
```

```
m5 = mean(mydata$x5)
```

```
m6 = mean(mydata$x6)
```

```
m7 = mean(mydata$x7)
```

```
m8 = mean(mydata$x8)
```

```

s1 = sd(mydata$x1)
s2 = sd(mydata$x2)
s3 = sd(mydata$x3)
s4 = sd(mydata$x4)
s5 = sd(mydata$x5)
s6 = sd(mydata$x6)
s7 = sd(mydata$x7)
s8 = sd(mydata$x8)

data$x1 = (mydata$x1 - m1) / s1
data$x2 = (mydata$x2 - m2) / s2
data$x3 = (mydata$x3 - m3) / s3
data$x4 = (mydata$x4 - m4) / s4
data$x5 = (mydata$x5 - m5) / s5
data$x6 = (mydata$x6 - m6) / s6
data$x7 = (mydata$x7 - m7) / s7
data$x8 = (mydata$x8 - m8) / s8

lm_x1 = lm("x1 ~ x3 + x4 + x3:x4 + x3:x5 +x3:x4:x5", data = data)
summary(lm_x1)
AIC(lm_x1)
plot(lm_x1)

lm_x2 = lm("x2 ~ x3-1", data = data)
summary(lm_x2)
AIC(lm_x2)
plot(lm_x2)

lm_x3 = lm("x3 ~ x1 + x2 + x1:x2 + I(x1^2) + I(x2^2)", data = data)
summary(lm_x3)
AIC(lm_x3)
plot(lm_x3)

lm_x4 = lm("x4 ~ x1 + x2 + x1:x2 + I(x1^2) + I(x2^2)", data = data)
summary(lm_x4)
AIC(lm_x4)
plot(lm_x4)

lm_x5 = lm("x5 ~ x1 + x2 + I(x1^2) + I(x2^2)", data = data)
summary(lm_x5)
AIC(lm_x5)
plot(lm_x5)

```

```

# ===== Q3 =====
library(car)
scatterplotMatrix(mydata[, 3:8], spread = FALSE, smoother.args =
list(lty=2),
                    main = "散点图矩阵")

lm_x3 = lm("x3 ~ x7+x8+x6:x7+x7:x8+x6:x7:x8+I(x6^2)+I(x7^2)+I(x8^2)+I(x6^3)+I(x8^3)",
data = data)
summary(lm_x3)
AIC(lm_x3)
plot(lm_x3)

lm_x4 = lm("x4 ~ x7+x8+x6:x8+x7:x8+x6:x7:x8+I(x6^2)+I(x7^2)+I(x6^3)",
data = data)
summary(lm_x4)
AIC(lm_x4)
plot(lm_x4)

lm_x5 = lm("x5 ~ x6+x8+x6:x8+x7:x8+x6:x7:x8+I(x6^2)+I(x8^2)+I(x6^3)+I(x7^3)", data =
data)
summary(lm_x5)
AIC(lm_x5)
plot(lm_x5)

lm_x6 = lm("x6 ~ x3+x5+x3:x5+I(x3^2)+I(x5^2)+I(x3^3)+I(x4^3)", data =
data)
summary(lm_x6)
AIC(lm_x6)
plot(lm_x6)

lm_x7 = lm("x7 ~ x3+x5+x3:x5+x3:x4:x5+I(x4^2)+I(x5^2)+I(x3^3)-1", data =
data)
summary(lm_x7)
AIC(lm_x7)
plot(lm_x7)

lm_x8 = lm("x8 ~ x3+x3:x5+x3:x4+I(x3^3)-1", data = data)
summary(lm_x8)
AIC(lm_x8)
plot(lm_x8)

lm_x3 = lm("x3 ~ x4 + x5 + x4:x5+ I(x4^3)", data = data)

```

```

summary(lm_x3)
AIC(lm_x3)
plot(lm_x3)

lm_x4 = lm("x4 ~ x3 + I(x3^2)", data = data)
summary(lm_x4)
AIC(lm_x4)
plot(lm_x4)

lm_x5 = lm("x5 ~ x3 + x3:x4 + I(x4^2)", data = data)
summary(lm_x5)
AIC(lm_x5)
plot(lm_x4)

lm_x6 = lm("x6 ~ x7 + x7:x8 + I(x7^2)+ I(x8^2) + I(x8^3)", data = data)
summary(lm_x6)
AIC(lm_x6)
plot(lm_x6)

lm_x7 = lm("x7 ~ x6 + x8 + I(x6^2)+ I(x8^2)", data = data)
summary(lm_x7)
AIC(lm_x7)

lm_x8 = lm("x8 ~ x6 + x7 + I(x6^2)+ I(x7^2)", data = data)
summary(lm_x8)
AIC(lm_x8)
plot(lm_x4)

```